# FinVest Main Files Code Export

Main backend project files only: Python, templates, CSS, SQL, JS, and requirements.

### backend\app__init__.py

```
from time import perf_counter
from uuid import uuid4

from flask import Flask, g, session

from extensions.db import db
from extensions.mongo import init_mongo
from app.routes.auth_routes import auth
from app.routes.budget_routes import budgets
from app.routes.category_routes import categories
from app.routes.dashboard_routes import dashboard
from app.routes.goal_routes import goals
from app.routes.notification_routes import notifications
from app.routes.report_routes import reports
from app.routes.transaction_routes import transactions

# Import models so SQLAlchemy sees all tables before create_all runs.
from app.models.account_model import Account
from app.models.active_session_model import ActiveSession
from app.models.activity_log_model import ActivityLog
from app.models.budget_model import Budget
from app.models.category_model import Category
from app.models.goal_model import Goal
from app.models.notification_model import Notification
from app.models.recurring_transaction_model import RecurringTransaction
from app.models.transaction_model import Transaction
from app.models.user_model import User
from app.models.user_settings_model import UserSettings
from app.services.active_session_service import touch_active_session
from app.services.automation_service import run_user_automations
from app.services.currency_service import EXCHANGE_RATES_INR, convert_from_inr, currency_symbol
from app.services.i18n_service import LANGUAGES, normalize_language
from app.services.notification_service import unread_notification_count
from app.services.schema_service import ensure_automation_schema


def create_app():
    app = Flask(__name__)
    app.config["SECRET_KEY"] = "finvest_secret_123"
    app.config.from_object("config")

    db.init_app(app)
    init_mongo(app)
    app.register_blueprint(auth)
    app.register_blueprint(dashboard)
    app.register_blueprint(budgets)
    app.register_blueprint(goals)
    app.register_blueprint(categories)
    app.register_blueprint(notifications)
    app.register_blueprint(reports)
    app.register_blueprint(transactions)

    @app.before_request
    def run_due_finance_jobs():
        g.request_started_at = perf_counter()
        try:
            ensure_automation_schema()
            if session.get("user_id"):
                if not session.get("session_token"):
                    session["session_token"] = uuid4().hex
                touch_active_session(session.get("user_id"), session.get("session_token"))
                run_user_automations(session["user_id"])
                db.session.commit()
        except Exception:
            db.session.rollback()

    @app.after_request
```

```
    def capture_response_time(response):
        started_at = getattr(g, "request_started_at", None)
        if started_at is not None:
            app.config["LAST_RESPONSE_MS"] = round((perf_counter() - started_at) * 1000, 2)
        return response

    @app.context_processor
    def inject_layout_data():
        count = 0
        settings = None
        currency = "INR"
        language = "en"
        if session.get("user_id"):
            try:
                count = unread_notification_count(session["user_id"])
                settings = UserSettings.query.filter_by(user_id=session["user_id"]).first()
                if settings:
                    currency = settings.currency or "INR"
                    language = normalize_language(settings.language or settings.locale)
            except Exception:
                count = 0
        return {
            "unread_notification_count": count,
            "display_currency": currency,
            "display_currency_symbol": currency_symbol(currency),
            "display_currency_rates": EXCHANGE_RATES_INR,
            "display_language": language,
            "available_languages": LANGUAGES,
        }

    @app.template_filter("money")
    def money_filter(amount):
        currency = "INR"
        if session.get("user_id"):
            settings = UserSettings.query.filter_by(user_id=session["user_id"]).first()
            if settings:
                currency = settings.currency or "INR"
        return f"{currency_symbol(currency)} {convert_from_inr(amount, currency):.2f}"

    return app
```

## backend\app\config.py

```
SQLALCHEMY_DATABASE_URI = "mysql+pymysql://root:root@localhost/finvestdb"
SQLALCHEMY_TRACK_MODIFICATIONS = False
```

## backend\app\db_scripts\table_scripts.sql

```
CREATE TABLE IF NOT EXISTS users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    role ENUM('user', 'admin') DEFAULT 'user',
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    password_hash VARCHAR(255) NOT NULL,
    security_answers TEXT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS active_sessions (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    session_token VARCHAR(120) NOT NULL UNIQUE,
    last_seen_at DATETIME NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS user_settings (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    currency VARCHAR(10) DEFAULT 'INR',
    locale VARCHAR(20) DEFAULT 'en-IN',
    language VARCHAR(10) DEFAULT 'en',
    month_start_day INT DEFAULT 1,
    monthly_salary DECIMAL(12, 2) DEFAULT 0,
    monthly_salary_last_added DATE NULL,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS accounts (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    account_name VARCHAR(100) NOT NULL,
    type ENUM('Cash', 'Bank', 'UPI') NOT NULL,
    balance DECIMAL(12, 2) DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS categories (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    name VARCHAR(100) NOT NULL,
    type ENUM('income', 'expense') NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS transactions (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    account_id INT NOT NULL,
    category_id INT NOT NULL,
    type ENUM('income', 'expense') NOT NULL,
    amount DECIMAL(12, 2) NOT NULL,
    date DATE NOT NULL,
    description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (account_id) REFERENCES accounts(id) ON DELETE CASCADE,
    FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS budgets (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    category_id INT NOT NULL,
    limit_amount DECIMAL(12, 2) NOT NULL,
    month INT NOT NULL,
    year INT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
```

```
    FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS recurring_transactions (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    account_id INT NOT NULL,
    category_id INT NOT NULL,
    type ENUM('income', 'expense') NOT NULL DEFAULT 'expense',
    amount DECIMAL(12, 2) NOT NULL,
    description VARCHAR(255),
    frequency ENUM('daily', 'monthly', 'yearly') NOT NULL,
    start_date DATE NOT NULL,
    next_run_date DATE NOT NULL,
    last_run_date DATE NULL,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (account_id) REFERENCES accounts(id) ON DELETE CASCADE,
    FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS goals (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    goal_name VARCHAR(150) NOT NULL,
    target_amount DECIMAL(12, 2) NOT NULL,
    current_amount DECIMAL(12, 2) DEFAULT 0,
    deadline DATE,
    status VARCHAR(50),
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS activity_logs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NULL,
    user_name VARCHAR(100) NOT NULL,
    user_email VARCHAR(150) NOT NULL,
    action VARCHAR(100) NOT NULL,
    details VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS notifications (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    title VARCHAR(120) NOT NULL,
    message VARCHAR(255) NOT NULL,
    notification_type VARCHAR(50) NOT NULL,
    reference_key VARCHAR(120),
    is_read BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

## backend\app\jobs\budget_jobs.py

```
from app.services.automation_service import run_all_automations


def run_due_budget_jobs(today=None):
    return run_all_automations(today=today)
```

## backend\app\jobs\scheduler.py

```
from app import create_app
from app.jobs.budget_jobs import run_due_budget_jobs


def run_once():
    app = create_app()
    with app.app_context():
        return run_due_budget_jobs()


if __name__ == "__main__":
    created = run_once()
    print(f"Created {created} scheduled transaction(s).")
```

## backend\app\main.py

```
from app import create_app
from app.services.admin_service import seed_default_admin
from app.services.schema_service import ensure_automation_schema
from extensions.db import db

app = create_app()

with app.app_context():
    db.create_all()
    ensure_automation_schema()
    seed_default_admin()

if __name__ == "__main__":
    app.run(debug=True)
```

## backend\app\models\account_model.py

```
from extensions.db import db


class Account(db.Model):
    __tablename__ = "accounts"

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(
        db.Integer, db.ForeignKey("users.id", ondelete="CASCADE"), nullable=False
    )
    account_name = db.Column(db.String(100), nullable=False)
    type = db.Column(db.Enum("Cash", "Bank", "UPI"), nullable=False)
    balance = db.Column(db.Numeric(12, 2), default=0)
    created_at = db.Column(
        db.TIMESTAMP, server_default=db.func.current_timestamp()
    )

    transactions = db.relationship("Transaction", backref="account", lazy=True)


def ensure_default_account(user_id):
    account = Account.query.filter_by(user_id=user_id).first()
    if account:
        return account

    account = Account(
        user_id=user_id,
        account_name="Cash Wallet",
        type="Cash",
        balance=0,
    )
    db.session.add(account)
    return account
```

## backend\app\models\active_session_model.py

```
from extensions.db import db


class ActiveSession(db.Model):
    __tablename__ = "active_sessions"

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(
        db.Integer, db.ForeignKey("users.id", ondelete="CASCADE"), nullable=False
    )
    session_token = db.Column(db.String(120), nullable=False, unique=True)
    last_seen_at = db.Column(db.DateTime, nullable=False)
    created_at = db.Column(
        db.TIMESTAMP, server_default=db.func.current_timestamp()
    )
```

## backend\app\models\activity_log_model.py

```
from extensions.db import db


class ActivityLog(db.Model):
    __tablename__ = "activity_logs"

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, nullable=True)
    user_name = db.Column(db.String(100), nullable=False)
    user_email = db.Column(db.String(150), nullable=False)
    action = db.Column(db.String(100), nullable=False)
    details = db.Column(db.String(255))
    created_at = db.Column(
        db.TIMESTAMP, server_default=db.func.current_timestamp()
    )
```

## backend\app\models\budget_model.py

```
from extensions.db import db


class Budget(db.Model):
    __tablename__ = "budgets"

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(
        db.Integer, db.ForeignKey("users.id", ondelete="CASCADE"), nullable=False
    )
    category_id = db.Column(
        db.Integer, db.ForeignKey("categories.id", ondelete="CASCADE"), nullable=False
    )
    limit_amount = db.Column(db.Numeric(12, 2), nullable=False)
    month = db.Column(db.Integer, nullable=False)
    year = db.Column(db.Integer, nullable=False)

    category = db.relationship("Category", backref="budgets", lazy=True)
```

## backend\app\models\category_model.py

```
from extensions.db import db


DEFAULT_CATEGORIES = [
    ("Salary", "income"),
    ("Freelance", "income"),
    ("Bonus", "income"),
    ("Food", "expense"),
    ("Rent", "expense"),
    ("Clothes", "expense"),
    ("Electric Bills", "expense"),
    ("Transport", "expense"),
    ("Shopping", "expense"),
    ("Health", "expense"),
]


class Category(db.Model):
    __tablename__ = "categories"

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(
        db.Integer, db.ForeignKey("users.id", ondelete="CASCADE"), nullable=False
    )
    name = db.Column(db.String(100), nullable=False)
    type = db.Column(db.Enum("income", "expense"), nullable=False)

    transactions = db.relationship("Transaction", backref="category", lazy=True)


def ensure_default_categories(user_id):
    existing_count = Category.query.filter_by(user_id=user_id).count()
    if existing_count:
        return

    for name, category_type in DEFAULT_CATEGORIES:
        db.session.add(Category(user_id=user_id, name=name, type=category_type))
```

## backend\app\models\goal_model.py

```
from extensions.db import db


class Goal(db.Model):
    __tablename__ = "goals"

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(
        db.Integer, db.ForeignKey("users.id", ondelete="CASCADE"), nullable=False
    )
    goal_name = db.Column(db.String(150), nullable=False)
    target_amount = db.Column(db.Numeric(12, 2), nullable=False)
    current_amount = db.Column(db.Numeric(12, 2), default=0)
    deadline = db.Column(db.Date)
    status = db.Column(db.String(50), default="In Progress")
```

## backend\app\models\notification_model.py

```
from extensions.db import db


class Notification(db.Model):
    __tablename__ = "notifications"

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(
        db.Integer, db.ForeignKey("users.id", ondelete="CASCADE"), nullable=False
    )
    title = db.Column(db.String(120), nullable=False)
    message = db.Column(db.String(255), nullable=False)
    notification_type = db.Column(db.String(50), nullable=False)
    reference_key = db.Column(db.String(120))
    is_read = db.Column(db.Boolean, nullable=False, server_default=db.text("0"))
    created_at = db.Column(
        db.TIMESTAMP, server_default=db.func.current_timestamp()
    )
```

## backend\app\models\recurring_transaction_model.py

```
from extensions.db import db


class RecurringTransaction(db.Model):
    __tablename__ = "recurring_transactions"

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(
        db.Integer, db.ForeignKey("users.id", ondelete="CASCADE"), nullable=False
    )
    account_id = db.Column(
        db.Integer, db.ForeignKey("accounts.id", ondelete="CASCADE"), nullable=False
    )
    category_id = db.Column(
        db.Integer, db.ForeignKey("categories.id", ondelete="CASCADE"), nullable=False
    )
    type = db.Column(db.Enum("income", "expense"), nullable=False, default="expense")
    amount = db.Column(db.Numeric(12, 2), nullable=False)
    description = db.Column(db.String(255))
    frequency = db.Column(db.Enum("daily", "monthly", "yearly"), nullable=False)
    start_date = db.Column(db.Date, nullable=False)
    next_run_date = db.Column(db.Date, nullable=False)
    last_run_date = db.Column(db.Date)
    is_active = db.Column(db.Boolean, nullable=False, server_default=db.text("1"))
    created_at = db.Column(
        db.TIMESTAMP, server_default=db.func.current_timestamp()
    )

    account = db.relationship("Account", backref="recurring_transactions", lazy=True)
    category = db.relationship("Category", backref="recurring_transactions", lazy=True)
```

## backend\app\models\transaction_model.py

```
from extensions.db import db


class Transaction(db.Model):
    __tablename__ = "transactions"

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(
        db.Integer, db.ForeignKey("users.id", ondelete="CASCADE"), nullable=False
    )
    account_id = db.Column(
        db.Integer, db.ForeignKey("accounts.id", ondelete="CASCADE"), nullable=False
    )
    category_id = db.Column(
        db.Integer, db.ForeignKey("categories.id", ondelete="CASCADE"), nullable=False
    )
    type = db.Column(db.Enum("income", "expense"), nullable=False)
    amount = db.Column(db.Numeric(12, 2), nullable=False)
    date = db.Column(db.Date, nullable=False)
    description = db.Column(db.Text)
    created_at = db.Column(
        db.TIMESTAMP, server_default=db.func.current_timestamp()
    )
```

## backend\app\models\user_model.py

```
from extensions.db import db


class User(db.Model):
    __tablename__ = "users"

    id = db.Column(db.Integer, primary_key=True, autoincrement=True)
    name = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(150), unique=True, nullable=False)
    role = db.Column(db.Enum("user", "admin"), server_default="user")
    is_active = db.Column(db.Boolean, nullable=False, server_default=db.text("1"))
    password_hash = db.Column(db.String(255), nullable=False)
    security_answers = db.Column(db.Text)
    created_at = db.Column(
        db.TIMESTAMP, server_default=db.func.current_timestamp()
    )
```

## backend\app\models\user_settings_model.py

```
from extensions.db import db


class UserSettings(db.Model):
    __tablename__ = "user_settings"

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(
        db.Integer, db.ForeignKey("users.id", ondelete="CASCADE"), nullable=False
    )
    currency = db.Column(db.String(10), default="INR")
    locale = db.Column(db.String(20), default="en-IN")
    language = db.Column(db.String(10), default="en")
    month_start_day = db.Column(db.Integer, default=1)
    monthly_salary = db.Column(db.Numeric(12, 2), default=0)
    monthly_salary_last_added = db.Column(db.Date)


def get_or_create_user_settings(user_id):
    settings = UserSettings.query.filter_by(user_id=user_id).first()
    if settings:
        return settings

    settings = UserSettings(user_id=user_id)
    db.session.add(settings)
    return settings
```

## backend\app\routes\admin_routes.py

```
[Empty file]
```

## backend\app\routes\auth_routes.py

```
from datetime import datetime, timedelta
from time import perf_counter
import json
import re
from uuid import uuid4

from flask import Blueprint, flash, redirect, render_template, request, session, url_for
from werkzeug.security import check_password_hash, generate_password_hash

from app.models.account_model import ensure_default_account
from app.models.category_model import ensure_default_categories
from app.models.transaction_model import Transaction
from app.models.user_model import User
from app.models.user_settings_model import get_or_create_user_settings
from app.services.active_session_service import end_active_session, start_active_session
from app.utils.activity_logger import log_activity
from app.utils.decorators import ADMIN_EMAIL
from extensions.db import db
from extensions.mongo import get_collection

auth = Blueprint("auth", __name__)

USERNAME_PATTERN = re.compile(r"^[A-Za-z0-9_.]+$")
PASSWORD_PATTERN = re.compile(
    r"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*()_\-+=\[\]{}|\\:;\"'<>,.?/]).{8,}$"
)

SECURITY_QUESTIONS = [
    "What was the name of your first school?",
    "What is your mother's maiden name?",
    "What city were you born in?",
    "What was your first pet's name?",
    "What is your favorite childhood book?",
]


def _normalize_answer(answer):
    return " ".join((answer or "").strip().lower().split())


def _security_payload_from_form():
    payload = []
    for index, question in enumerate(SECURITY_QUESTIONS):
        answer = _normalize_answer(request.form.get(f"security_answer_{index}", ""))
        if not answer:
            return None
        payload.append(
            {
                "question": question,
                "answer_hash": generate_password_hash(answer),
            }
        )
    return json.dumps(payload)


@auth.route("/")
def onboarding():
    stats = {
        "users": 0,
        "transactions": 0,
        "logs": 0,
        "db_ms": 0,
    }
    try:
        started_at = perf_counter()
        stats["users"] = User.query.filter_by(is_active=True).count()
        stats["transactions"] = Transaction.query.count()
        stats["db_ms"] = round((perf_counter() - started_at) * 1000, 2)
        activity_collection = get_collection("activity_logs")
        if activity_collection is not None:
            stats["logs"] = activity_collection.count_documents({})
    except Exception:
        db.session.rollback()
```

```
    return render_template("auth/onboarding.html", stats=stats)


@auth.route("/auth")
def auth_page():
    mode = request.args.get("mode", "login")
    return render_template(
        "auth/auth.html",
        mode=mode,
        security_questions=SECURITY_QUESTIONS,
        reset_email=None,
        reset_questions=[],
    )


@auth.route("/register", methods=["POST"])
def register():
    username = request.form.get("username")
    email = request.form.get("email")
    password = request.form.get("password")
    confirm_password = request.form.get("confirm_password")
    security_answers = _security_payload_from_form()

    if not username or not email or not password or not security_answers:
        flash("Please fill all fields")
        return redirect(url_for("auth.auth_page", mode="signup"))

    username = username.strip()
    email = email.strip().lower()

    if not USERNAME_PATTERN.match(username):
        flash("Username can use only letters, numbers, underscore and dot")
        return redirect(url_for("auth.auth_page", mode="signup"))

    if not PASSWORD_PATTERN.match(password):
        flash("Password must be 8+ chars with uppercase, lowercase, number and special symbol")
        return redirect(url_for("auth.auth_page", mode="signup"))

    if password != confirm_password:
        flash("Passwords do not match")
        return redirect(url_for("auth.auth_page", mode="signup"))

    existing_user = User.query.filter_by(email=email).first()
    if existing_user:
        flash("Email already registered")
        return redirect(url_for("auth.auth_page", mode="signup"))

    if email.lower() == ADMIN_EMAIL:
        flash("This email is reserved for system admin")
        return redirect(url_for("auth.auth_page", mode="signup"))

    hashed_password = generate_password_hash(password)
    new_user = User(
        name=username,
        email=email,
        password_hash=hashed_password,
        security_answers=security_answers,
        role="user",
    )

    db.session.add(new_user)
    db.session.commit()

    get_or_create_user_settings(new_user.id)
    ensure_default_categories(new_user.id)
    ensure_default_account(new_user.id)
    db.session.commit()

    session.pop("login_attempts", None)
    session.pop("last_attempt_time", None)

    flash("Account created successfully! Please login")
    return redirect(url_for("auth.auth_page", mode="login"))


@auth.route("/login", methods=["POST"])
```

```
def login():
    email = request.form.get("email")
    password = request.form.get("password")

    if email:
        email = email.strip().lower()

    if "login_attempts" not in session:
        session["login_attempts"] = 0
        session["last_attempt_time"] = str(datetime.utcnow())

    last_time = datetime.fromisoformat(session["last_attempt_time"])
    if datetime.utcnow() - last_time > timedelta(minutes=2):
        session["login_attempts"] = 0

    if session["login_attempts"] >= 3:
        flash("Too many failed attempts. Try again after 2 minutes")
        return redirect(url_for("auth.auth_page", mode="login"))

    user = User.query.filter_by(email=email).first()
    if not user:
        session["login_attempts"] += 1
        session["last_attempt_time"] = str(datetime.utcnow())
        flash(f"User not found. Attempts left: {3 - session['login_attempts']}")
        return redirect(url_for("auth.auth_page", mode="login"))

    if not check_password_hash(user.password_hash, password):
        session["login_attempts"] += 1
        session["last_attempt_time"] = str(datetime.utcnow())
        flash(f"Wrong Password! Attempts left: {3 - session['login_attempts']}")
        return redirect(url_for("auth.auth_page", mode="login"))

    if not user.is_active:
        flash("Your account is blocked. Please contact admin")
        return redirect(url_for("auth.auth_page", mode="login"))

    session["user_id"] = user.id
    session["username"] = user.name
    session["email"] = user.email
    session["role"] = user.role
    session["session_token"] = uuid4().hex
    session.pop("login_attempts", None)
    session.pop("last_attempt_time", None)
    start_active_session(user.id, session["session_token"])
    log_activity(
        "login",
        "Logged into the system",
        user_id=user.id,
        user_name=user.name,
        user_email=user.email,
    )
    db.session.commit()

    if user.role == "admin":
        return redirect(url_for("dashboard.admin_dashboard"))

    return redirect(url_for("dashboard.home"))


@auth.route("/forgot-password/questions", methods=["POST"])
def forgot_password_questions():
    email = request.form.get("reset_email", "").strip().lower()
    user = User.query.filter_by(email=email).first()
    if not user or not user.security_answers:
        flash("Recovery questions are not available for this email")
        return redirect(url_for("auth.auth_page", mode="login"))

    questions = json.loads(user.security_answers)
    selected_indexes = [0, 3] if len(questions) >= 4 else [0, 1]
    reset_questions = [
        {"index": index, "question": questions[index]["question"]}
        for index in selected_indexes
    ]
    return render_template(
        "auth/auth.html",
        mode="login",
```

```
        security_questions=SECURITY_QUESTIONS,
        reset_email=email,
        reset_questions=reset_questions,
    )


@auth.route("/forgot-password/reset", methods=["POST"])
def forgot_password_reset():
    email = request.form.get("reset_email", "").strip().lower()
    user = User.query.filter_by(email=email).first()
    if not user or not user.security_answers:
        flash("Recovery questions are not available for this email")
        return redirect(url_for("auth.auth_page", mode="login"))

    answers = json.loads(user.security_answers)
    for key, value in request.form.items():
        if not key.startswith("security_answer_"):
            continue
        index = int(key.replace("security_answer_", ""))
        expected = answers[index]["answer_hash"]
        if not check_password_hash(expected, _normalize_answer(value)):
            flash("Security answers did not match")
            return redirect(url_for("auth.auth_page", mode="login"))

    new_password = request.form.get("new_password", "")
    confirm_password = request.form.get("confirm_password", "")
    if not PASSWORD_PATTERN.match(new_password):
        flash("Password must be 8+ chars with uppercase, lowercase, number and special symbol")
        return redirect(url_for("auth.auth_page", mode="login"))
    if new_password != confirm_password:
        flash("Passwords do not match")
        return redirect(url_for("auth.auth_page", mode="login"))

    user.password_hash = generate_password_hash(new_password)
    log_activity(
        "password_reset",
        "Reset password using security questions",
        user_id=user.id,
        user_name=user.name,
        user_email=user.email,
    )
    db.session.commit()
    flash("Password reset successfully. Please login")
    return redirect(url_for("auth.auth_page", mode="login"))


@auth.route("/logout")
def logout():
    session_token = session.get("session_token")
    if session.get("user_id"):
        log_activity(
            "logout",
            "Logged out of the system",
            user_id=session.get("user_id"),
            user_name=session.get("username"),
            user_email=session.get("email"),
        )
        end_active_session(session_token)
        db.session.commit()
    session.clear()
    flash("Logged out successfully")
    return redirect(url_for("auth.onboarding"))
```

## backend\app\routes\budget_routes.py

```
from datetime import date
from decimal import Decimal

from flask import Blueprint, flash, redirect, render_template, request, session, url_for

from app.models.budget_model import Budget
from app.models.account_model import Account, ensure_default_account
from app.models.category_model import Category, ensure_default_categories
from app.models.recurring_transaction_model import RecurringTransaction
from app.models.transaction_model import Transaction
from app.models.user_settings_model import get_or_create_user_settings
from app.utils.activity_logger import log_activity
from app.utils.decorators import login_required
from extensions.db import db

budgets = Blueprint("budgets", __name__)


@budgets.route("/budgets")
@login_required
def budget_list():
    user_id = session["user_id"]
    today = date.today()
    month = int(request.args.get("month", today.month))
    year = int(request.args.get("year", today.year))

    ensure_default_categories(user_id)
    ensure_default_account(user_id)
    settings = get_or_create_user_settings(user_id)
    db.session.commit()

    accounts = Account.query.filter_by(user_id=user_id).order_by(Account.account_name.asc()).all()
    expense_categories = (
        Category.query.filter_by(user_id=user_id, type="expense")
        .order_by(Category.name.asc())
        .all()
    )
    items = (
        Budget.query.filter_by(user_id=user_id, month=month, year=year)
        .order_by(Budget.id.desc())
        .all()
    )
    recurring_expenses = (
        RecurringTransaction.query.filter_by(user_id=user_id, type="expense")
        .order_by(RecurringTransaction.is_active.desc(), RecurringTransaction.next_run_date.asc())
        .all()
    )

    spent_map = {}
    monthly_expenses = (
        Transaction.query.filter_by(user_id=user_id, type="expense")
        .filter(db.extract("month", Transaction.date) == month)
        .filter(db.extract("year", Transaction.date) == year)
        .all()
    )
    for txn in monthly_expenses:
        spent_map[txn.category_id] = spent_map.get(txn.category_id, Decimal("0")) + Decimal(txn.amount or 0)

    budget_rows = []
    for item in items:
        spent = spent_map.get(item.category_id, Decimal("0"))
        limit_amount = Decimal(item.limit_amount or 0)
        percent = int((spent / limit_amount) * 100) if limit_amount else 0
        budget_rows.append(
            {
                "budget": item,
                "spent": spent,
                "remaining": limit_amount - spent,
                "percent": max(0, percent),
            }
        )

    return render_template(
```

```
        "dashboard/budgets.html",
        active_page="budgets",
        budgets=budget_rows,
        accounts=accounts,
        categories=expense_categories,
        recurring_expenses=recurring_expenses,
        month=month,
        year=year,
        settings=settings,
    )


@budgets.route("/budgets", methods=["POST"])
@login_required
def add_budget():
    user_id = session["user_id"]
    category_id = int(request.form.get("category_id"))
    month = int(request.form.get("month"))
    year = int(request.form.get("year"))
    limit_amount = Decimal(request.form.get("limit_amount", "0") or "0")
    if limit_amount <= 0:
        flash("Budget amount must be greater than 0")
        return redirect(url_for("budgets.budget_list", month=month, year=year))

    existing = Budget.query.filter_by(
        user_id=user_id, category_id=category_id, month=month, year=year
    ).first()
    if existing:
        existing.limit_amount = limit_amount
        log_activity(
            "budget_updated",
            f"Updated budget for category #{category_id} to {limit_amount:.2f}",
        )
        flash("Budget updated successfully")
    else:
        db.session.add(
            Budget(
                user_id=user_id,
                category_id=category_id,
                limit_amount=limit_amount,
                month=month,
                year=year,
            )
        )
        log_activity(
            "budget_added",
            f"Added budget for category #{category_id} with limit {limit_amount:.2f}",
        )
        flash("Budget added successfully")

    db.session.commit()
    return redirect(url_for("budgets.budget_list", month=month, year=year))


@budgets.route("/budgets/recurring", methods=["POST"])
@login_required
def add_recurring_expense():
    user_id = session["user_id"]
    account_id = int(request.form.get("account_id"))
    category_id = int(request.form.get("category_id"))
    amount = Decimal(request.form.get("amount", "0") or "0")
    frequency = request.form.get("frequency", "monthly").strip()
    try:
        start_date = date.fromisoformat(request.form.get("start_date", ""))
    except ValueError:
        flash("Choose a valid start date")
        return redirect(url_for("budgets.budget_list"))
    description = request.form.get("description", "").strip()

    account = Account.query.filter_by(id=account_id, user_id=user_id).first()
    category = Category.query.filter_by(id=category_id, user_id=user_id, type="expense").first()
    if not account or not category:
        flash("Choose a valid account and expense category")
        return redirect(url_for("budgets.budget_list"))
    if amount <= 0:
        flash("Recurring expense amount must be greater than 0")
```

```
        return redirect(url_for("budgets.budget_list"))
    if frequency not in {"daily", "monthly", "yearly"}:
        flash("Choose a valid recurring frequency")
        return redirect(url_for("budgets.budget_list"))

    db.session.add(
        RecurringTransaction(
            user_id=user_id,
            account_id=account_id,
            category_id=category_id,
            type="expense",
            amount=amount,
            description=description or f"{category.name} recurring expense",
            frequency=frequency,
            start_date=start_date,
            next_run_date=start_date,
        )
    )
    log_activity("recurring_expense_added", f"Added {frequency} recurring expense for {category.name}")
    db.session.commit()
    flash("Recurring expense saved successfully")
    return redirect(url_for("budgets.budget_list"))


@budgets.route("/budgets/recurring/<int:recurring_id>/toggle", methods=["POST"])
@login_required
def toggle_recurring_expense(recurring_id):
    item = RecurringTransaction.query.filter_by(
        id=recurring_id, user_id=session["user_id"], type="expense"
    ).first_or_404()
    item.is_active = not bool(item.is_active)
    log_activity(
        "recurring_expense_toggled",
        f"{'Activated' if item.is_active else 'Paused'} recurring expense #{item.id}",
    )
    db.session.commit()
    flash("Recurring expense updated successfully")
    return redirect(url_for("budgets.budget_list"))


@budgets.route("/budgets/recurring/<int:recurring_id>/delete", methods=["POST"])
@login_required
def delete_recurring_expense(recurring_id):
    item = RecurringTransaction.query.filter_by(
        id=recurring_id, user_id=session["user_id"], type="expense"
    ).first_or_404()
    log_activity("recurring_expense_deleted", f"Deleted recurring expense #{item.id}")
    db.session.delete(item)
    db.session.commit()
    flash("Recurring expense deleted successfully")
    return redirect(url_for("budgets.budget_list"))


@budgets.route("/budgets/<int:budget_id>/delete", methods=["POST"])
@login_required
def delete_budget(budget_id):
    item = Budget.query.filter_by(id=budget_id, user_id=session["user_id"]).first_or_404()
    month = item.month
    year = item.year
    log_activity("budget_deleted", f"Deleted budget #{item.id}")
    db.session.delete(item)
    db.session.commit()
    flash("Budget deleted successfully")
    return redirect(url_for("budgets.budget_list", month=month, year=year))
```

## backend\app\routes\category_routes.py

```
from flask import Blueprint, flash, redirect, render_template, request, session, url_for

from app.models.category_model import Category, ensure_default_categories
from app.utils.activity_logger import log_activity
from app.utils.decorators import login_required
from extensions.db import db

categories = Blueprint("categories", __name__)


@categories.route("/categories")
@login_required
def category_list():
    ensure_default_categories(session["user_id"])
    db.session.commit()
    items = (
        Category.query.filter_by(user_id=session["user_id"])
        .order_by(Category.type.asc(), Category.name.asc())
        .all()
    )
    return render_template(
        "dashboard/categories.html",
        active_page="categories",
        categories=items,
    )


@categories.route("/categories", methods=["POST"])
@login_required
def add_category():
    name = request.form.get("name", "").strip()
    category_type = request.form.get("type", "expense").strip()

    if not name:
        flash("Category name is required")
        return redirect(url_for("categories.category_list"))

    existing = Category.query.filter_by(
        user_id=session["user_id"], name=name, type=category_type
    ).first()
    if existing:
        flash("Category already exists")
        return redirect(url_for("categories.category_list"))

    db.session.add(
        Category(user_id=session["user_id"], name=name, type=category_type)
    )
    log_activity("category_added", f"Added {category_type} category {name}")
    db.session.commit()
    flash("Category added successfully")
    return redirect(url_for("categories.category_list"))


@categories.route("/categories/<int:category_id>/delete", methods=["POST"])
@login_required
def delete_category(category_id):
    category = Category.query.filter_by(
        id=category_id, user_id=session["user_id"]
    ).first_or_404()
    log_activity("category_deleted", f"Deleted category {category.name}")
    db.session.delete(category)
    db.session.commit()
    flash("Category deleted successfully")
    return redirect(url_for("categories.category_list"))
```

## backend\app\routes\dashboard_routes.py

```
from collections import defaultdict
from datetime import date, timedelta
from decimal import Decimal

from time import perf_counter

from flask import Blueprint, current_app, flash, redirect, render_template, request, session, url_for
from flask import jsonify
from werkzeug.security import check_password_hash, generate_password_hash

from app.models.account_model import Account, ensure_default_account
from app.models.activity_log_model import ActivityLog
from app.models.budget_model import Budget
from app.models.category_model import Category, ensure_default_categories
from app.models.goal_model import Goal
from app.models.transaction_model import Transaction
from app.models.user_model import User
from app.models.user_settings_model import get_or_create_user_settings
from app.services.automation_service import process_monthly_salary
from app.services.active_session_service import count_active_sessions
from app.services.i18n_service import normalize_language
from app.utils.activity_logger import log_activity
from app.utils.decorators import admin_required, login_required
from extensions.db import db
from extensions.mongo import get_collection

dashboard = Blueprint("dashboard", __name__)


def _currency_symbol(currency_code):
    symbols = {
        "INR": "Rs.",
        "USD": "$",
        "EUR": "EUR",
        "GBP": "GBP",
    }
    return symbols.get(currency_code, currency_code)


def _build_dashboard_data(user_id):
    settings = get_or_create_user_settings(user_id)
    ensure_default_categories(user_id)
    ensure_default_account(user_id)
    db.session.commit()

    today = date.today()
    transactions = (
        Transaction.query.filter_by(user_id=user_id)
        .order_by(Transaction.date.desc(), Transaction.id.desc())
        .all()
    )
    accounts = Account.query.filter_by(user_id=user_id).order_by(Account.id.desc()).all()
    goals = Goal.query.filter_by(user_id=user_id).order_by(Goal.id.desc()).all()
    budgets = (
        Budget.query.filter_by(user_id=user_id, month=today.month, year=today.year)
        .order_by(Budget.id.desc())
        .all()
    )

    current_month_transactions = [
        txn
        for txn in transactions
        if txn.date and txn.date.month == today.month and txn.date.year == today.year
    ]
    monthly_income = sum(
        Decimal(txn.amount or 0)
        for txn in current_month_transactions
        if txn.type == "income"
    )
    monthly_expense = sum(
        Decimal(txn.amount or 0)
        for txn in current_month_transactions
        if txn.type == "expense"
```

```
    )
    total_balance = sum(Decimal(account.balance or 0) for account in accounts)
    average_monthly_expense = Decimal("0")

    monthly_map = defaultdict(lambda: {"income": Decimal("0"), "expense": Decimal("0")})
    for txn in transactions:
        if not txn.date:
            continue
        key = (txn.date.year, txn.date.month)
        monthly_map[key][txn.type] += Decimal(txn.amount or 0)

    month_labels = []
    year = today.year
    month = today.month
    for _ in range(4):
        month_labels.append((year, month))
        month -= 1
        if month == 0:
            month = 12
            year -= 1
    month_labels.reverse()

    monthly_stats = []
    max_month_value = Decimal("1")
    for stat_year, stat_month in month_labels:
        stats = monthly_map[(stat_year, stat_month)]
        max_month_value = max(max_month_value, stats["income"], stats["expense"])
        monthly_stats.append(
            {
                "label": f"{date(stat_year, stat_month, 1):%b}",
                "income": stats["income"],
                "expense": stats["expense"],
            }
        )

    for item in monthly_stats:
        item["income_width"] = int((item["income"] / max_month_value) * 100) if max_month_value else 0
        item["expense_width"] = int((item["expense"] / max_month_value) * 100) if max_month_value else 0

    expense_totals = [item["expense"] for item in monthly_stats if Decimal(item["expense"] or 0) > 0]
    if expense_totals:
        average_monthly_expense = sum(expense_totals) / Decimal(len(expense_totals))

    expense_by_category = defaultdict(Decimal)
    for txn in current_month_transactions:
        if txn.type == "expense" and txn.category_id:
            category_name = getattr(txn.category, "name", "Other")
            expense_by_category[category_name] += Decimal(txn.amount or 0)

    budget_rows = []
    for item in budgets:
        spent = expense_by_category.get(getattr(item.category, "name", ""), Decimal("0"))
        limit_amount = Decimal(item.limit_amount or 0)
        percent = int((spent / limit_amount) * 100) if limit_amount else 0
        budget_rows.append(
            {
                "budget": item,
                "spent": spent,
                "percent": max(0, percent),
            }
        )

    total_expense_for_chart = sum(expense_by_category.values()) or Decimal("0")
    chart_colors = ["#C9A84C", "#4f8ef7", "#5ECB8C", "#e07070", "#8A8F9E", "#8090f0"]
    chart_segments = []
    chart_legend = []
    top_spending_category = None
    start = 0
    for index, (name, amount) in enumerate(sorted(expense_by_category.items(), key=lambda item: item[1], reverse=True)
[:6]):
        percent = float((amount / total_expense_for_chart) * 100) if total_expense_for_chart else 0
        sweep = (percent / 100) * 360
        end = start + sweep
        color = chart_colors[index % len(chart_colors)]
        if top_spending_category is None:
            top_spending_category = name
```

```
        chart_segments.append(f"{color} {start:.2f}deg {end:.2f}deg")
        chart_legend.append(
            {
                "name": name,
                "amount": amount,
                "percent": round(percent, 1),
                "color": color,
            }
        )
        start = end

    pie_chart_style = (
        f"background: conic-gradient(from -90deg, {', '.join(chart_segments)}, #1A2035 {start:.2f}deg 360deg);"
        if chart_segments
        else "background: conic-gradient(from -90deg, #1A2035 0deg 360deg);"
    )

    recent_transactions = transactions[:5]
    top_goal = goals[0] if goals else None
    goal_percent = 0
    if top_goal and top_goal.target_amount:
        goal_percent = int((Decimal(top_goal.current_amount or 0) / Decimal(top_goal.target_amount)) * 100)
        goal_percent = max(0, min(goal_percent, 100))

    return {
        "settings": settings,
        "currency_symbol": _currency_symbol(settings.currency or "INR"),
        "total_balance": total_balance,
        "monthly_income": monthly_income,
        "monthly_expense": monthly_expense,
        "average_monthly_expense": average_monthly_expense,
        "monthly_stats": monthly_stats,
        "recent_transactions": recent_transactions,
        "pie_chart_style": pie_chart_style,
        "chart_legend": chart_legend,
        "top_spending_category": top_spending_category,
        "accounts": accounts,
        "budget_rows": budget_rows[:4],
        "top_goal": top_goal,
        "goal_percent": goal_percent,
    }


@dashboard.route("/home")
@login_required
def home():
    data = _build_dashboard_data(session["user_id"])
    data["show_guide"] = not session.get("guide_dismissed", False)
    return render_template("dashboard/home.html", active_page="dashboard", **data)


@dashboard.route("/account")
@login_required
def account():
    user = User.query.get_or_404(session["user_id"])
    settings = get_or_create_user_settings(user.id)
    ensure_default_categories(user.id)
    ensure_default_account(user.id)
    db.session.commit()
    accounts = Account.query.filter_by(user_id=user.id).order_by(Account.id.desc()).all()
    return render_template(
        "dashboard/account.html",
        active_page="account",
        user=user,
        settings=settings,
        accounts=accounts,
    )


@dashboard.route("/account/profile", methods=["POST"])
@login_required
def update_profile():
    user = User.query.get_or_404(session["user_id"])
    name = request.form.get("name", "").strip()
    if not name:
        flash("Name is required")
```

```
        return redirect(url_for("dashboard.account"))

    user.name = name
    session["username"] = name
    log_activity("profile_updated", "Updated account profile name")
    db.session.commit()
    flash("Profile updated successfully")
    return redirect(url_for("dashboard.account"))


@dashboard.route("/account/settings", methods=["POST"])
@login_required
def update_settings():
    settings = get_or_create_user_settings(session["user_id"])
    settings.currency = request.form.get("currency", "INR").strip() or "INR"
    settings.locale = request.form.get("locale", "en-IN").strip() or "en-IN"
    settings.language = normalize_language(request.form.get("language", "en"))
    settings.month_start_day = int(request.form.get("month_start_day", 1) or 1)
    settings.month_start_day = min(max(settings.month_start_day, 1), 28)
    settings.monthly_salary = Decimal(request.form.get("monthly_salary", "0") or "0")
    log_activity("settings_updated", "Updated account settings")
    process_monthly_salary(session["user_id"])
    db.session.commit()
    flash("Settings updated successfully")
    return redirect(url_for("dashboard.account"))


@dashboard.route("/account/password", methods=["POST"])
@login_required
def update_password():
    user = User.query.get_or_404(session["user_id"])
    current_password = request.form.get("current_password", "")
    new_password = request.form.get("new_password", "")
    confirm_password = request.form.get("confirm_password", "")

    if not check_password_hash(user.password_hash, current_password):
        flash("Current password is incorrect")
        return redirect(url_for("dashboard.account"))
    if len(new_password) < 8:
        flash("New password must be at least 8 characters")
        return redirect(url_for("dashboard.account"))
    if new_password != confirm_password:
        flash("New password and confirmation do not match")
        return redirect(url_for("dashboard.account"))

    user.password_hash = generate_password_hash(new_password)
    log_activity("password_updated", "Updated account password")
    db.session.commit()
    flash("Password updated successfully")
    return redirect(url_for("dashboard.account"))


@dashboard.route("/account/accounts", methods=["POST"])
@login_required
def add_account():
    account_name = request.form.get("account_name", "").strip()
    account_type = request.form.get("type", "Cash").strip()
    balance = Decimal(request.form.get("balance", "0") or "0")

    if not account_name:
        flash("Account name is required")
        return redirect(url_for("dashboard.account"))

    db.session.add(
        Account(
            user_id=session["user_id"],
            account_name=account_name,
            type=account_type,
            balance=balance,
        )
    )
    log_activity("account_added", f"Added account {account_name}")
    db.session.commit()
    flash("Account added successfully")
    return redirect(url_for("dashboard.account"))
```

```
@dashboard.route("/account/accounts/<int:account_id>/delete", methods=["POST"])
@login_required
def delete_account(account_id):
    account = Account.query.filter_by(id=account_id, user_id=session["user_id"]).first_or_404()
    log_activity("account_deleted", f"Deleted account {account.account_name}")
    db.session.delete(account)
    db.session.commit()
    flash("Account deleted successfully")
    return redirect(url_for("dashboard.account"))


@dashboard.route("/admin")
@login_required
@admin_required
def admin_dashboard():
    selected_user_id = request.args.get("user_id", "").strip()
    today = date.today()
    total_users = User.query.count()
    total_admins = User.query.filter_by(role="admin").count()
    blocked_users = User.query.filter_by(is_active=False).count()
    total_transactions = Transaction.query.count()
    total_categories = db.session.query(db.func.count(Category.id)).scalar() or 0
    total_budgets = Budget.query.count()
    total_goals = Goal.query.count()
    users = User.query.order_by(User.created_at.desc(), User.id.desc()).all()
    users_created_today = [user for user in users if user.created_at and user.created_at.date() == today]

    transaction_query = Transaction.query.order_by(Transaction.created_at.desc(), Transaction.id.desc())
    activity_query = ActivityLog.query.order_by(ActivityLog.created_at.desc(), ActivityLog.id.desc())
    if selected_user_id:
        transaction_query = transaction_query.filter_by(user_id=int(selected_user_id))
        activity_query = activity_query.filter_by(user_id=int(selected_user_id))

    system_transactions = transaction_query.all()
    activity_logs = []
    activity_collection = get_collection("activity_logs")
    if activity_collection is not None:
        mongo_filter = {}
        if selected_user_id:
            mongo_filter["user_id"] = int(selected_user_id)
        mongo_logs = list(activity_collection.find(mongo_filter).sort("_id", -1).limit(25))
        if mongo_logs:
            activity_logs = mongo_logs
    if not activity_logs:
        activity_logs = activity_query.limit(25).all()

    activity_rows = []
    for item in activity_logs:
        created_at = getattr(item, "created_at", None)
        if isinstance(item, dict):
            created_at = item.get("created_at")
            created_label = str(created_at or "-")
            activity_rows.append(
                {
                    "user_name": item.get("user_name", "Unknown User"),
                    "action": item.get("action", "activity"),
                    "details": item.get("details", ""),
                    "created_at_label": created_label,
                    "created_on": str(created_at or "")[:10],
                }
            )
        else:
            created_label = created_at.strftime("%d %b %Y %I:%M %p") if created_at else "-"
            activity_rows.append(
                {
                    "user_name": item.user_name,
                    "action": item.action,
                    "details": item.details,
                    "created_at_label": created_label,
                    "created_on": created_at.strftime("%Y-%m-%d") if created_at else "",
                }
            )

    activity_source_rows = activity_rows
    today_key = today.strftime("%Y-%m-%d")
```

```
    login_events_today = sum(1 for item in activity_source_rows if item["action"] == "login" and item["created_on"] ==
today_key)
    logout_events_today = sum(1 for item in activity_source_rows if item["action"] == "logout" and item["created_on"] ==
today_key)
    transactions_today = [item for item in system_transactions if item.created_at and item.created_at.date() == today]
    monthly_category_map = defaultdict(lambda: {"income": Decimal("0"), "expense": Decimal("0"), "count": 0})
    for item in system_transactions:
        if not item.date:
            continue
        category_name = item.category.name if item.category else "Uncategorized"
        key = (item.date.strftime("%b %Y"), category_name)
        monthly_category_map[key][item.type] += Decimal(item.amount or 0)
        monthly_category_map[key]["count"] += 1

    monthly_category_rows = [
        {
            "month": month_label,
            "category": category_name,
            "income": values["income"],
            "expense": values["expense"],
            "count": values["count"],
        }
        for (month_label, category_name), values in monthly_category_map.items()
    ][:30]

    overview_chart_rows = [
        {"label": "Users", "value": total_users, "tone": "gold"},
        {"label": "Transactions", "value": total_transactions, "tone": "blue"},
        {"label": "Budgets", "value": total_budgets, "tone": "green"},
        {"label": "Goals", "value": total_goals, "tone": "red"},
    ]
    overview_max = max([row["value"] for row in overview_chart_rows] + [1])
    for row in overview_chart_rows:
        row["width"] = max(8, int((row["value"] / overview_max) * 100)) if row["value"] else 0

    daily_activity_rows = []
    for offset in range(6, -1, -1):
        day = today - timedelta(days=offset)
        label = day.strftime("%d %b")
        logins = sum(
            1
            for item in activity_source_rows
            if item["action"] == "login" and item["created_on"] == day.strftime("%Y-%m-%d")
        )
        signups = sum(
            1 for user in users
            if user.created_at and user.created_at.date() == day
        )
        daily_activity_rows.append(
            {
                "label": label,
                "logins": logins,
                "signups": signups,
            }
        )
    activity_max = max([max(row["logins"], row["signups"]) for row in daily_activity_rows] + [1])
    for row in daily_activity_rows:
        row["login_width"] = max(8, int((row["logins"] / activity_max) * 100)) if row["logins"] else 0
        row["signup_width"] = max(8, int((row["signups"] / activity_max) * 100)) if row["signups"] else 0

    return render_template(
        "dashboard/admin.html",
        active_page="admin",
        total_users=total_users,
        total_admins=total_admins,
        blocked_users=blocked_users,
        total_transactions=total_transactions,
        total_categories=total_categories,
        total_budgets=total_budgets,
        total_goals=total_goals,
        users=users,
        user_lookup={user.id: user for user in users},
        activity_logs=activity_rows,
        monthly_category_rows=monthly_category_rows,
        selected_user_id=selected_user_id,
        users_created_today=len(users_created_today),
```

```
        login_events_today=login_events_today,
        logout_events_today=logout_events_today,
        transactions_today=len(transactions_today),
        overview_chart_rows=overview_chart_rows,
        daily_activity_rows=daily_activity_rows,
    )


@dashboard.route("/admin/users/<int:user_id>/toggle-block", methods=["POST"])
@login_required
@admin_required
def toggle_user_block(user_id):
    user = User.query.get_or_404(user_id)
    if user.email == session.get("email"):
        flash("Admin account cannot be blocked")
        return redirect(url_for("dashboard.admin_dashboard"))

    user.is_active = not bool(user.is_active)
    action_text = "unblocked" if user.is_active else "blocked"
    log_activity(
        "admin_user_status_changed",
        f"Admin {action_text} user {user.email}",
        user_id=session["user_id"],
        user_name=session.get("username"),
        user_email=session.get("email"),
    )
    db.session.commit()
    flash(f"User {action_text} successfully")
    return redirect(url_for("dashboard.admin_dashboard"))


@dashboard.route("/admin/users/<int:user_id>/delete", methods=["POST"])
@login_required
@admin_required
def delete_user(user_id):
    user = User.query.get_or_404(user_id)
    if user.email == session.get("email"):
        flash("Admin account cannot be deleted")
        return redirect(url_for("dashboard.admin_dashboard"))

    user_email = user.email
    log_activity(
        "admin_user_deleted",
        f"Admin deleted user {user_email}",
        user_id=session["user_id"],
        user_name=session.get("username"),
        user_email=session.get("email"),
    )
    db.session.delete(user)
    db.session.commit()
    flash("User deleted successfully")
    return redirect(url_for("dashboard.admin_dashboard"))


@dashboard.route("/admin/performance")
@login_required
@admin_required
def admin_performance():
    today = date.today()
    active_users = count_active_sessions()
    activity_collection = get_collection("activity_logs")
    total_logs = (
        activity_collection.count_documents({})
        if activity_collection is not None
        else ActivityLog.query.count()
    )
    db_started = perf_counter()
    db.session.query(db.func.count(User.id)).scalar()
    db_response_ms = round((perf_counter() - db_started) * 1000, 2)

    total_users = User.query.count()
    total_transactions = Transaction.query.count()
    last_response_ms = current_app.config.get("LAST_RESPONSE_MS", 0)

    performance_rows = [
        {"label": "Active Logins", "value": active_users, "tone": "green"},
```

```
        {"label": "DB Response", "value": db_response_ms, "tone": "blue"},
        {"label": "Last Request", "value": last_response_ms, "tone": "gold"},
        {"label": "Activity Logs", "value": total_logs, "tone": "red"},
    ]
    max_value = max([Decimal(str(row["value"] or 0)) for row in performance_rows] + [Decimal("1")])
    for row in performance_rows:
        value = Decimal(str(row["value"] or 0))
        row["width"] = max(8, int((value / max_value) * 100)) if value else 0

    growth_rows = [
        {"label": "Users", "value": total_users, "tone": "green"},
        {"label": "Transactions", "value": total_transactions, "tone": "blue"},
        {"label": "Logs", "value": total_logs, "tone": "gold"},
    ]
    growth_max = max([row["value"] for row in growth_rows] + [1])
    for row in growth_rows:
        row["width"] = max(8, int((row["value"] / growth_max) * 100)) if row["value"] else 0

    return render_template(
        "dashboard/admin_performance.html",
        active_page="admin",
        performance_rows=performance_rows,
        growth_rows=growth_rows,
        active_users=active_users,
        db_response_ms=db_response_ms,
        last_response_ms=last_response_ms,
        total_logs=total_logs,
    )


@dashboard.route("/admin/performance/active-users")
@login_required
@admin_required
def admin_active_users():
    active_users = count_active_sessions()
    db.session.commit()
    return jsonify({"active_users": active_users})


@dashboard.route("/guide/dismiss", methods=["POST"])
@login_required
def dismiss_guide():
    session["guide_dismissed"] = True
    flash("Guide hidden. You can still use the modules from the sidebar.")
    return redirect(url_for("dashboard.home"))


@dashboard.route("/guide/show", methods=["POST"])
@login_required
def show_guide():
    session["guide_dismissed"] = False
    return redirect(url_for("dashboard.home"))
```

## backend\app\routes\goal_routes.py

```
from datetime import datetime
from decimal import Decimal

from flask import Blueprint, flash, redirect, render_template, request, session, url_for

from app.models.account_model import ensure_default_account
from app.models.goal_model import Goal
from app.models.user_settings_model import get_or_create_user_settings
from app.services.transaction_service import apply_transaction_effect
from app.utils.activity_logger import log_activity
from app.utils.decorators import login_required
from extensions.db import db

goals = Blueprint("goals", __name__)


@goals.route("/goals")
@login_required
def goal_list():
    settings = get_or_create_user_settings(session["user_id"])
    db.session.commit()

    items = (
        Goal.query.filter_by(user_id=session["user_id"])
        .order_by(Goal.id.desc())
        .all()
    )

    goal_rows = []
    for item in items:
        target = Decimal(item.target_amount or 0)
        current = Decimal(item.current_amount or 0)
        percent = int((current / target) * 100) if target else 0
        goal_rows.append(
            {
                "goal": item,
                "percent": max(0, min(percent, 100)),
                "remaining": max(Decimal("0"), target - current),
            }
        )

    return render_template(
        "dashboard/goals.html",
        active_page="goals",
        goals=goal_rows,
        settings=settings,
    )


@goals.route("/goals", methods=["POST"])
@login_required
def add_goal():
    goal_name = request.form.get("goal_name", "").strip()
    target_amount = Decimal(request.form.get("target_amount", "0") or "0")
    current_amount = Decimal(request.form.get("current_amount", "0") or "0")
    deadline_raw = request.form.get("deadline", "").strip()
    status = request.form.get("status", "In Progress").strip() or "In Progress"

    if not goal_name:
        flash("Goal name is required")
        return redirect(url_for("goals.goal_list"))
    if target_amount <= 0:
        flash("Target amount must be greater than 0")
        return redirect(url_for("goals.goal_list"))
    if current_amount < 0:
        flash("Current amount cannot be negative")
        return redirect(url_for("goals.goal_list"))

    existing = Goal.query.filter(
        Goal.user_id == session["user_id"],
        db.func.lower(Goal.goal_name) == goal_name.lower(),
        Goal.target_amount == target_amount,
    ).first()
```

```
    if existing:
        flash("Goal already exists with the same name and amount")
        return redirect(url_for("goals.goal_list"))

    deadline = None
    if deadline_raw:
        deadline = datetime.strptime(deadline_raw, "%Y-%m-%d").date()

    db.session.add(
        Goal(
            user_id=session["user_id"],
            goal_name=goal_name,
            target_amount=target_amount,
            current_amount=current_amount,
            deadline=deadline,
            status=status,
        )
    )
    log_activity("goal_added", f"Added saving goal {goal_name}")
    db.session.commit()
    flash("Goal added successfully")
    return redirect(url_for("goals.goal_list"))


@goals.route("/goals/<int:goal_id>/update", methods=["POST"])
@login_required
def update_goal(goal_id):
    item = Goal.query.filter_by(id=goal_id, user_id=session["user_id"]).first_or_404()
    goal_name = request.form.get("goal_name", "").strip() or item.goal_name
    target_amount = Decimal(request.form.get("target_amount", item.target_amount) or item.target_amount)
    duplicate = Goal.query.filter(
        Goal.user_id == session["user_id"],
        Goal.id != item.id,
        db.func.lower(Goal.goal_name) == goal_name.lower(),
        Goal.target_amount == target_amount,
    ).first()
    if duplicate:
        flash("Goal already exists with the same name and amount")
        return redirect(url_for("goals.goal_list"))
    if target_amount <= 0:
        flash("Target amount must be greater than 0")
        return redirect(url_for("goals.goal_list"))
    if Decimal(request.form.get("current_amount", item.current_amount) or item.current_amount) < 0:
        flash("Current amount cannot be negative")
        return redirect(url_for("goals.goal_list"))

    item.goal_name = goal_name
    item.target_amount = target_amount
    item.current_amount = Decimal(request.form.get("current_amount", item.current_amount) or item.current_amount)
    item.status = request.form.get("status", item.status).strip() or item.status

    deadline_raw = request.form.get("deadline", "").strip()
    item.deadline = datetime.strptime(deadline_raw, "%Y-%m-%d").date() if deadline_raw else None
    if Decimal(item.current_amount or 0) >= Decimal(item.target_amount or 0):
        item.status = "Completed"

    log_activity("goal_updated", f"Updated saving goal {item.goal_name}")
    db.session.commit()
    flash("Goal updated successfully")
    return redirect(url_for("goals.goal_list"))


@goals.route("/goals/<int:goal_id>/add-amount", methods=["POST"])
@login_required
def add_goal_amount(goal_id):
    item = Goal.query.filter_by(id=goal_id, user_id=session["user_id"]).first_or_404()
    amount_to_add = Decimal(request.form.get("amount_to_add", "0") or "0")

    if amount_to_add <= 0:
        flash("Add amount must be greater than 0")
        return redirect(url_for("goals.goal_list"))

    account = ensure_default_account(session["user_id"])
    if Decimal(account.balance or 0) < amount_to_add:
        db.session.rollback()
        flash("Not enough balance in your default account to add this goal amount")
```

```
        return redirect(url_for("goals.goal_list"))

    item.current_amount = Decimal(item.current_amount or 0) + amount_to_add
    apply_transaction_effect(account, "expense", amount_to_add)
    if Decimal(item.current_amount or 0) >= Decimal(item.target_amount or 0):
        item.status = "Completed"

    log_activity(
        "goal_progress_added",
        f"Added {amount_to_add:.2f} to goal {item.goal_name}",
    )
    db.session.commit()
    flash("Goal amount updated successfully")
    return redirect(url_for("goals.goal_list"))


@goals.route("/goals/<int:goal_id>/delete", methods=["POST"])
@login_required
def delete_goal(goal_id):
    item = Goal.query.filter_by(id=goal_id, user_id=session["user_id"]).first_or_404()
    log_activity("goal_deleted", f"Deleted saving goal {item.goal_name}")
    db.session.delete(item)
    db.session.commit()
    flash("Goal deleted successfully")
    return redirect(url_for("goals.goal_list"))
```

## backend\app\routes\notification_routes.py

```
from flask import Blueprint, flash, redirect, render_template, session, url_for

from app.models.notification_model import Notification
from app.services.notification_service import sync_notifications
from app.utils.decorators import login_required
from extensions.db import db

notifications = Blueprint("notifications", __name__)


@notifications.route("/notifications")
@login_required
def notification_list():
    sync_notifications(session["user_id"])
    items = (
        Notification.query.filter_by(user_id=session["user_id"])
        .order_by(Notification.created_at.desc(), Notification.id.desc())
        .all()
    )
    return render_template(
        "dashboard/notifications.html",
        active_page="notifications",
        notifications=items,
    )


@notifications.route("/notifications/<int:notification_id>/read", methods=["POST"])
@login_required
def mark_notification_read(notification_id):
    item = Notification.query.filter_by(
        id=notification_id, user_id=session["user_id"]
    ).first_or_404()
    item.is_read = True
    db.session.commit()
    flash("Notification marked as read")
    return redirect(url_for("notifications.notification_list"))


@notifications.route("/notifications/read-all", methods=["POST"])
@login_required
def mark_all_notifications_read():
    Notification.query.filter_by(user_id=session["user_id"], is_read=False).update(
        {"is_read": True}
    )
    db.session.commit()
    flash("All notifications marked as read")
    return redirect(url_for("notifications.notification_list"))
```

## backend\app\routes\report_routes.py

```
import csv
import io
import json
from datetime import datetime

from flask import Blueprint, Response, flash, redirect, render_template, request, session, url_for

from app.models.account_model import Account
from app.models.activity_log_model import ActivityLog
from app.models.category_model import Category
from app.models.goal_model import Goal
from app.models.recurring_transaction_model import RecurringTransaction
from app.models.transaction_model import Transaction
from app.models.user_model import User
from app.models.user_settings_model import UserSettings, get_or_create_user_settings
from app.services.notification_service import sync_notifications
from app.services.report_service import build_report_summary
from app.utils.activity_logger import log_activity
from app.utils.decorators import login_required
from extensions.db import db
from extensions.mongo import get_collection

reports = Blueprint("reports", __name__)


@reports.route("/reports")
@login_required
def report_list():
    user_id = session["user_id"]
    settings = get_or_create_user_settings(user_id)
    db.session.commit()

    start_raw = request.args.get("start_date", "").strip()
    end_raw = request.args.get("end_date", "").strip()
    category_id = request.args.get("category_id", "").strip()
    transaction_type = request.args.get("type", "").strip()

    start_date = datetime.strptime(start_raw, "%Y-%m-%d").date() if start_raw else None
    end_date = datetime.strptime(end_raw, "%Y-%m-%d").date() if end_raw else None
    category_filter = int(category_id) if category_id else None

    summary = build_report_summary(
        user_id,
        start_date=start_date,
        end_date=end_date,
        category_id=category_filter,
        transaction_type=transaction_type,
        save_snapshot=True,
    )
    categories = Category.query.filter_by(user_id=user_id).order_by(Category.name.asc()).all()
    report_snapshots = []
    collection = get_collection("report_snapshots")
    if collection is not None:
        report_snapshots = list(collection.find({"user_id": user_id}).sort("created_on", -1).limit(10))

    sync_notifications(user_id)

    return render_template(
        "dashboard/reports.html",
        active_page="reports",
        settings=settings,
        categories=categories,
        filters={
            "start_date": start_raw,
            "end_date": end_raw,
            "category_id": category_id,
            "type": transaction_type,
        },
        report_snapshots=report_snapshots,
        **summary,
    )
```

```
@reports.route("/reports/export/csv")
@login_required
def export_transactions_csv():
    user_id = session["user_id"]
    summary = build_report_summary(user_id)
    output = io.StringIO()
    writer = csv.writer(output)
    writer.writerow(["FinVest Analytics Summary"])
    writer.writerow(["Metric", "Value"])
    writer.writerow(["Total Income", f"{summary['total_income']:.2f}"])
    writer.writerow(["Total Expense", f"{summary['total_expense']:.2f}"])
    writer.writerow(["Net", f"{summary['net']:.2f}"])
    writer.writerow(["Average Monthly Spending", f"{summary['average_monthly_spending']:.2f}"])
    writer.writerow(["Savings Rate", f"{summary['savings_rate']:.2f}%"])
    writer.writerow(["Transaction Count", summary["transaction_count"]])
    if summary["top_category"]:
        writer.writerow(["Highest Expense Category", summary["top_category"][0]])
    writer.writerow([])
    writer.writerow(["Spending and Income Trends"])
    writer.writerow(["Trend", "Value", "Detail"])
    for item in summary["trend_rows"]:
        writer.writerow([item["label"], item["value"], item["detail"]])
    writer.writerow([])
    writer.writerow(["Income Source Chart Data"])
    writer.writerow(["Source", "Amount", "Percent"])
    for item in summary["income_source_rows"]:
        writer.writerow([item["name"], f"{item['amount']:.2f}", item["percent"]])
    writer.writerow([])
    writer.writerow(["Monthly Chart Data"])
    writer.writerow(["Month", "Income", "Expense", "Net"])
    for item in summary["monthly_rows"]:
        writer.writerow([item["label"], f"{item['income']:.2f}", f"{item['expense']:.2f}", f"{item['net']:.2f}"])
    writer.writerow([])
    writer.writerow(["Category Chart Data"])
    writer.writerow(["Category", "Amount", "Percent"])
    for item in summary["category_chart_rows"]:
        writer.writerow([item["name"], f"{item['amount']:.2f}", item["percent"]])
    writer.writerow([])
    writer.writerow(["Transactions"])
    writer.writerow(["Date", "Type", "Category", "Account", "Amount", "Description"])
    for item in summary["transactions"]:
        writer.writerow(
            [
                item.date.isoformat() if item.date else "",
                item.type,
                item.category.name if item.category else "",
                item.account.account_name if item.account else "",
                f"{item.amount:.2f}",
                item.description or "",
            ]
        )
    log_activity("transactions_exported", "Exported transactions to CSV")
    db.session.commit()
    return Response(
        output.getvalue(),
        mimetype="text/csv",
        headers={"Content-Disposition": "attachment; filename=finvest-transactions.csv"},
    )


@reports.route("/reports/backup/json")
@login_required
def backup_user_data():
    user_id = session["user_id"]
    user = User.query.get_or_404(user_id)
    settings = UserSettings.query.filter_by(user_id=user_id).first()
    accounts = Account.query.filter_by(user_id=user_id).all()
    categories = Category.query.filter_by(user_id=user_id).all()
    goals = Goal.query.filter_by(user_id=user_id).all()
    recurring = RecurringTransaction.query.filter_by(user_id=user_id).all()
    summary = build_report_summary(user_id)
    notifications = get_collection("activity_logs")

    payload = {
        "user": {"id": user.id, "name": user.name, "email": user.email},
        "settings": {
```

```
            "currency": settings.currency if settings else "INR",
            "locale": settings.locale if settings else "en-IN",
            "month_start_day": settings.month_start_day if settings else 1,
            "monthly_salary": float(settings.monthly_salary or 0) if settings else 0,
        },
        "accounts": [
            {
                "account_name": item.account_name,
                "type": item.type,
                "balance": float(item.balance or 0),
            }
            for item in accounts
        ],
        "categories": [
            {"name": item.name, "type": item.type}
            for item in categories
        ],
        "goals": [
            {
                "goal_name": item.goal_name,
                "target_amount": float(item.target_amount or 0),
                "current_amount": float(item.current_amount or 0),
                "deadline": item.deadline.isoformat() if item.deadline else None,
                "status": item.status,
            }
            for item in goals
        ],
        "transactions": [
            {
                "date": item.date.isoformat() if item.date else None,
                "type": item.type,
                "category": item.category.name if item.category else "",
                "account": item.account.account_name if item.account else "",
                "amount": float(item.amount or 0),
                "description": item.description or "",
            }
            for item in Transaction.query.filter_by(user_id=user_id).order_by(Transaction.date.desc()).all()
        ],
        "recurring_transactions": [
            {
                "account": item.account.account_name if item.account else "",
                "category": item.category.name if item.category else "",
                "type": item.type,
                "amount": float(item.amount or 0),
                "frequency": item.frequency,
                "description": item.description or "",
                "start_date": item.start_date.isoformat() if item.start_date else None,
                "next_run_date": item.next_run_date.isoformat() if item.next_run_date else None,
                "is_active": bool(item.is_active),
            }
            for item in recurring
        ],
        "analytics": {
            "total_income": float(summary["total_income"]),
            "total_expense": float(summary["total_expense"]),
            "net": float(summary["net"]),
            "average_monthly_spending": float(summary["average_monthly_spending"]),
            "savings_rate": float(summary["savings_rate"]),
            "transaction_count": summary["transaction_count"],
            "top_category": summary["top_category"][0] if summary["top_category"] else None,
            "monthly_chart": [
                {
                    "label": item["label"],
                    "income": float(item["income"]),
                    "expense": float(item["expense"]),
                    "net": float(item["net"]),
                }
                for item in summary["monthly_rows"]
            ],
            "category_chart": [
                {
                    "name": item["name"],
                    "amount": float(item["amount"]),
                    "percent": item["percent"],
                }
                for item in summary["category_chart_rows"]
```

```
            ],
            "income_source_chart": [
                {
                    "name": item["name"],
                    "amount": float(item["amount"]),
                    "percent": item["percent"],
                }
                for item in summary["income_source_rows"]
            ],
            "trends": summary["trend_rows"],
        },
        "mongo_logs_available": notifications is not None,
    }

    log_activity("backup_downloaded", "Downloaded JSON backup")
    db.session.commit()
    return Response(
        json.dumps(payload, indent=2),
        mimetype="application/json",
        headers={"Content-Disposition": "attachment; filename=finvest-backup.json"},
    )
```

## backend\app\routes\transaction_routes.py

```
import csv
import io
from datetime import datetime
from decimal import Decimal, InvalidOperation

from flask import Blueprint, flash, redirect, render_template, request, session, url_for

from app.models.account_model import Account, ensure_default_account
from app.models.category_model import Category, ensure_default_categories
from app.models.transaction_model import Transaction
from app.services.transaction_service import apply_transaction_effect
from app.utils.activity_logger import log_activity
from app.utils.decorators import login_required
from extensions.db import db

transactions = Blueprint("transactions", __name__)


def _load_transaction_support_data(user_id):
    ensure_default_categories(user_id)
    ensure_default_account(user_id)
    db.session.commit()
    accounts = Account.query.filter_by(user_id=user_id).order_by(Account.account_name.asc()).all()
    categories = (
        Category.query.filter_by(user_id=user_id)
        .order_by(Category.type.asc(), Category.name.asc())
        .all()
    )
    return accounts, categories


def _clean_import_row(row):
    return {
        (key or "").strip().lower(): (value or "").strip()
        for key, value in row.items()
    }


def _find_or_create_import_category(user_id, name, transaction_type):
    category = Category.query.filter_by(
        user_id=user_id, name=name, type=transaction_type
    ).first()
    if category:
        return category

    category = Category(user_id=user_id, name=name, type=transaction_type)
    db.session.add(category)
    db.session.flush()
    return category


def _find_import_account(user_id, account_name):
    if account_name:
        account = Account.query.filter_by(
            user_id=user_id, account_name=account_name
        ).first()
        if account:
            return account
    account = ensure_default_account(user_id)
    db.session.flush()
    return account


def _import_transaction_exists(user_id, account_id, category_id, transaction_type, amount, transaction_date,
description):
    return Transaction.query.filter_by(
        user_id=user_id,
        account_id=account_id,
        category_id=category_id,
        type=transaction_type,
        amount=amount,
        date=transaction_date,
        description=description,
```

```
    ).first() is not None


@transactions.route("/transactions")
@login_required
def transaction_list():
    user_id = session["user_id"]
    accounts, categories_data = _load_transaction_support_data(user_id)

    query = Transaction.query.filter_by(user_id=user_id)
    transaction_type = request.args.get("type", "").strip()
    category_id = request.args.get("category_id", "").strip()
    start_date = request.args.get("start_date", "").strip()
    end_date = request.args.get("end_date", "").strip()
    min_amount = request.args.get("min_amount", "").strip()
    max_amount = request.args.get("max_amount", "").strip()

    if transaction_type:
        query = query.filter_by(type=transaction_type)
    if category_id:
        query = query.filter_by(category_id=int(category_id))
    if start_date:
        query = query.filter(Transaction.date >= datetime.strptime(start_date, "%Y-%m-%d").date())
    if end_date:
        query = query.filter(Transaction.date <= datetime.strptime(end_date, "%Y-%m-%d").date())
    if min_amount:
        query = query.filter(Transaction.amount >= Decimal(min_amount))
    if max_amount:
        query = query.filter(Transaction.amount <= Decimal(max_amount))

    items = query.order_by(Transaction.date.desc(), Transaction.id.desc()).all()
    return render_template(
        "dashboard/transactions.html",
        active_page="transactions",
        transactions=items,
        accounts=accounts,
        categories=categories_data,
        filters={
            "type": transaction_type,
            "category_id": category_id,
            "start_date": start_date,
            "end_date": end_date,
            "min_amount": min_amount,
            "max_amount": max_amount,
        },
    )


@transactions.route("/transactions", methods=["POST"])
@login_required
def add_transaction():
    user_id = session["user_id"]
    account_id = int(request.form.get("account_id"))
    category_id = int(request.form.get("category_id"))
    transaction_type = request.form.get("type", "expense").strip()
    description = request.form.get("description", "").strip()

    try:
        amount = Decimal(request.form.get("amount", "0"))
    except InvalidOperation:
        flash("Amount must be a valid number")
        return redirect(url_for("transactions.transaction_list"))

    if amount <= 0:
        flash("Amount must be greater than 0")
        return redirect(url_for("transactions.transaction_list"))

    transaction_date = datetime.strptime(request.form.get("date"), "%Y-%m-%d").date()
    category = Category.query.filter_by(id=category_id, user_id=user_id).first()
    account = Account.query.filter_by(id=account_id, user_id=user_id).first()

    if not category or not account:
        flash("Please choose a valid account and category")
        return redirect(url_for("transactions.transaction_list"))

    if category.type != transaction_type:
```

```
        flash("Category type and transaction type must match")
        return redirect(url_for("transactions.transaction_list"))

    item = Transaction(
        user_id=user_id,
        account_id=account_id,
        category_id=category_id,
        type=transaction_type,
        amount=amount,
        date=transaction_date,
        description=description,
    )
    db.session.add(item)
    apply_transaction_effect(account, transaction_type, amount)
    log_activity(
        "transaction_added",
        f"Added {transaction_type} of {amount:.2f} in {category.name}",
    )
    db.session.commit()
    flash("Transaction added successfully")
    return redirect(url_for("transactions.transaction_list"))


@transactions.route("/transactions/import/csv", methods=["POST"])
@login_required
def import_transactions_csv():
    user_id = session["user_id"]
    upload = request.files.get("transaction_csv")
    if not upload or not upload.filename:
        flash("Choose a CSV file to import")
        return redirect(url_for("transactions.transaction_list"))

    try:
        stream = io.StringIO(upload.stream.read().decode("utf-8-sig"), newline=None)
        reader = csv.DictReader(stream)
        if not reader.fieldnames:
            flash("CSV file is empty")
            return redirect(url_for("transactions.transaction_list"))

        required_headers = {"date", "type", "category", "amount"}
        available_headers = {(header or "").strip().lower() for header in reader.fieldnames}
        missing_headers = required_headers - available_headers
        if missing_headers:
            flash(f"CSV is missing required columns: {', '.join(sorted(missing_headers))}")
            return redirect(url_for("transactions.transaction_list"))

        imported_count = 0
        skipped_count = 0
        error_rows = []

        for row_number, raw_row in enumerate(reader, start=2):
            row = _clean_import_row(raw_row)
            transaction_type = row.get("type", "").lower()
            category_name = row.get("category", "")
            description = row.get("description", "")

            if transaction_type not in {"income", "expense"} or not category_name:
                error_rows.append(str(row_number))
                continue

            try:
                amount = Decimal(row.get("amount", "0"))
                transaction_date = datetime.strptime(row.get("date", ""), "%Y-%m-%d").date()
            except (InvalidOperation, ValueError):
                error_rows.append(str(row_number))
                continue

            if amount <= 0:
                error_rows.append(str(row_number))
                continue

            category = _find_or_create_import_category(user_id, category_name, transaction_type)
            account = _find_import_account(user_id, row.get("account", ""))

            if _import_transaction_exists(
                user_id,
```

```
                account.id,
                category.id,
                transaction_type,
                amount,
                transaction_date,
                description,
            ):
                skipped_count += 1
                continue

            item = Transaction(
                user_id=user_id,
                account_id=account.id,
                category_id=category.id,
                type=transaction_type,
                amount=amount,
                date=transaction_date,
                description=description,
            )
            db.session.add(item)
            apply_transaction_effect(account, transaction_type, amount)
            imported_count += 1

        log_activity(
            "transactions_imported",
            f"Imported {imported_count} transaction(s), skipped {skipped_count} duplicate(s)",
        )
        db.session.commit()

        message = f"Imported {imported_count} transaction(s)"
        if skipped_count:
            message += f", skipped {skipped_count} duplicate(s)"
        if error_rows:
            message += f". Rows with errors: {', '.join(error_rows[:8])}"
            if len(error_rows) > 8:
                message += "..."
        flash(message)
    except UnicodeDecodeError:
        db.session.rollback()
        flash("CSV must be saved as UTF-8 text")
    except Exception:
        db.session.rollback()
        flash("Could not import CSV. Please check the file format and try again")

    return redirect(url_for("transactions.transaction_list"))


@transactions.route("/transactions/<int:transaction_id>/edit", methods=["POST"])
@login_required
def edit_transaction(transaction_id):
    user_id = session["user_id"]
    item = Transaction.query.filter_by(id=transaction_id, user_id=user_id).first_or_404()
    category_id = int(request.form.get("category_id"))
    account_id = int(request.form.get("account_id"))
    transaction_type = request.form.get("type", "expense").strip()
    new_amount = Decimal(request.form.get("amount", "0") or "0")
    category = Category.query.filter_by(id=category_id, user_id=user_id).first()
    account = Account.query.filter_by(id=account_id, user_id=user_id).first()

    if not category or not account:
        flash("Please choose a valid account and category")
        return redirect(url_for("transactions.transaction_list"))

    if category.type != transaction_type:
        flash("Category type and transaction type must match")
        return redirect(url_for("transactions.transaction_list"))
    if new_amount <= 0:
        flash("Amount must be greater than 0")
        return redirect(url_for("transactions.transaction_list"))

    old_account = Account.query.filter_by(id=item.account_id, user_id=user_id).first()
    if old_account:
        apply_transaction_effect(old_account, item.type, item.amount, reverse=True)

    item.account_id = account_id
    item.category_id = category_id
```

```
    item.type = transaction_type
    item.amount = new_amount
    item.date = datetime.strptime(request.form.get("date"), "%Y-%m-%d").date()
    item.description = request.form.get("description", "").strip()
    apply_transaction_effect(account, item.type, item.amount)
    log_activity(
        "transaction_updated",
        f"Updated transaction #{item.id} to {item.type} {item.amount:.2f} in {category.name}",
    )
    db.session.commit()
    flash("Transaction updated successfully")
    return redirect(url_for("transactions.transaction_list"))


@transactions.route("/transactions/<int:transaction_id>/delete", methods=["POST"])
@login_required
def delete_transaction(transaction_id):
    item = Transaction.query.filter_by(id=transaction_id, user_id=session["user_id"]).first_or_404()
    account = Account.query.filter_by(id=item.account_id, user_id=session["user_id"]).first()
    if account:
        apply_transaction_effect(account, item.type, item.amount, reverse=True)
    log_activity(
        "transaction_deleted",
        f"Deleted {item.type} transaction of {Decimal(item.amount or 0):.2f}",
    )
    db.session.delete(item)
    db.session.commit()
    flash("Transaction deleted successfully")
    return redirect(url_for("transactions.transaction_list"))
```

## backend\app\services\active_session_service.py

```
from datetime import datetime, timedelta

from app.models.active_session_model import ActiveSession
from extensions.db import db


ACTIVE_WINDOW_MINUTES = 5


def cleanup_stale_sessions(now=None):
    now = now or datetime.utcnow()
    cutoff = now - timedelta(minutes=ACTIVE_WINDOW_MINUTES)
    ActiveSession.query.filter(ActiveSession.last_seen_at < cutoff).delete()


def start_active_session(user_id, session_token):
    now = datetime.utcnow()
    cleanup_stale_sessions(now)
    active = ActiveSession.query.filter_by(session_token=session_token).first()
    if active:
        active.user_id = user_id
        active.last_seen_at = now
    else:
        db.session.add(
            ActiveSession(
                user_id=user_id,
                session_token=session_token,
                last_seen_at=now,
            )
        )


def touch_active_session(user_id, session_token):
    if not user_id or not session_token:
        return
    now = datetime.utcnow()
    active = ActiveSession.query.filter_by(session_token=session_token).first()
    if active:
        active.last_seen_at = now
    else:
        db.session.add(
            ActiveSession(
                user_id=user_id,
                session_token=session_token,
                last_seen_at=now,
            )
        )


def end_active_session(session_token):
    if session_token:
        ActiveSession.query.filter_by(session_token=session_token).delete()


def count_active_sessions():
    cleanup_stale_sessions()
    db.session.flush()
    return ActiveSession.query.count()
```

## backend\app\services\admin_service.py

```
from werkzeug.security import generate_password_hash

from app.models.user_model import User
from app.utils.decorators import ADMIN_EMAIL
from extensions.db import db


def seed_default_admin():
    existing = User.query.filter_by(email=ADMIN_EMAIL).first()
    if existing:
        if existing.role != "admin":
            existing.role = "admin"
        return existing

    admin = User(
        name="Admin",
        email=ADMIN_EMAIL,
        role="admin",
        password_hash=generate_password_hash("admin@123"),
    )
    db.session.add(admin)
    db.session.commit()
    return admin
```

## backend\app\services\auth_service.py

```
[Empty file]
```

## backend\app\services\automation_service.py

```
from calendar import monthrange
from datetime import date, timedelta
from decimal import Decimal

from app.models.account_model import Account, ensure_default_account
from app.models.category_model import Category, ensure_default_categories
from app.models.recurring_transaction_model import RecurringTransaction
from app.models.transaction_model import Transaction
from app.models.user_model import User
from app.models.user_settings_model import UserSettings, get_or_create_user_settings
from app.services.transaction_service import apply_transaction_effect
from extensions.db import db


def ensure_income_category(user_id, name="Salary"):
    category = Category.query.filter_by(user_id=user_id, name=name, type="income").first()
    if category:
        return category
    category = Category(user_id=user_id, name=name, type="income")
    db.session.add(category)
    db.session.flush()
    return category


def _add_months(day, months=1):
    month = day.month - 1 + months
    year = day.year + month // 12
    month = month % 12 + 1
    return day.replace(year=year, month=month, day=min(day.day, monthrange(year, month)[1]))


def _next_date(day, frequency):
    if frequency == "daily":
        return day + timedelta(days=1)
    if frequency == "yearly":
        try:
            return day.replace(year=day.year + 1)
        except ValueError:
            return day.replace(year=day.year + 1, day=28)
    return _add_months(day, 1)


def _salary_due(settings, today):
    if not settings.monthly_salary or Decimal(settings.monthly_salary or 0) <= 0:
        return False
    if not settings.monthly_salary_last_added:
        return True
    return (
        settings.monthly_salary_last_added.year,
        settings.monthly_salary_last_added.month,
    ) != (today.year, today.month)


def process_monthly_salary(user_id, today=None):
    today = today or date.today()
    ensure_default_categories(user_id)
    account = ensure_default_account(user_id)
    settings = get_or_create_user_settings(user_id)
    db.session.flush()

    if not _salary_due(settings, today):
        return 0

    category = ensure_income_category(user_id)
    amount = Decimal(settings.monthly_salary or 0)
    transaction = Transaction(
        user_id=user_id,
        account_id=account.id,
        category_id=category.id,
        type="income",
        amount=amount,
        date=today,
        description=f"Automatic monthly income for {today:%B %Y}",
```

```
    )
    db.session.add(transaction)
    apply_transaction_effect(account, "income", amount)
    settings.monthly_salary_last_added = today
    return 1


def process_recurring_transactions(user_id, today=None):
    today = today or date.today()
    created = 0
    items = RecurringTransaction.query.filter_by(user_id=user_id, is_active=True).all()

    for item in items:
        while item.next_run_date and item.next_run_date <= today:
            account = Account.query.filter_by(id=item.account_id, user_id=user_id).first()
            category = Category.query.filter_by(id=item.category_id, user_id=user_id).first()
            if not account or not category:
                item.is_active = False
                break

            run_date = item.next_run_date
            transaction = Transaction(
                user_id=user_id,
                account_id=account.id,
                category_id=category.id,
                type=item.type,
                amount=Decimal(item.amount or 0),
                date=run_date,
                description=item.description or f"Automatic {item.frequency} {item.type}",
            )
            db.session.add(transaction)
            apply_transaction_effect(account, item.type, item.amount)
            item.last_run_date = run_date
            item.next_run_date = _next_date(run_date, item.frequency)
            created += 1

    return created


def run_user_automations(user_id, today=None, commit=True):
    created = process_monthly_salary(user_id, today=today)
    created += process_recurring_transactions(user_id, today=today)
    if commit:
        db.session.commit()
    return created


def run_all_automations(today=None):
    total = 0
    for user in User.query.filter_by(is_active=True).all():
        total += run_user_automations(user.id, today=today, commit=False)
    db.session.commit()
    return total
```

## backend\app\services\budget_service.py

```
[Empty file]
```

## backend\app\services\currency_service.py

```
EXCHANGE_RATES_INR = {
    "INR": 1.0,
    "USD": 1 / 92.70,
    "EUR": 1 / 106.88,
    "GBP": 1 / 125.92,
}

CURRENCY_SYMBOLS = {
    "INR": "Rs.",
    "USD": "$",
    "EUR": "EUR",
    "GBP": "GBP",
}


def convert_from_inr(amount, currency):
    return float(amount or 0) * EXCHANGE_RATES_INR.get(currency or "INR", 1.0)


def currency_symbol(currency):
    return CURRENCY_SYMBOLS.get(currency or "INR", currency or "INR")
```

## backend\app\services\i18n_service.py

```
LANGUAGES = {
    "en": "English",
    "hi": "Hindi",
    "mr": "Marathi",
}


def normalize_language(value):
    value = (value or "en").strip()
    if value in LANGUAGES:
        return value
    if value.startswith("hi"):
        return "hi"
    if value.startswith("mr"):
        return "mr"
    return "en"
```

## backend\app\services\notification_service.py

```
from datetime import date
from decimal import Decimal

from app.models.budget_model import Budget
from app.models.goal_model import Goal
from app.models.notification_model import Notification
from app.models.transaction_model import Transaction
from extensions.db import db


def create_notification(user_id, title, message, notification_type, reference_key):
    existing = Notification.query.filter_by(
        user_id=user_id,
        notification_type=notification_type,
        reference_key=reference_key,
        message=message,
    ).first()
    if existing:
        return existing

    notification = Notification(
        user_id=user_id,
        title=title,
        message=message,
        notification_type=notification_type,
        reference_key=reference_key,
    )
    db.session.add(notification)
    return notification


def sync_notifications(user_id):
    today = date.today()

    welcome_key = f"welcome-{user_id}"
    create_notification(
        user_id,
        "Welcome to FinVest",
        "Use the dashboard, budgets, goals, and reports to manage your finances.",
        "welcome",
        welcome_key,
    )

    budgets = Budget.query.filter_by(user_id=user_id, month=today.month, year=today.year).all()
    for budget in budgets:
        spent = Decimal("0")
        expenses = (
            Transaction.query.filter_by(user_id=user_id, category_id=budget.category_id, type="expense")
            .filter(db.extract("month", Transaction.date) == today.month)
            .filter(db.extract("year", Transaction.date) == today.year)
            .all()
        )
        for item in expenses:
            spent += Decimal(item.amount or 0)

        limit_amount = Decimal(budget.limit_amount or 0)
        if not limit_amount:
            continue
        percent = int((spent / limit_amount) * 100)
        if percent >= 100:
            create_notification(
                user_id,
                "Budget Exceeded",
                f"{budget.category.name} crossed 100% of this month's budget.",
                "budget",
                f"budget-{budget.id}-100",
            )
        elif percent >= 80:
            create_notification(
                user_id,
                "Budget Alert",
                f"{budget.category.name} reached {percent}% of this month's budget.",
                "budget",
```

```
                f"budget-{budget.id}-80",
            )

    goals = Goal.query.filter_by(user_id=user_id).all()
    for goal in goals:
        if goal.deadline:
            days_left = (goal.deadline - today).days
            if 0 <= days_left <= 7:
                create_notification(
                    user_id,
                    "Goal Deadline Near",
                    f"{goal.goal_name} is due in {days_left} day(s).",
                    "goal",
                    f"goal-{goal.id}-deadline",
                )

    db.session.commit()


def unread_notification_count(user_id):
    return Notification.query.filter_by(user_id=user_id, is_read=False).count()
```

## backend\app\services\report_service.py

```
from collections import defaultdict
from datetime import date
from decimal import Decimal
import hashlib
import json

from app.models.transaction_model import Transaction
from extensions.db import db
from extensions.mongo import get_collection


def build_report_summary(
    user_id,
    start_date=None,
    end_date=None,
    category_id=None,
    transaction_type="",
    save_snapshot=False,
):
    query = Transaction.query.filter_by(user_id=user_id)
    if start_date:
        query = query.filter(Transaction.date >= start_date)
    if end_date:
        query = query.filter(Transaction.date <= end_date)
    if category_id:
        query = query.filter_by(category_id=category_id)
    if transaction_type:
        query = query.filter_by(type=transaction_type)

    transactions = query.order_by(Transaction.date.desc(), Transaction.id.desc()).all()
    total_income = sum(Decimal(item.amount or 0) for item in transactions if item.type == "income")
    total_expense = sum(Decimal(item.amount or 0) for item in transactions if item.type == "expense")

    category_totals = defaultdict(Decimal)
    income_source_totals = defaultdict(Decimal)
    expense_category_totals = defaultdict(Decimal)
    monthly_totals = defaultdict(lambda: {"income": Decimal("0"), "expense": Decimal("0")})
    for item in transactions:
        if item.category:
            category_totals[item.category.name] += Decimal(item.amount or 0)
            if item.type == "expense":
                expense_category_totals[item.category.name] += Decimal(item.amount or 0)
            elif item.type == "income":
                income_source_totals[item.category.name] += Decimal(item.amount or 0)
        if item.date:
            key = item.date.strftime("%Y-%m")
            monthly_totals[key][item.type] += Decimal(item.amount or 0)

    monthly_rows = [
        {
            "label": date(int(label[:4]), int(label[5:]), 1).strftime("%b %Y"),
            "income": values["income"],
            "expense": values["expense"],
            "net": values["income"] - values["expense"],
        }
        for label, values in sorted(monthly_totals.items())
    ]
    average_monthly_spending = Decimal("0")
    expense_months = [row["expense"] for row in monthly_rows if row["expense"] > 0]
    if expense_months:
        average_monthly_spending = sum(expense_months) / Decimal(len(expense_months))

    savings_rate = Decimal("0")
    if total_income:
        savings_rate = ((total_income - total_expense) / total_income) * Decimal("100")

    top_category = None
    if expense_category_totals:
        top_category = max(expense_category_totals.items(), key=lambda row: row[1])

    monthly_max = max(
        [max(row["income"], row["expense"]) for row in monthly_rows] + [Decimal("1")]
    )
```

```
    for row in monthly_rows:
        row["income_width"] = int((row["income"] / monthly_max) * 100)
        row["expense_width"] = int((row["expense"] / monthly_max) * 100)

    category_max = max(list(expense_category_totals.values()) + [Decimal("1")])
    category_chart_rows = [
        {
            "name": name,
            "amount": amount,
            "percent": round(float((amount / total_expense) * 100), 1) if total_expense else 0,
            "width": int((amount / category_max) * 100),
        }
        for name, amount in sorted(expense_category_totals.items(), key=lambda row: row[1], reverse=True)
    ]
    category_pie_rows = [
        {"name": row["name"], "amount": float(row["amount"]), "percent": row["percent"]}
        for row in category_chart_rows
    ]

    income_max = max(list(income_source_totals.values()) + [Decimal("1")])
    income_source_rows = [
        {
            "name": name,
            "amount": amount,
            "percent": round(float((amount / total_income) * 100), 1) if total_income else 0,
            "width": int((amount / income_max) * 100),
        }
        for name, amount in sorted(income_source_totals.items(), key=lambda row: row[1], reverse=True)
    ]

    trend_rows = []
    if top_category:
        trend_rows.append(
            {
                "label": "Highest spending category",
                "value": top_category[0],
                "detail": f"{top_category[0]} accounts for {round(float((top_category[1] / total_expense) * 100), 1) if
total_expense else 0}% of expenses.",
                "tone": "red",
            }
        )
    if income_source_rows:
        top_income = income_source_rows[0]
        trend_rows.append(
            {
                "label": "Main income source",
                "value": top_income["name"],
                "detail": f"{top_income['name']} contributes {top_income['percent']}% of income.",
                "tone": "green",
            }
        )
    if len(monthly_rows) >= 2:
        previous = monthly_rows[-2]["expense"]
        current = monthly_rows[-1]["expense"]
        if previous:
            change = ((current - previous) / previous) * Decimal("100")
            direction = "increased" if change >= 0 else "decreased"
            trend_rows.append(
                {
                    "label": "Recent spending movement",
                    "value": f"{abs(change):.1f}%",
                    "detail": f"Spending {direction} compared with the previous month.",
                    "tone": "red" if change >= 0 else "green",
                }
            )

    summary = {
        "transactions": transactions,
        "total_income": total_income,
        "total_expense": total_expense,
        "net": total_income - total_expense,
        "category_rows": sorted(category_totals.items(), key=lambda row: row[1], reverse=True),
        "monthly_rows": monthly_rows,
        "category_chart_rows": category_chart_rows,
        "category_pie_rows": category_pie_rows,
        "income_source_rows": income_source_rows,
```

```
        "trend_rows": trend_rows,
        "average_monthly_spending": average_monthly_spending,
        "savings_rate": savings_rate,
        "top_category": top_category,
        "transaction_count": len(transactions),
    }

    if save_snapshot:
        snapshot = {
            "user_id": user_id,
            "created_on": date.today().isoformat(),
            "start_date": start_date.isoformat() if start_date else None,
            "end_date": end_date.isoformat() if end_date else None,
            "category_id": category_id,
            "transaction_type": transaction_type,
            "total_income": float(total_income),
            "total_expense": float(total_expense),
            "net": float(summary["net"]),
            "transaction_count": len(transactions),
            "last_transaction_id": transactions[0].id if transactions else None,
            "trends": trend_rows,
            "category_chart": [
                {"name": row["name"], "amount": float(row["amount"]), "percent": row["percent"]}
                for row in category_chart_rows
            ],
            "income_sources": [
                {"name": row["name"], "amount": float(row["amount"]), "percent": row["percent"]}
                for row in income_source_rows
            ],
        }
        snapshot["fingerprint"] = hashlib.sha256(
            json.dumps(snapshot, sort_keys=True, default=str).encode("utf-8")
        ).hexdigest()
        collection = get_collection("report_snapshots")
        if collection is not None:
            existing = collection.find_one(
                {"user_id": user_id, "fingerprint": snapshot["fingerprint"]}
            )
            if existing is None:
                collection.insert_one(snapshot)

    return summary
```

## backend\app\services\schema_service.py

```
from sqlalchemy import inspect, text

from extensions.db import db


_schema_checked = False


def ensure_automation_schema():
    global _schema_checked
    if _schema_checked:
        return

    inspector = inspect(db.engine)
    tables = set(inspector.get_table_names())
    if "users" in tables:
        columns = {column["name"] for column in inspector.get_columns("users")}
        if "security_answers" not in columns:
            db.session.execute(
                text("ALTER TABLE users ADD COLUMN security_answers TEXT NULL")
            )
    if "user_settings" in tables:
        columns = {column["name"] for column in inspector.get_columns("user_settings")}
        if "monthly_salary_last_added" not in columns:
            db.session.execute(
                text("ALTER TABLE user_settings ADD COLUMN monthly_salary_last_added DATE NULL")
            )
        if "language" not in columns:
            db.session.execute(
                text("ALTER TABLE user_settings ADD COLUMN language VARCHAR(10) DEFAULT 'en'")
            )

    if "recurring_transactions" not in tables or "active_sessions" not in tables:
        db.create_all()

    db.session.commit()
    _schema_checked = True
```

## backend\app\services\transaction_service.py

```
from decimal import Decimal


def apply_transaction_effect(account, transaction_type, amount, reverse=False):
    signed_amount = Decimal(amount or 0)
    if transaction_type == "expense":
        signed_amount *= Decimal("-1")
    if reverse:
        signed_amount *= Decimal("-1")
    account.balance = Decimal(account.balance or 0) + signed_amount
```

## backend\app\static\css\auth.css

```
@import url("https://fonts.googleapis.com/css2?
family=Cormorant+Garamond:wght@300;400;600;700&family=DM+Sans:wght@300;400;500&display=swap");

*,
*::before,
*::after {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

:root {
    --gold: #c9a84c;
    --gold-light: #e5c97a;
    --gold-dim: rgba(201, 168, 76, 0.15);
    --dark: #0a0d12;
    --dark-2: #111520;
    --dark-3: #1a2035;
    --text: #eae6df;
    --muted: #8a8f9e;
    --border: rgba(201, 168, 76, 0.18);
    --input-bg: rgba(255, 255, 255, 0.03);
    --input-border: rgba(255, 255, 255, 0.1);
}

html,
body {
    height: 100%;
    font-family: "DM Sans", sans-serif;
    font-weight: 400;
    background: var(--dark);
    color: var(--text);
    overflow: hidden;
}

body::before {
    content: "";
    position: fixed;
    inset: 0;
    background:
        radial-gradient(
            ellipse 70% 60% at 15% 50%,
            rgba(30, 50, 120, 0.2) 0%,
            transparent 60%
        ),
        radial-gradient(
            ellipse 60% 50% at 85% 20%,
            rgba(201, 168, 76, 0.08) 0%,
            transparent 65%
        ),
        linear-gradient(135deg, #0a0d12 0%, #111520 60%, #0a0d12 100%);
    pointer-events: none;
}

nav {
    position: fixed;
    top: 0;
    left: 0;
    right: 0;
    z-index: 999;
    height: 64px;
    display: flex;
    align-items: center;
    padding: 0 2.5rem;
    border-bottom: 1px solid var(--border);
    background: rgba(10, 13, 18, 0.85);
    backdrop-filter: blur(16px);
    -webkit-backdrop-filter: blur(16px);
}

.logo {
    font-family: "Cormorant Garamond", serif;
    font-size: 1.75rem;
```

```
    font-weight: 700;
    letter-spacing: 0.04em;
    color: var(--gold);
    text-decoration: none;
}

.logo span {
    color: var(--text);
    opacity: 0.85;
}

body {
    display: flex;
    justify-content: center;
    align-items: center;
}

.container {
    position: relative;
    z-index: 1;
    background: var(--dark-2);
    border: 1px solid var(--border);
    border-radius: 20px;
    overflow: hidden;
    width: 920px;
    max-width: calc(100vw - 2rem);
    min-height: 580px;
    margin-top: 64px;
    box-shadow:
        0 40px 80px rgba(0, 0, 0, 0.5),
        inset 0 1px 0 rgba(255, 255, 255, 0.04);
}

.form-container {
    position: absolute;
    top: 0;
    height: 100%;
    width: 50%;
    transition: all 0.65s cubic-bezier(0.77, 0, 0.175, 1);
    display: flex;
    align-items: center;
}

.sign-in {
    left: 0;
    z-index: 2;
}

.sign-up {
    left: 0;
    opacity: 0;
    z-index: 1;
    pointer-events: none;
}

.container.right-panel-active .sign-in {
    transform: translateX(100%);
    opacity: 0;
}

.container.right-panel-active .sign-up {
    transform: translateX(100%);
    opacity: 1;
    z-index: 5;
    pointer-events: all;
}

.sign-up form {
    max-height: 580px;
    overflow-y: auto;
}

form {
    width: 100%;
    display: flex;
    flex-direction: column;
```

```
    gap: 0.85rem;
    padding: 2.75rem;
}

.form-header {
    margin-bottom: 0.5rem;
}

.form-header h1 {
    font-family: "Cormorant Garamond", serif;
    font-size: 2rem;
    font-weight: 600;
    letter-spacing: -0.01em;
    color: var(--text);
    margin-bottom: 0.3rem;
}

.form-header p {
    font-size: 0.82rem;
    color: var(--muted);
    font-weight: 300;
}

.input-group {
    position: relative;
    display: flex;
    align-items: center;
    min-width: 0;
}

.input-icon {
    position: absolute;
    left: 0.9rem;
    color: var(--muted);
    pointer-events: none;
    transition: color 0.25s;
    flex-shrink: 0;
    z-index: 1;
}

.input-group:focus-within .input-icon {
    color: var(--gold);
}

input,
select {
    width: 100%;
    background: var(--input-bg);
    border: 1px solid var(--input-border);
    border-radius: 10px;
    color: var(--text);
    font-family: "DM Sans", sans-serif;
    font-size: 0.875rem;
    font-weight: 400;
    padding: 0.78rem 1rem 0.78rem 2.6rem;
    outline: none;
    transition:
        border-color 0.25s,
        background 0.25s,
        box-shadow 0.25s;
    appearance: none;
    min-width: 0;
}

form > * {
    min-width: 0;
}

.field-note {
    font-size: 0.74rem;
    color: var(--muted);
    line-height: 1.5;
    margin-top: -0.15rem;
}

input::placeholder {
```

```
    color: rgba(138, 143, 158, 0.55);
}

input:focus,
select:focus {
    border-color: var(--gold);
    background: rgba(201, 168, 76, 0.05);
    box-shadow: 0 0 0 3px rgba(201, 168, 76, 0.12);
}

input:hover,
select:hover {
    border-color: rgba(201, 168, 76, 0.35);
}

.select-group {
    position: relative;
}

.select-group select {
    cursor: pointer;
    padding-right: 2.5rem;
}

.select-group select option {
    background: var(--dark-2);
    color: var(--text);
}

.select-arrow {
    position: absolute;
    right: 0.9rem;
    color: var(--muted);
    pointer-events: none;
    transition: color 0.25s;
}

.select-group:focus-within .select-arrow {
    color: var(--gold);
}

.forgot {
    font-size: 0.75rem;
    color: var(--muted);
    text-decoration: none;
    text-align: right;
    margin-top: -0.25rem;
    transition: color 0.2s;
    letter-spacing: 0.02em;
    background: transparent;
    border: 0;
    cursor: pointer;
    font-family: "DM Sans", sans-serif;
}

.forgot:hover {
    color: var(--gold);
}

.btn-main {
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 0.6rem;
    width: 100%;
    padding: 0.85rem 1.5rem;
    margin-top: 0.25rem;
    background: var(--gold);
    border: 1px solid var(--gold);
    border-radius: 10px;
    font-family: "DM Sans", sans-serif;
    font-size: 0.9rem;
    font-weight: 600;
    letter-spacing: 0.06em;
    text-transform: uppercase;
    color: var(--dark);
```

```
    cursor: pointer;
    position: relative;
    overflow: hidden;
    transition:
        all 0.3s cubic-bezier(0.22, 1, 0.36, 1),
        transform 0.3s ease;
}

.btn-main::before {
    content: "";
    position: absolute;
    inset: 0;
    background: linear-gradient(135deg, rgba(255, 255, 255, 0.2) 0%, transparent 60%);
    opacity: 0;
    transition: opacity 0.3s;
}

.btn-main::after {
    content: "";
    position: absolute;
    bottom: -2px;
    left: 50%;
    transform: translateX(-50%);
    width: 0;
    height: 2px;
    background: var(--dark);
    border-radius: 2px;
    transition: width 0.3s ease;
}

.btn-main:hover {
    background: var(--gold-light);
    border-color: var(--gold-light);
    color: var(--dark);
    transform: translateY(-2px);
    box-shadow:
        0 8px 25px rgba(201, 168, 76, 0.45),
        0 2px 8px rgba(201, 168, 76, 0.3);
}

.btn-main:hover::before {
    opacity: 1;
}

.btn-main:hover::after {
    width: 40%;
}

.btn-main:active {
    transform: translateY(0);
    box-shadow: 0 4px 12px rgba(201, 168, 76, 0.3);
}

.btn-main svg {
    transition: transform 0.3s ease;
    flex-shrink: 0;
}

.btn-main:hover svg {
    transform: translateX(4px);
}

.switch-text {
    font-size: 0.78rem;
    color: var(--muted);
    text-align: center;
    font-weight: 300;
}

.switch {
    color: var(--gold);
    font-weight: 500;
    cursor: pointer;
    border-bottom: 1px solid transparent;
    transition: border-color 0.2s;
    padding-bottom: 1px;
```

```
}

.switch:hover {
    border-color: var(--gold);
}

.security-questions {
    display: grid;
    gap: 0.6rem;
}

.security-questions label,
.reset-field {
    display: grid;
    gap: 0.35rem;
}

.security-questions span,
.reset-field span {
    color: var(--muted);
    font-size: 0.72rem;
    line-height: 1.35;
}

.security-questions input,
.reset-field input {
    padding-left: 1rem;
}

.reset-overlay {
    position: fixed;
    inset: 0;
    z-index: 1300;
    display: none;
    align-items: center;
    justify-content: center;
    background: rgba(10, 13, 18, 0.72);
    backdrop-filter: blur(9px);
    -webkit-backdrop-filter: blur(9px);
}

.reset-overlay.is-visible {
    display: flex;
}

.reset-card {
    position: relative;
    width: min(420px, calc(100vw - 2rem));
    background: var(--dark-2);
    border: 1px solid var(--border);
    border-radius: 18px;
    box-shadow:
        0 30px 70px rgba(0, 0, 0, 0.5),
        inset 0 1px 0 rgba(255, 255, 255, 0.04);
    animation: popup 0.28s ease;
}

.reset-form {
    padding: 2rem;
}

.reset-close {
    position: absolute;
    top: 12px;
    right: 14px;
    background: transparent;
    border: 0;
    color: var(--muted);
    font-size: 1.5rem;
    cursor: pointer;
    line-height: 1;
}

.reset-close:hover {
    color: var(--gold);
}
```

```
.overlay-container {
    position: absolute;
    top: 0;
    left: 50%;
    width: 50%;
    height: 100%;
    overflow: hidden;
    transition: transform 0.65s cubic-bezier(0.77, 0, 0.175, 1);
    z-index: 10;
    border-radius: 0 20px 20px 0;
}

.container.right-panel-active .overlay-container {
    transform: translateX(-100%);
}

.overlay {
    background: linear-gradient(145deg, #1a2035 0%, #0f1520 50%, #0a0d12 100%);
    position: relative;
    left: -100%;
    width: 200%;
    height: 100%;
    transform: translateX(0);
    transition: transform 0.65s cubic-bezier(0.77, 0, 0.175, 1);
}

.overlay::before {
    content: "";
    position: absolute;
    top: 0;
    left: 0;
    right: 0;
    height: 2px;
    background: linear-gradient(90deg, transparent, var(--gold), transparent);
    opacity: 0.6;
}

.overlay::after {
    content: "";
    position: absolute;
    inset: 0;
    background: radial-gradient(
        ellipse 80% 60% at 50% 40%,
        rgba(201, 168, 76, 0.07) 0%,
        transparent 70%
    );
}

.container.right-panel-active .overlay {
    transform: translateX(50%);
}

.overlay-panel {
    position: absolute;
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
    padding: 0 3rem;
    text-align: center;
    height: 100%;
    width: 50%;
    gap: 1rem;
    transition: transform 0.65s cubic-bezier(0.77, 0, 0.175, 1);
}

.overlay-left {
    left: 0;
    transform: translateX(-20%);
    opacity: 0;
    transition:
        transform 0.65s cubic-bezier(0.77, 0, 0.175, 1),
        opacity 0.4s ease;
}
```

```
.container.right-panel-active .overlay-left {
    transform: translateX(0);
    opacity: 1;
}

.overlay-right {
    right: 0;
    transform: translateX(0);
    opacity: 1;
    transition:
        transform 0.65s cubic-bezier(0.77, 0, 0.175, 1),
        opacity 0.4s ease;
}

.container.right-panel-active .overlay-right {
    transform: translateX(20%);
    opacity: 0;
}

.overlay-logo {
    font-family: "Cormorant Garamond", serif;
    font-size: 2rem;
    font-weight: 700;
    letter-spacing: 0.04em;
    color: var(--gold);
    margin-bottom: 0.5rem;
}

.overlay-logo span {
    color: var(--text);
    opacity: 0.85;
}

.overlay-panel h1 {
    font-family: "Cormorant Garamond", serif;
    font-size: 1.75rem;
    font-weight: 600;
    color: var(--text);
    line-height: 1.2;
    letter-spacing: -0.01em;
}

.overlay-panel p {
    font-size: 0.85rem;
    font-weight: 300;
    color: var(--muted);
    line-height: 1.65;
    max-width: 220px;
}

.overlay-panel::after {
    content: "";
    display: block;
    order: 1;
    width: 36px;
    height: 1px;
    background: var(--gold);
    opacity: 0.5;
    margin: -0.25rem 0 0.25rem;
}

@media (max-width: 700px) {
    .container {
        min-height: unset;
        width: 100%;
        border-radius: 0;
        margin-top: 64px;
        height: calc(100vh - 64px);
        overflow-y: auto;
    }

    .overlay-container {
        display: none;
    }

    .form-container {
```

```
        width: 100%;
        position: relative;
        top: unset;
    }

    .sign-in,
    .sign-up {
        position: relative;
        display: none;
    }

    .sign-in {
        display: flex;
    }

    .container.right-panel-active .sign-in {
        display: none;
        transform: none;
    }

    .container.right-panel-active .sign-up {
        display: flex;
        transform: none;
        position: relative;
        opacity: 1;
        pointer-events: all;
    }
}

#flash-overlay {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background: rgba(10, 13, 18, 0.75);
    backdrop-filter: blur(6px);
    display: flex;
    align-items: center;
    justify-content: center;
    z-index: 1200;
    animation: fadeIn 0.25s ease;
}

.flash-box {
    background: var(--dark-2);
    border: 1px solid var(--border);
    color: var(--text);
    padding: 2rem 2.5rem;
    border-radius: 16px;
    width: 320px;
    text-align: center;
    box-shadow:
        0 30px 60px rgba(0, 0, 0, 0.45),
        inset 0 1px 0 rgba(255, 255, 255, 0.04);
    animation: popup 0.28s ease;
}

.flash-text {
    font-size: 1rem;
    font-weight: 600;
    color: var(--text);
    margin-bottom: 1.5rem;
    line-height: 1.5;
}

#flash-ok {
    background: var(--gold);
    border: 1px solid var(--gold);
    color: var(--dark);
    font-family: "DM Sans", sans-serif;
    font-size: 0.85rem;
    font-weight: 600;
    letter-spacing: 0.05em;
    text-transform: uppercase;
    padding: 0.7rem 1.6rem;
```

```
    border-radius: 10px;
    cursor: pointer;
    transition:
        transform 0.2s ease,
        background 0.2s ease,
        border-color 0.2s ease;
}

#flash-ok:hover {
    background: var(--gold-light);
    border-color: var(--gold-light);
    transform: translateY(-2px);
}

@keyframes popup {
    from {
        transform: scale(0.85);
        opacity: 0;
    }

    to {
        transform: scale(1);
        opacity: 1;
    }
}

@keyframes fadeIn {
    from {
        opacity: 0;
    }

    to {
        opacity: 1;
    }
}
```

## backend\app\static\css\dashboard.css

```
body {
    margin: 0;
    font-family: "DM Sans", sans-serif;
    background: #0A0D12;
    color: #EAE6DF;
    display: flex;
}

:root {
    color-scheme: dark;
}

*,
*::before,
*::after {
    box-sizing: border-box;
}

.sidebar {
    width: 220px;
    min-height: 100vh;
    background: #111520;
    padding: 24px 20px;
    display: flex;
    flex-direction: column;
    border-right: 1px solid rgba(255, 255, 255, 0.08);
}

.logo {
    font-size: 22px;
    font-weight: 700;
    color: #C9A84C;
    margin-bottom: 24px;
}

.logo span {
    color: #EAE6DF;
}

.nav {
    display: flex;
    flex-direction: column;
    gap: 6px;
}

.nav-item {
    display: block;
    padding: 11px 12px;
    color: #8A8F9E;
    text-decoration: none;
    border-radius: 10px;
    transition: background 0.2s ease, color 0.2s ease;
}

.nav-item:hover,
.nav-item.active {
    background: rgba(201, 168, 76, 0.08);
    color: #C9A84C;
}

.nav-badge {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    min-width: 18px;
    height: 18px;
    margin-left: 6px;
    border-radius: 999px;
    background: #C9A84C;
    color: #0A0D12;
    font-size: 11px;
    font-weight: 700;
}
```

```
.main {
    flex: 1;
    display: flex;
    flex-direction: column;
    min-width: 0;
}

.header {
    padding: 20px 28px;
    border-bottom: 1px solid rgba(255, 255, 255, 0.08);
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 16px;
}

.header h1 {
    margin: 0 0 4px;
    font-size: 28px;
    color: #EAE6DF;
}

.page-subtitle {
    margin: 0;
    color: #8A8F9E;
    font-size: 14px;
}

.header-right {
    display: flex;
    align-items: center;
    gap: 12px;
}

.profile-chip {
    display: flex;
    align-items: center;
    gap: 10px;
    text-decoration: none;
    color: inherit;
    cursor: pointer;
    list-style: none;
}

.profile-chip::-webkit-details-marker {
    display: none;
}

.profile-menu {
    position: relative;
}

.icon-button {
    position: relative;
    width: 42px;
    height: 42px;
    display: inline-flex;
    align-items: center;
    justify-content: center;
    border-radius: 50%;
    text-decoration: none;
    color: #EAE6DF;
    background: #111520;
    border: 1px solid rgba(201, 168, 76, 0.12);
}

.bell-icon {
    font-size: 18px;
}

.icon-badge {
    position: absolute;
    top: -2px;
    right: -2px;
    min-width: 18px;
```

```
    height: 18px;
    border-radius: 999px;
    background: #C9A84C;
    color: #0A0D12;
    font-size: 11px;
    font-weight: 700;
    display: flex;
    align-items: center;
    justify-content: center;
    padding: 0 4px;
}

.avatar {
    width: 42px;
    height: 42px;
    border-radius: 50%;
    background: linear-gradient(135deg, #C9A84C, #E5C97A);
    color: #0A0D12;
    display: flex;
    align-items: center;
    justify-content: center;
    font-weight: 700;
}

.profile-meta {
    display: flex;
    flex-direction: column;
    gap: 2px;
}

.profile-meta strong {
    font-size: 14px;
}

.profile-meta span {
    color: #8A8F9E;
    font-size: 12px;
}

.profile-dropdown {
    position: absolute;
    top: calc(100% + 10px);
    right: 0;
    min-width: 180px;
    padding: 8px;
    background: #111520;
    border: 1px solid rgba(201, 168, 76, 0.12);
    border-radius: 14px;
    box-shadow: 0 12px 24px rgba(0, 0, 0, 0.28);
    z-index: 10;
}

.profile-dropdown-link {
    display: block;
    padding: 10px 12px;
    border-radius: 10px;
    color: #EAE6DF;
    text-decoration: none;
}

.profile-dropdown-link:hover {
    background: rgba(201, 168, 76, 0.08);
    color: #C9A84C;
}

.danger-link {
    color: #e07070;
}

.flash-stack {
    padding: 18px 28px 0;
}

.flash-banner {
    background: rgba(201, 168, 76, 0.12);
    color: #EAE6DF;
```

```
    border: 1px solid rgba(201, 168, 76, 0.24);
    padding: 12px 14px;
    border-radius: 12px;
    margin-bottom: 10px;
}

.page-content {
    padding: 24px 28px 28px;
}

.stats-grid {
    display: grid;
    grid-template-columns: repeat(4, minmax(0, 1fr));
    gap: 16px;
    margin-bottom: 18px;
}

.content-grid {
    display: grid;
    grid-template-columns: minmax(0, 2fr) minmax(280px, 1fr);
    gap: 18px;
}

.content-grid.single-page {
    align-items: start;
}

.content-main,
.content-side {
    display: flex;
    flex-direction: column;
    gap: 18px;
}

.card,
.panel {
    background: #111520;
    border: 1px solid rgba(201, 168, 76, 0.12);
    border-radius: 16px;
    padding: 18px;
}

.stat-card {
    min-height: 120px;
}

.card-title {
    font-size: 14px;
    color: #8A8F9E;
    margin-bottom: 8px;
}

.card-value {
    font-size: 28px;
    color: #C9A84C;
    font-weight: 700;
    margin-top: 6px;
    margin-bottom: 6px;
}

.helper-text {
    color: #8A8F9E;
    font-size: 13px;
}

.up {
    color: #5ECB8C;
}

.warn {
    color: #e07070;
}

.panel-header {
    display: flex;
    justify-content: space-between;
```

```
    align-items: center;
    gap: 12px;
    margin-bottom: 14px;
    color: #EAE6DF;
    font-weight: 600;
}

.link {
    color: #C9A84C;
    text-decoration: none;
    font-size: 13px;
}

.compare-list {
    display: flex;
    flex-direction: column;
    gap: 14px;
}

.compare-row {
    display: grid;
    grid-template-columns: 50px minmax(0, 1fr) 160px;
    gap: 12px;
    align-items: center;
}

.compare-label {
    color: #EAE6DF;
    font-weight: 600;
}

.compare-bars {
    display: grid;
    gap: 10px;
}

.compare-bar-group {
    display: grid;
    grid-template-columns: 56px minmax(0, 1fr);
    gap: 8px;
    align-items: center;
}

.mini-label {
    color: #8A8F9E;
    font-size: 12px;
}

.compare-values {
    display: flex;
    flex-direction: column;
    gap: 6px;
    font-size: 13px;
    color: #EAE6DF;
    text-align: right;
}

.bar {
    height: 8px;
    background: #1A2035;
    border-radius: 999px;
    overflow: hidden;
}

.bar.tall {
    height: 10px;
}

.bar div {
    height: 100%;
    border-radius: 999px;
}

.bar-income {
    background: linear-gradient(90deg, #5ECB8C, #9de4b5);
}
```

```
.bar-expense,
.bar.red div {
    background: linear-gradient(90deg, #e07070, #f0aaaa);
}

.chart-layout {
    display: grid;
    grid-template-columns: 220px minmax(0, 1fr);
    gap: 18px;
    align-items: center;
}

.pie-wrap {
    display: flex;
    justify-content: center;
    align-items: center;
}

.pie-chart {
    position: relative;
    width: 180px;
    height: 180px;
    border-radius: 50%;
    box-shadow: inset 0 0 0 1px rgba(255, 255, 255, 0.04);
}

.pie-chart-hole {
    position: absolute;
    inset: 28px;
    border-radius: 50%;
    background: #111520;
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    text-align: center;
    padding: 10px;
}

.pie-chart-hole strong {
    font-size: 14px;
}

.pie-chart-hole span {
    color: #8A8F9E;
    font-size: 11px;
}

.legend-list {
    display: flex;
    flex-direction: column;
    gap: 12px;
}

.legend-item {
    display: flex;
    align-items: center;
    gap: 10px;
}

.legend-dot {
    width: 12px;
    height: 12px;
    border-radius: 50%;
    flex-shrink: 0;
}

.legend-text {
    display: flex;
    flex-direction: column;
    gap: 2px;
}

.legend-text strong {
    font-size: 14px;
```

```
}

.legend-text small {
    color: #8A8F9E;
}

.tx,
.list-row {
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 12px;
    padding: 12px 0;
    border-bottom: 1px solid rgba(255, 255, 255, 0.06);
}

.tx:last-child,
.list-row:last-child {
    border-bottom: 0;
}

.goal {
    display: flex;
    gap: 12px;
    align-items: center;
}

.circle {
    width: 58px;
    height: 58px;
    border-radius: 50%;
    background: #1A2035;
    color: #C9A84C;
    display: flex;
    align-items: center;
    justify-content: center;
    font-weight: 700;
}

.form-card,
.stack-form {
    display: flex;
    flex-direction: column;
    gap: 14px;
}

.form-grid {
    display: grid;
    gap: 14px;
    align-items: start;
}

.form-grid.two-col {
    grid-template-columns: repeat(2, minmax(0, 1fr));
}

.form-grid.four-col {
    grid-template-columns: repeat(4, minmax(0, 1fr));
    align-items: end;
}

.form-grid.six-col {
    grid-template-columns: repeat(6, minmax(0, 1fr));
    align-items: end;
}

label {
    display: flex;
    flex-direction: column;
    gap: 6px;
    color: #EAE6DF;
    font-size: 13px;
    min-width: 0;
}

input,
```

```
select {
    width: 100%;
    background: #0A0D12;
    border: 1px solid rgba(255, 255, 255, 0.1);
    color: #EAE6DF;
    border-radius: 10px;
    padding: 11px 12px;
    font-family: "DM Sans", sans-serif;
    min-width: 0;
}

input:focus,
select:focus {
    outline: none;
    border-color: #C9A84C;
    box-shadow: 0 0 0 3px rgba(201, 168, 76, 0.12);
}

input[type="date"] {
    color-scheme: dark;
}

input[type="date"]::-webkit-datetime-edit,
input[type="date"]::-webkit-datetime-edit-text,
input[type="date"]::-webkit-datetime-edit-month-field,
input[type="date"]::-webkit-datetime-edit-day-field,
input[type="date"]::-webkit-datetime-edit-year-field {
    color: #EAE6DF;
}

input[type="date"]::-webkit-calendar-picker-indicator {
    filter: brightness(0) invert(1);
    cursor: pointer;
    opacity: 1;
}

.form-grid > *,
.stack-form > *,
.row-actions > * {
    min-width: 0;
}

.form-grid button,
.stack-form button,
.inline-filter button {
    width: 100%;
}

.btn-primary-action {
    background: #C9A84C;
    color: #0A0D12;
    border: 0;
    border-radius: 10px;
    padding: 12px 16px;
    font-weight: 700;
    cursor: pointer;
}

.btn-primary-action.small {
    padding: 10px 12px;
}

.btn-link-danger {
    background: transparent;
    border: 0;
    color: #e07070;
    cursor: pointer;
    padding: 0;
}

.btn-secondary-action {
    background: rgba(255, 255, 255, 0.08);
    color: #EAE6DF;
    border: 1px solid rgba(255, 255, 255, 0.08);
    border-radius: 10px;
    padding: 10px 12px;
```

```
    font-weight: 600;
    cursor: pointer;
}

.btn-secondary-action.small {
    padding: 8px 12px;
}

.row-actions {
    display: flex;
    align-items: center;
    gap: 12px;
}

.table-wrap {
    overflow-x: auto;
}

.scroll-panel {
    max-height: 430px;
    overflow: auto;
    padding-right: 4px;
}

.admin-spaced-section {
    margin-bottom: 18px;
}

.data-table {
    width: 100%;
    border-collapse: collapse;
}

.data-table th,
.data-table td {
    text-align: left;
    padding: 12px 10px;
    border-bottom: 1px solid rgba(255, 255, 255, 0.06);
    vertical-align: top;
}

.data-table th {
    color: #8A8F9E;
    font-size: 13px;
    font-weight: 600;
}

.type-chip {
    display: inline-flex;
    align-items: center;
    border-radius: 999px;
    padding: 4px 10px;
    font-size: 12px;
    text-transform: capitalize;
    background: rgba(255, 255, 255, 0.08);
}

.type-chip.income {
    color: #5ECB8C;
}

.type-chip.expense {
    color: #e07070;
}

.inline-editor summary {
    cursor: pointer;
    color: #C9A84C;
    margin-bottom: 8px;
}

.compact-form input,
.compact-form select {
    margin-bottom: 8px;
}
```

```
.inline-filter {
    display: flex;
    align-items: center;
    gap: 8px;
    flex-wrap: wrap;
}

.inline-filter input,
.inline-filter select {
    width: 120px;
}

.budget-list,
.goal-list {
    display: flex;
    flex-direction: column;
    gap: 14px;
}

.budget-card-row,
.goal-card {
    padding: 14px;
    border-radius: 14px;
    background: #0F141E;
    border: 1px solid rgba(255, 255, 255, 0.05);
}

.budget-card-header,
.goal-card-top,
.budget-amounts {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 12px;
}

.budget-card-header,
.goal-card-top {
    margin-bottom: 10px;
}

.budget-amounts {
    margin-bottom: 10px;
    color: #8A8F9E;
    font-size: 13px;
    flex-wrap: wrap;
}

.inline-amount-form {
    display: flex;
    gap: 8px;
    align-items: center;
    margin: 12px 0;
}

.inline-amount-form input {
    flex: 1;
}

.activity-list {
    display: flex;
    flex-direction: column;
    gap: 12px;
}

.activity-row {
    padding: 12px;
    border-radius: 12px;
    background: #0F141E;
    border: 1px solid rgba(255, 255, 255, 0.05);
}

.unread-card {
    border-color: rgba(201, 168, 76, 0.24);
}
```

```
.activity-header,
.activity-body {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 10px;
}

.activity-body {
    margin-top: 8px;
    align-items: flex-start;
    flex-direction: column;
}

.admin-grid {
    align-items: start;
}

.admin-actions {
    align-items: center;
}

.admin-stats {
    grid-template-columns: repeat(3, minmax(0, 1fr));
}

.admin-stats .small-value {
    font-size: 20px;
    line-height: 1.2;
}

.admin-charts {
    display: grid;
    grid-template-columns: minmax(0, 1.2fr) minmax(0, 0.8fr);
    gap: 18px;
    margin-bottom: 18px;
}

.chart-card {
    background: #111520;
    border: 1px solid rgba(201, 168, 76, 0.12);
    border-radius: 16px;
    padding: 18px;
}

.chart-stack {
    display: flex;
    flex-direction: column;
    gap: 14px;
}

.chart-row {
    display: grid;
    grid-template-columns: 120px minmax(0, 1fr) 56px;
    gap: 10px;
    align-items: center;
}

.chart-label {
    font-size: 13px;
    color: #EAE6DF;
}

.chart-track {
    height: 10px;
    border-radius: 999px;
    background: #1A2035;
    overflow: hidden;
}

.chart-track-spaced {
    margin: 3px 0;
}

.chart-track-spaced + .chart-track-spaced {
    margin-top: 8px;
```

```
}

.chart-fill {
    height: 100%;
    border-radius: 999px;
    background: linear-gradient(90deg, #C9A84C, #e5c97a);
}

.chart-fill.blue {
    background: linear-gradient(90deg, #4f8ef7, #86b2ff);
}

.chart-fill.green {
    background: linear-gradient(90deg, #5ECB8C, #9de4b5);
}

.chart-fill.red {
    background: linear-gradient(90deg, #e07070, #f0aaaa);
}

.chart-value {
    text-align: right;
    font-size: 13px;
    color: #EAE6DF;
}

.monthly-column-chart {
    min-height: 230px;
    display: grid;
    grid-auto-flow: column;
    grid-auto-columns: minmax(68px, 1fr);
    gap: 14px;
    align-items: end;
    overflow-x: auto;
    padding-bottom: 8px;
}

.monthly-column {
    display: grid;
    gap: 8px;
    justify-items: center;
}

.column-pair {
    height: 170px;
    display: flex;
    align-items: end;
    gap: 8px;
}

.column-fill {
    width: 18px;
    min-height: 4px;
    border-radius: 999px 999px 4px 4px;
    background: #5ECB8C;
}

.column-fill.expense {
    background: #e07070;
}

.report-dot:nth-of-type(1),
.legend-item:nth-child(1) .report-dot {
    background: #C9A84C;
}

.legend-item:nth-child(2) .report-dot {
    background: #4f8ef7;
}

.legend-item:nth-child(3) .report-dot {
    background: #5ECB8C;
}

.legend-item:nth-child(4) .report-dot {
    background: #e07070;
```

```
}

.legend-item:nth-child(5) .report-dot {
    background: #8A8F9E;
}

.legend-item:nth-child(6) .report-dot {
    background: #8090f0;
}

.mini-stats-grid {
    display: grid;
    grid-template-columns: repeat(2, minmax(0, 1fr));
    gap: 12px;
}

.mini-stat {
    padding: 14px;
    border-radius: 14px;
    background: #0F141E;
    border: 1px solid rgba(255, 255, 255, 0.05);
}

.mini-stat strong {
    display: block;
    font-size: 24px;
    color: #EAE6DF;
    margin-top: 4px;
}

.no-margin {
    margin: 0;
}

.guide-panel {
    margin-bottom: 18px;
}

.guide-timeline {
    display: grid;
    grid-template-columns: repeat(3, minmax(0, 1fr));
    gap: 12px;
}

.guide-step {
    display: grid;
    grid-template-columns: 34px minmax(0, 1fr);
    gap: 6px 10px;
    padding: 14px;
    border-radius: 14px;
    background: #0F141E;
    border: 1px solid rgba(255, 255, 255, 0.05);
    color: #EAE6DF;
    text-decoration: none;
}

.guide-step:hover {
    border-color: rgba(201, 168, 76, 0.32);
}

.guide-dot {
    grid-row: span 2;
    width: 34px;
    height: 34px;
    border-radius: 50%;
    background: rgba(201, 168, 76, 0.12);
    color: #C9A84C;
    display: flex;
    align-items: center;
    justify-content: center;
    font-weight: 700;
}

.guide-step small {
    color: #8A8F9E;
    line-height: 1.4;
```

```
}

.guide-compact {
    margin-bottom: 18px;
    padding: 12px 14px;
    border-radius: 14px;
    background: #111520;
    border: 1px solid rgba(201, 168, 76, 0.12);
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 12px;
    color: #8A8F9E;
}

.export-link {
    text-decoration: none;
    text-align: center;
}

.empty-text {
    color: #8A8F9E;
    margin: 0;
}

@media (max-width: 1100px) {
    .content-grid,
    .content-grid.single-page {
        grid-template-columns: 1fr;
    }

    .chart-layout {
        grid-template-columns: 1fr;
    }

    .admin-charts {
        grid-template-columns: 1fr;
    }

    .form-grid.four-col,
    .form-grid.six-col,
    .form-grid.two-col,
    .guide-timeline,
    .mini-stats-grid,
    .stats-grid {
        grid-template-columns: 1fr;
    }

    .inline-filter {
        flex-wrap: wrap;
        justify-content: flex-start;
    }

    .inline-filter input,
    .inline-filter select,
    .inline-filter button {
        width: 100%;
    }
}

@media (max-width: 820px) {
    body {
        flex-direction: column;
    }

    .sidebar {
        width: auto;
        min-height: auto;
    }

    .page-content,
    .flash-stack,
    .header {
        padding-left: 18px;
        padding-right: 18px;
    }
```

```
    .compare-row {
        grid-template-columns: 1fr;
    }

    .compare-values {
        text-align: left;
    }
}
```

## backend\app\static\css\onboarding.css

```
*,
*::before,
*::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

:root {
  --gold: #C9A84C;
  --gold-light: #E5C97A;
  --dark: #0A0D12;
  --dark-2: #111520;
  --dark-3: #1A2035;
  --text: #EAE6DF;
  --muted: #8A8F9E;
  --border: rgba(201, 168, 76, 0.18);
}

html,
body {
  height: 100%;
  font-family: "DM Sans", sans-serif;
  background: var(--dark);
  color: var(--text);
  overflow-x: hidden;
}

body::before {
  content: "";
  position: fixed;
  inset: 0;
  background:
    radial-gradient(ellipse 80% 60% at 70% 20%, rgba(201, 168, 76, 0.08) 0%, transparent 60%),
    radial-gradient(ellipse 60% 80% at 10% 80%, rgba(30, 50, 100, 0.25) 0%, transparent 60%),
    radial-gradient(ellipse 40% 40% at 90% 85%, rgba(201, 168, 76, 0.05) 0%, transparent 50%);
  pointer-events: none;
  z-index: 0;
}

nav {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  z-index: 100;
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0 3rem;
  height: 72px;
  border-bottom: 1px solid var(--border);
  background: rgba(10, 13, 18, 0.75);
  backdrop-filter: blur(18px);
  animation: slideDown 0.6s ease both;
}

@keyframes slideDown {
  from { opacity: 0; transform: translateY(-16px); }
  to { opacity: 1; transform: translateY(0); }
}

.logo {
  font-family: "Cormorant Garamond", serif;
  font-size: 1.8rem;
  font-weight: 700;
  letter-spacing: 0.04em;
  color: var(--gold);
}

.logo span {
  color: var(--text);
  opacity: 0.85;
```

```
}

.nav-actions {
  display: flex;
  gap: 0.75rem;
  align-items: center;
}

.btn {
  font-family: "DM Sans", sans-serif;
  font-size: 0.875rem;
  font-weight: 500;
  letter-spacing: 0.04em;
  border-radius: 4px;
  padding: 0.55rem 1.4rem;
  cursor: pointer;
  transition: all 0.25s ease;
  text-decoration: none;
  display: inline-flex;
  align-items: center;
}

.btn-ghost {
  background: transparent;
  border: 1px solid var(--border);
  color: var(--text);
}

.btn-ghost:hover {
  border-color: var(--gold);
  color: var(--gold);
}

.btn-primary {
  background: var(--gold);
  border: 1px solid var(--gold);
  color: var(--dark);
  font-weight: 600;
}

.btn-primary:hover {
  background: var(--gold-light);
  border-color: var(--gold-light);
  transform: translateY(-2px);
  box-shadow: 0 8px 24px rgba(201, 168, 76, 0.3);
}

.hero {
  position: relative;
  z-index: 1;
  min-height: 92vh;
  display: grid;
  grid-template-columns: 1fr 1fr;
  align-items: center;
  gap: 4rem;
  padding: 72px 5% 0;
  max-width: 1300px;
  margin: 0 auto;
}

.hero-left {
  animation: fadeUp 0.9s 0.2s ease both;
}

@keyframes fadeUp {
  from { opacity: 0; transform: translateY(32px); }
  to { opacity: 1; transform: translateY(0); }
}

.eyebrow {
  display: inline-flex;
  align-items: center;
  gap: 0.5rem;
  font-size: 0.72rem;
  font-weight: 500;
  letter-spacing: 0.2em;
```

```
  text-transform: uppercase;
  color: var(--gold);
  margin-bottom: 1.5rem;
}

.eyebrow::before {
  content: "";
  width: 28px;
  height: 1px;
  background: var(--gold);
}

h1 {
  font-family: "Cormorant Garamond", serif;
  font-size: clamp(3rem, 5.5vw, 5.5rem);
  font-weight: 300;
  line-height: 1.05;
  letter-spacing: -0.01em;
  margin-bottom: 1.75rem;
}

h1 em {
  font-style: italic;
  color: var(--gold);
}

.hero-desc {
  font-size: 1.05rem;
  line-height: 1.75;
  color: var(--muted);
  max-width: 480px;
  margin-bottom: 2.75rem;
}

.hero-cta {
  display: flex;
  gap: 1rem;
  align-items: center;
}

.btn-lg {
  padding: 0.85rem 2.2rem;
  font-size: 0.95rem;
}

.hero-note {
  font-size: 0.78rem;
  color: var(--muted);
  display: flex;
  align-items: center;
  gap: 0.4rem;
}

.hero-right {
  animation: fadeUp 0.9s 0.45s ease both;
  position: relative;
}

.dashboard-card {
  background: var(--dark-2);
  border: 1px solid var(--border);
  border-radius: 16px;
  padding: 1.75rem;
  box-shadow:
    0 40px 80px rgba(0, 0, 0, 0.5),
    inset 0 1px 0 rgba(255, 255, 255, 0.04);
  position: relative;
  overflow: hidden;
}

.dashboard-card::before {
  content: "";
  position: absolute;
  top: -60px;
  right: -60px;
  width: 200px;
```

```
  height: 200px;
  border-radius: 50%;
  background: radial-gradient(circle, rgba(201, 168, 76, 0.12), transparent 70%);
  pointer-events: none;
}

.card-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 1.5rem;
}

.card-title {
  font-size: 0.75rem;
  letter-spacing: 0.12em;
  text-transform: uppercase;
  color: var(--muted);
}

.badge {
  font-size: 0.7rem;
  padding: 0.2rem 0.55rem;
  border-radius: 20px;
  background: rgba(201, 168, 76, 0.12);
  color: var(--gold);
  border: 1px solid rgba(201, 168, 76, 0.25);
}

.portfolio-value {
  font-family: "Cormorant Garamond", serif;
  font-size: 2.75rem;
  font-weight: 600;
  letter-spacing: -0.02em;
  margin-bottom: 0.3rem;
}

.portfolio-change {
  font-size: 0.82rem;
  color: #5ECB8C;
  display: flex;
  align-items: center;
  gap: 0.3rem;
  margin-bottom: 1.75rem;
}

.mini-chart {
  height: 80px;
  margin-bottom: 1.75rem;
}

.performance-chart {
  display: flex;
  align-items: end;
  gap: 0.7rem;
  padding: 0.8rem 0 0.2rem;
  border-bottom: 1px solid rgba(255, 255, 255, 0.06);
}

.performance-chart span {
  flex: 1;
  min-height: 14px;
  border-radius: 999px 999px 4px 4px;
  background: linear-gradient(180deg, var(--gold), rgba(201, 168, 76, 0.12));
  animation: barPulse 3.6s ease-in-out infinite;
}

.performance-chart span:nth-child(2n) {
  animation-delay: 0.35s;
}

.performance-chart span:nth-child(3n) {
  animation-delay: 0.7s;
}

@keyframes barPulse {
```

```
  0%, 100% { filter: brightness(0.9); transform: scaleY(0.96); }
  50% { filter: brightness(1.15); transform: scaleY(1.04); }
}

.assets {
  display: flex;
  flex-direction: column;
  gap: 0.9rem;
}

.asset-row {
  display: flex;
  align-items: center;
  gap: 0.85rem;
}

.asset-icon {
  width: 32px;
  height: 32px;
  border-radius: 8px;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 0.75rem;
  font-weight: 700;
  flex-shrink: 0;
}

.asset-info {
  flex: 1;
}

.asset-name {
  font-size: 0.82rem;
  font-weight: 500;
  margin-bottom: 0.15rem;
}

.asset-ticker {
  font-size: 0.7rem;
  color: var(--muted);
}

.asset-bar-wrap {
  width: 80px;
}

.asset-bar-bg {
  height: 4px;
  border-radius: 2px;
  background: rgba(255, 255, 255, 0.06);
  overflow: hidden;
}

.asset-bar-fill {
  height: 100%;
  border-radius: 2px;
  background: var(--gold);
}

.asset-val {
  text-align: right;
  font-size: 0.82rem;
  font-weight: 500;
}

.asset-pct {
  font-size: 0.68rem;
  color: var(--muted);
}

.float-card {
  position: absolute;
  bottom: -24px;
  left: -32px;
  background: var(--dark-3);
```

```
  border: 1px solid var(--border);
  border-radius: 12px;
  padding: 1rem 1.25rem;
  width: 175px;
  box-shadow: 0 20px 50px rgba(0, 0, 0, 0.4);
  animation: float 4s ease-in-out infinite;
}

@keyframes float {
  0%, 100% { transform: translateY(0); }
  50% { transform: translateY(-8px); }
}

.float-label {
  font-size: 0.68rem;
  color: var(--muted);
  margin-bottom: 0.35rem;
}

.float-val {
  font-family: "Cormorant Garamond", serif;
  font-size: 1.5rem;
  font-weight: 600;
  color: #5ECB8C;
}

.workflow-section {
  position: relative;
  z-index: 1;
  max-width: 1300px;
  margin: 0 auto 4rem;
  padding: 4rem 5%;
  border-top: 1px solid var(--border);
  display: grid;
  grid-template-columns: minmax(260px, 0.8fr) minmax(0, 1.2fr);
  gap: 4rem;
  align-items: center;
  animation: fadeUp 0.9s 0.7s ease both;
}

.workflow-rail {
  position: relative;
  min-height: 360px;
  border-radius: 18px;
  background: var(--dark-2);
  border: 1px solid var(--border);
  overflow: hidden;
}

.rail-line {
  position: absolute;
  top: 58px;
  left: 50%;
  width: 2px;
  height: 250px;
  background: linear-gradient(180deg, transparent, var(--gold), transparent);
  animation: railGlow 3s ease-in-out infinite;
}

@keyframes railGlow {
  0%, 100% { opacity: 0.45; }
  50% { opacity: 1; }
}

.flow-node {
  position: absolute;
  left: calc(50% - 22px);
  width: 44px;
  height: 44px;
  border-radius: 50%;
  background: var(--dark-3);
  border: 1px solid var(--border);
  color: var(--gold);
  display: flex;
  align-items: center;
  justify-content: center;
```

```
  font-weight: 700;
  animation: nodeFloat 4s ease-in-out infinite;
}

.node-one { top: 42px; }
.node-two { top: 120px; animation-delay: 0.25s; }
.node-three { top: 198px; animation-delay: 0.5s; }
.node-four { top: 276px; animation-delay: 0.75s; }

@keyframes nodeFloat {
  0%, 100% { transform: translateX(-24px); }
  50% { transform: translateX(24px); }
}

.workflow-copy h2 {
  font-family: "Cormorant Garamond", serif;
  font-size: clamp(2.2rem, 4vw, 4rem);
  font-weight: 400;
  line-height: 1.05;
  margin-bottom: 1.5rem;
}

.workflow-steps {
  display: grid;
  gap: 1rem;
}

.workflow-steps div {
  display: grid;
  gap: 0.25rem;
  padding: 1rem 1.1rem;
  border-radius: 14px;
  background: rgba(17, 21, 32, 0.72);
  border: 1px solid rgba(255, 255, 255, 0.06);
}

.workflow-steps strong {
  color: var(--text);
}

.workflow-steps span {
  color: var(--muted);
  line-height: 1.5;
  font-size: 0.9rem;
}

.site-footer {
  position: relative;
  z-index: 1;
  max-width: 1300px;
  margin: 0 auto;
  padding: 2rem 5% 2.4rem;
  border-top: 1px solid var(--border);
  display: grid;
  grid-template-columns: minmax(0, 1.2fr) minmax(240px, 0.8fr) auto;
  gap: 1.5rem;
  align-items: center;
  animation: fadeUp 0.9s ease both;
}

.footer-logo {
  margin-bottom: 0.5rem;
}

.site-footer p,
.footer-copy {
  color: var(--muted);
  font-size: 0.84rem;
  line-height: 1.6;
}

.footer-links {
  display: flex;
  gap: 1rem;
  flex-wrap: wrap;
}
```

```
.footer-links a {
  color: var(--muted);
  text-decoration: none;
  transition: color 0.2s ease;
}

.footer-links a:hover {
  color: var(--gold);
}

@media (max-width: 860px) {
  .hero {
    grid-template-columns: 1fr;
    padding-top: 100px;
  }

  .hero-right {
    display: none;
  }

  .workflow-section,
  .site-footer {
    grid-template-columns: 1fr;
  }

  nav {
    padding: 0 1.5rem;
  }
}
```

## backend\app\templates\auth\auth.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Finvest | Auth</title>
    <link
        href="https://fonts.googleapis.com/css2?
family=Cormorant+Garamond:wght@300;400;600;700&family=DM+Sans:wght@300;400;500&display=swap"
        rel="stylesheet"
    />
    <link href="{{ url_for('static', filename='css/auth.css') }}" rel="stylesheet" />
</head>
<body>
    <nav>
        <a href="{{ url_for('auth.onboarding') }}" class="logo">Fin<span>vest</span></a>
    </nav>

    {% with messages = get_flashed_messages() %}
        {% if messages %}
            <div id="flash-overlay">
                <div class="flash-box">
                    {% for message in messages %}
                        <p class="flash-text">{{ message }}</p>
                    {% endfor %}
                    <button id="flash-ok">OK</button>
                </div>
            </div>
        {% endif %}
    {% endwith %}

    <div class="container" id="container">
        <div class="form-container sign-up">
            <form method="POST" action="{{ url_for('auth.register') }}">
                <div class="form-header">
                    <h1>Create Account</h1>
                    <p>Start your financial journey today</p>
                </div>

                <div class="input-group">
                    <svg class="input-icon" width="16" height="16" fill="none" stroke="currentColor" stroke-width="1.8"
viewBox="0 0 24 24">
                        <circle cx="12" cy="8" r="4"></circle>
                        <path d="M4 20c0-4 3.6-7 8-7s8 3 8 7"></path>
                    </svg>
                    <input type="text" name="username" placeholder="Username" pattern="[A-Za-z0-9_.]+" title="Use only
letters, numbers, underscore and dot" required />
                </div>

                <div class="input-group">
                    <svg class="input-icon" width="16" height="16" fill="none" stroke="currentColor" stroke-width="1.8"
viewBox="0 0 24 24">
                        <rect x="2" y="4" width="20" height="16" rx="2"></rect>
                        <path d="M2 7l10 7 10-7"></path>
                    </svg>
                    <input type="email" name="email" placeholder="Email Address" required />
                </div>

                <!-- <div class="input-group select-group">
                    <svg class="input-icon" width="16" height="16" fill="none" stroke="currentColor" stroke-width="1.8"
viewBox="0 0 24 24">
                        <path d="M17 21v-2a4 4 0 00-4-4H5a4 4 0 00-4 4v2"></path>
                        <circle cx="9" cy="7" r="4"></circle>
                        <path d="M23 21v-2a4 4 0 00-3-3.87M16 3.13a4 4 0 010 7.75"></path>
                    </svg>
                    <select name="role">
                        <option value="user" selected>User</option>
                        <option value="admin">Admin</option>
                    </select>
                    <svg class="select-arrow" width="14" height="14" fill="none" stroke="currentColor" stroke-width="2"
viewBox="0 0 24 24">
                        <path d="M6 9l6 6 6-6"></path>
                    </svg>
```

```
                </div> -->

                <div class="input-group">
                    <svg class="input-icon" width="16" height="16" fill="none" stroke="currentColor" stroke-width="1.8"
viewBox="0 0 24 24">
                        <rect x="5" y="11" width="14" height="10" rx="2"></rect>
                        <path d="M8 11V7a4 4 0 018 0v4"></path>
                    </svg>
                    <input type="password" name="password" placeholder="Password" minlength="8" title="8+ chars with
uppercase, lowercase, number and special symbol" required />
                </div>

                <div class="input-group">
                    <svg class="input-icon" width="16" height="16" fill="none" stroke="currentColor" stroke-width="1.8"
viewBox="0 0 24 24">
                        <path d="M9 12l2 2 4-4"></path>
                        <rect x="5" y="11" width="14" height="10" rx="2"></rect>
                        <path d="M8 11V7a4 4 0 018 0v4"></path>
                    </svg>
                    <input type="password" name="confirm_password" placeholder="Confirm Password" minlength="8" required
/>
                </div>

                <div class="security-questions">
                    {% for question in security_questions %}
                        <label>
                            <span>{{ question }}</span>
                            <input type="text" name="security_answer_{{ loop.index0 }}" placeholder="Answer" required />
                        </label>
                    {% endfor %}
                </div>

                <p class="field-note">Email must be unique. Password: 8+ chars with uppercase, lowercase, number and
symbol.</p>

                <button type="submit" class="btn-main">
                    <span>Sign Up</span>
                    <svg width="16" height="16" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24">
                        <path d="M5 12h14"></path>
                        <path d="M12 5l7 7-7 7"></path>
                    </svg>
                </button>

                <p class="switch-text">
                    Already have an account?
                    <span class="switch" id="toLogin">Login</span>
                </p>
            </form>
        </div>

        <div class="form-container sign-in">
            <form method="POST" action="{{ url_for('auth.login') }}">
                <div class="form-header">
                    <h1>Welcome Back</h1>
                    <p>Sign in to your account</p>
                </div>

                <div class="input-group">
                    <svg class="input-icon" width="16" height="16" fill="none" stroke="currentColor" stroke-width="1.8"
viewBox="0 0 24 24">
                        <rect x="2" y="4" width="20" height="16" rx="2"></rect>
                        <path d="M2 7l10 7 10-7"></path>
                    </svg>
                    <input type="email" name="email" placeholder="Email Address" required />
                </div>

                <div class="input-group">
                    <svg class="input-icon" width="16" height="16" fill="none" stroke="currentColor" stroke-width="1.8"
viewBox="0 0 24 24">
                        <rect x="5" y="11" width="14" height="10" rx="2"></rect>
                        <path d="M8 11V7a4 4 0 018 0v4"></path>
                    </svg>
                    <input type="password" name="password" placeholder="Password" required />
                </div>

                <button type="button" class="forgot" id="forgotOpen">Forgot Password?</button>
```

```
            <button type="submit" class="btn-main">
                <span>Login</span>
                <svg width="16" height="16" fill="none" stroke="currentColor" stroke-width="2" viewBox="0 0 24 24">
                    <path d="M5 12h14"></path>
                    <path d="M12 5l7 7-7 7"></path>
                </svg>
            </button>

            <p class="switch-text">
                Don't have an account?
                <span class="switch" id="toSignup">Sign Up</span>
            </p>
        </form>
    </div>

    <div class="overlay-container">
        <div class="overlay">
            <div class="overlay-panel overlay-left">
                <div class="overlay-logo">Fin<span>vest</span></div>
                <h1>Welcome to Finvest</h1>
                <p>Start managing your wealth smarter. Your financial future begins here.</p>
            </div>

            <div class="overlay-panel overlay-right">
                <div class="overlay-logo">Fin<span>vest</span></div>
                <h1>Welcome Back</h1>
                <p>Login to continue your financial journey and track your portfolio.</p>
            </div>
        </div>
    </div>
</div>

<div class="reset-overlay {% if reset_email %}is-visible{% endif %}" id="resetOverlay">
    <div class="reset-card">
        <button type="button" class="reset-close" id="resetClose">×</button>
        {% if reset_email and reset_questions %}
            <form method="POST" action="{{ url_for('auth.forgot_password_reset') }}" class="reset-form">
                <div class="form-header">
                    <h1>Reset Password</h1>
                    <p>Answer your recovery questions to set a new password.</p>
                </div>
                <input type="hidden" name="reset_email" value="{{ reset_email }}">
                {% for item in reset_questions %}
                    <label class="reset-field">
                        <span>{{ item.question }}</span>
                        <input type="text" name="security_answer_{{ item.index }}" placeholder="Answer" required>
                    </label>
                {% endfor %}
                <label class="reset-field">
                    <span>New Password</span>
                    <input type="password" name="new_password" minlength="8" required>
                </label>
                <label class="reset-field">
                    <span>Confirm Password</span>
                    <input type="password" name="confirm_password" minlength="8" required>
                </label>
                <button type="submit" class="btn-main">Reset Password</button>
            </form>
        {% else %}
            <form method="POST" action="{{ url_for('auth.forgot_password_questions') }}" class="reset-form">
                <div class="form-header">
                    <h1>Forgot Password</h1>
                    <p>Enter your registered email to load recovery questions.</p>
                </div>
                <label class="reset-field">
                    <span>Email Address</span>
                    <input type="email" name="reset_email" placeholder="Email Address" required>
                </label>
                <button type="submit" class="btn-main">Continue</button>
            </form>
        {% endif %}
    </div>
</div>

<script>
```

```
        const container = document.getElementById("container");
        const mode = "{{ mode }}";
        const flash = document.getElementById("flash-overlay");
        const btn = document.getElementById("flash-ok");
        const forgotOpen = document.getElementById("forgotOpen");
        const resetOverlay = document.getElementById("resetOverlay");
        const resetClose = document.getElementById("resetClose");

        if (mode === "signup") {
            container.classList.add("right-panel-active");
        } else {
            container.classList.remove("right-panel-active");
        }

        document.getElementById("toLogin").onclick = () => {
            container.classList.remove("right-panel-active");
        };

        document.getElementById("toSignup").onclick = () => {
            container.classList.add("right-panel-active");
        };

        if (btn) {
            btn.onclick = () => {
                flash.remove();
            };
        }

        if (forgotOpen) {
            forgotOpen.onclick = () => {
                resetOverlay.classList.add("is-visible");
            };
        }

        if (resetClose) {
            resetClose.onclick = () => {
                resetOverlay.classList.remove("is-visible");
            };
        }

        setTimeout(() => {
            if (flash) flash.remove();
        }, 3500);
    </script>
</body>
</html>
```

## backend\app\templates\auth\onboarding.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>Finvest - Personal Finance Management</title>
  <link rel="stylesheet" href="https://fonts.googleapis.com/css2?
family=Cormorant+Garamond:wght@400;500;600;700&family=DM+Sans:wght@400;500;600;700&display=swap">
  <link href="{{ url_for('static', filename='css/onboarding.css') }}" rel="stylesheet"/>
</head>
<body>
  <nav>
    <div class="logo">Fin<span>vest</span></div>
    <div class="nav-actions">
      <a href="{{ url_for('auth.auth_page') }}" class="btn btn-primary">Get Started</a>
    </div>
  </nav>

  <section class="hero">
    <div class="hero-left">
      <div class="eyebrow">Personal finance control center</div>
      <h1>
        Understand &amp;amp; <em>control</em><br>
        your money<br>
        smarter.
      </h1>

      <p class="hero-desc">
        FinVest helps you track income, expenses, budgets, goals, recurring bills, and reports in one calm dashboard, so
daily money decisions become clearer and easier to manage.
      </p>

      <div class="hero-cta">
        <a href="{{ url_for('auth.auth_page') }}" class="btn btn-primary btn-lg">
          Get Started Free
        </a>
        <span class="hero-note">
          <svg width="14" height="14" fill="none" stroke="#8A8F9E" stroke-width="1.5" viewBox="0 0 24 24">
            <path d="M12 22s8-4 8-10V5l-8-3-8 3v7c0 6 8 10 8 10z"/>
          </svg>
          Secure personal dashboard
        </span>
      </div>
    </div>

    <div class="hero-right">
      <div class="dashboard-card">
        <div class="card-header">
          <span class="card-title">System Snapshot</span>
          <span class="badge">Live</span>
        </div>

        <div class="portfolio-value">{{ stats.transactions }}</div>
        <div class="portfolio-change">
          <svg width="12" height="12" fill="currentColor" viewBox="0 0 24 24">
            <path d="M7 14l5-5 5 5"/>
          </svg>
          tracked financial entries
        </div>

        <div class="mini-chart performance-chart">
          <span style="height: 48%"></span>
          <span style="height: 64%"></span>
          <span style="height: 42%"></span>
          <span style="height: 76%"></span>
          <span style="height: 58%"></span>
          <span style="height: 88%"></span>
          <span style="height: 70%"></span>
        </div>

        <div class="assets">
          <div class="asset-row">
            <div class="asset-icon" style="background: rgba(94,203,140,0.12); color: #5ECB8C;">U</div>
```

```
            <div class="asset-info">
              <div class="asset-name">Active users</div>
              <div class="asset-ticker">Registered accounts</div>
            </div>
            <div class="asset-bar-wrap">
              <div class="asset-bar-bg"><div class="asset-bar-fill" style="width: 72%"></div></div>
            </div>
            <div class="asset-val">
              <div>{{ stats.users }}</div>
              <div class="asset-pct" style="color: #5ECB8C">online ready</div>
            </div>
          </div>

          <div class="asset-row">
            <div class="asset-icon" style="background: rgba(201,168,76,0.12); color: #C9A84C;">T</div>
            <div class="asset-info">
              <div class="asset-name">Transactions</div>
              <div class="asset-ticker">Income and expense records</div>
            </div>
            <div class="asset-bar-wrap">
              <div class="asset-bar-bg"><div class="asset-bar-fill" style="width: 58%"></div></div>
            </div>
            <div class="asset-val">
              <div>{{ stats.transactions }}</div>
              <div class="asset-pct" style="color: #5ECB8C">tracked</div>
            </div>
          </div>

          <div class="asset-row">
            <div class="asset-icon" style="background: rgba(100,120,240,0.12); color: #8090f0;">D</div>
            <div class="asset-info">
              <div class="asset-name">Database check</div>
              <div class="asset-ticker">Current response sample</div>
            </div>
            <div class="asset-bar-wrap">
              <div class="asset-bar-bg"><div class="asset-bar-fill" style="width: 36%"></div></div>
            </div>
            <div class="asset-val">
              <div>{{ stats.db_ms }} ms</div>
              <div class="asset-pct" style="color: #5ECB8C">healthy</div>
            </div>
          </div>
        </div>
      </div>

      <div class="float-card">
        <div class="float-label">Budget Alerts</div>
        <div class="float-val">{{ stats.logs }}</div>
      </div>
    </div>
  </section>

  <section class="workflow-section">
    <div class="workflow-rail">
      <div class="rail-line"></div>
      <div class="flow-node node-one">1</div>
      <div class="flow-node node-two">2</div>
      <div class="flow-node node-three">3</div>
      <div class="flow-node node-four">4</div>
    </div>
    <div class="workflow-copy">
      <div class="eyebrow">How FinVest flows</div>
      <h2>From daily entries to useful decisions.</h2>
      <div class="workflow-steps">
        <div><strong>Add records</strong><span>Enter income, expenses, categories, and accounts.</span></div>
        <div><strong>Set plans</strong><span>Create budgets, goals, salary automation, and recurring bills.</span></div>
        <div><strong>Read signals</strong><span>Use charts, KPIs, notifications, and trend reports.</span></div>
        <div><strong>Export safely</strong><span>Download CSV or JSON summaries whenever you need them.</span></div>
      </div>
    </div>
  </section>

  <footer class="site-footer">
    <div>
      <div class="logo footer-logo">Fin<span>vest</span></div>
      <p>Personal finance management for clearer budgets, goals, and money habits.</p>
```

```
    </div>
    <div class="footer-links">
      <a href="mailto:support@finvest.local">Contact Us</a>
      <a href="{{ url_for('auth.auth_page') }}">Get Started</a>
      <a href="#">About Us</a>
    </div>
    <div class="footer-copy">© 2026 FinVest. All rights reserved.</div>
  </footer>
</body>
</html>
```

## backend\app\templates\dashboard\account.html

```
{% extends "dashboard/layout.html" %}

{% block title %}Account | FinVest{% endblock %}
{% block page_title %}Account Management{% endblock %}
{% block page_subtitle %}Update your name, currency, salary settings, and manage the accounts you use for transactions.{%
endblock %}

{% block content %}
<section class="content-grid single-page">
    <div class="content-main">
        <div class="card form-card">
            <div class="panel-header"><span>Profile</span></div>
            <form method="POST" action="{{ url_for('dashboard.update_profile') }}" class="stack-form">
                <div class="form-grid two-col">
                    <label>
                        <span>Name</span>
                        <input type="text" name="name" value="{{ user.name }}" required>
                    </label>
                    <label>
                        <span>Email</span>
                        <input type="text" value="{{ user.email }}" disabled>
                    </label>
                </div>
                <button type="submit" class="btn-primary-action">Save Profile</button>
            </form>
        </div>

        <div class="card form-card">
            <div class="panel-header"><span>Settings</span></div>
            <form method="POST" action="{{ url_for('dashboard.update_settings') }}" class="stack-form">
                <div class="form-grid">
                    <label>
                        <span>Currency</span>
                        <select name="currency">
                            {% for code in ['INR', 'USD', 'EUR', 'GBP'] %}
                                <option value="{{ code }}" {% if settings.currency == code %}selected{% endif %}>{{ code
}}</option>
                            {% endfor %}
                        </select>
                    </label>
                    <label>
                        <span>Locale</span>
                        <input type="text" name="locale" value="{{ settings.locale or 'en-IN' }}">
                    </label>
                    <label>
                        <span>Language</span>
                        <select name="language">
                            {% for code, label in available_languages.items() %}
                                <option value="{{ code }}" {% if (settings.language or 'en') == code %}selected{% endif
%}>{{ label }}</option>
                            {% endfor %}
                        </select>
                    </label>
                    <label>
                        <span>Month Start Day</span>
                        <input type="number" name="month_start_day" min="1" max="28" value="{{ settings.month_start_day
or 1 }}">
                    </label>
                    <label>
                        <span>Monthly Salary</span>
                        <input type="number" name="monthly_salary" min="0" step="0.01" value="{{ settings.monthly_salary
or 0 }}">
                    </label>
                </div>
                <button type="submit" class="btn-primary-action">Save Settings</button>
            </form>
        </div>

        <div class="card form-card">
            <div class="panel-header"><span>Reset Password</span></div>
            <form method="POST" action="{{ url_for('dashboard.update_password') }}" class="stack-form">
                <div class="form-grid">
                    <label>
```

```
                        <span>Current Password</span>
                        <input type="password" name="current_password" required>
                    </label>
                    <label>
                        <span>New Password</span>
                        <input type="password" name="new_password" minlength="8" required>
                    </label>
                    <label>
                        <span>Confirm Password</span>
                        <input type="password" name="confirm_password" minlength="8" required>
                    </label>
                </div>
                <button type="submit" class="btn-primary-action">Update Password</button>
            </form>
        </div>
    </div>

    <div class="content-side">
        <div class="panel form-card">
            <div class="panel-header"><span>Add Account</span></div>
            <form method="POST" action="{{ url_for('dashboard.add_account') }}" class="stack-form">
                <label>
                    <span>Account Name</span>
                    <input type="text" name="account_name" placeholder="Salary Account" required>
                </label>
                <label>
                    <span>Type</span>
                    <select name="type">
                        <option value="Cash">Cash</option>
                        <option value="Bank">Bank</option>
                        <option value="UPI">UPI</option>
                    </select>
                </label>
                <label>
                    <span>Balance</span>
                    <input type="number" name="balance" min="0" step="0.01" value="0">
                </label>
                <button type="submit" class="btn-primary-action">Add Account</button>
            </form>
        </div>

        <div class="panel">
            <div class="panel-header"><span>Your Accounts</span></div>
            {% if accounts %}
                {% for account in accounts %}
                    <div class="list-row">
                        <div>
                            <strong>{{ account.account_name }}</strong>
                            <div class="helper-text">{{ account.type }}</div>
                        </div>
                        <div class="row-actions">
                            <span>{{ settings.currency or 'INR' }} {{ '%.2f'|format(account.balance) }}</span>
                            <form method="POST" action="{{ url_for('dashboard.delete_account', account_id=account.id)
}}">
                                <button type="submit" class="btn-link-danger">Delete</button>
                            </form>
                        </div>
                    </div>
                {% endfor %}
            {% else %}
                <p class="empty-text">No accounts added yet.</p>
            {% endif %}
        </div>
    </div>
</section>
{% endblock %}
```

## backend\app\templates\dashboard\admin.html

```
{% extends "dashboard/layout.html" %}

{% block title %}Admin | FinVest{% endblock %}
{% block page_title %}Admin Dashboard{% endblock %}
{% block page_subtitle %}View users, monitor system activity, and manage user access from one clean control panel.{%
endblock %}

{% block content %}
<section class="stats-grid admin-stats">
    <div class="card stat-card">
        <div class="card-title">Total Users</div>
        <div class="card-value">{{ total_users }}</div>
        <div class="helper-text">Registered users in the system</div>
    </div>
    <div class="card stat-card">
        <div class="card-title">Blocked Users</div>
        <div class="card-value warn">{{ blocked_users }}</div>
        <div class="helper-text">Users currently blocked from login</div>
    </div>
    <div class="card stat-card">
        <div class="card-title">Admins</div>
        <div class="card-value">{{ total_admins }}</div>
        <div class="helper-text">Admin accounts in the system</div>
    </div>
</section>

<section class="admin-charts">
    <div class="chart-card">
        <div class="panel-header"><span>System Overview</span></div>
        <div class="chart-stack">
            {% for item in overview_chart_rows %}
                <div class="chart-row">
                    <span class="chart-label">{{ item.label }}</span>
                    <div class="chart-track">
                        <div class="chart-fill {% if item.tone == 'blue' %}blue{% elif item.tone == 'green' %}green{%
elif item.tone == 'red' %}red{% endif %}" style="width: {{ item.width }}%"></div>
                    </div>
                    <span class="chart-value">{{ item.value }}</span>
                </div>
            {% endfor %}
        </div>
    </div>

    <div class="chart-card">
        <div class="panel-header">
            <span>Today at a Glance</span>
            <a href="{{ url_for('dashboard.admin_performance') }}" class="link">Performance</a>
        </div>
        <div class="mini-stats-grid">
            <div class="mini-stat">
                <span class="helper-text">Logins Today</span>
                <strong>{{ login_events_today }}</strong>
            </div>
            <div class="mini-stat">
                <span class="helper-text">New Users Today</span>
                <strong>{{ users_created_today }}</strong>
            </div>
            <div class="mini-stat">
                <span class="helper-text">Logouts Today</span>
                <strong>{{ logout_events_today }}</strong>
            </div>
            <div class="mini-stat">
                <span class="helper-text">Transactions Today</span>
                <strong>{{ transactions_today }}</strong>
            </div>
        </div>
    </div>
</section>

<section class="chart-card admin-spaced-section">
    <div class="panel-header"><span>Last 7 Days Activity</span></div>
    <div class="chart-stack">
        {% for item in daily_activity_rows %}
```

```
            <div class="chart-row">
                <span class="chart-label">{{ item.label }}</span>
                <div>
                    <div class="chart-track chart-track-spaced">
                        <div class="chart-fill blue" style="width: {{ item.login_width }}%"></div>
                    </div>
                    <div class="chart-track chart-track-spaced">
                        <div class="chart-fill green" style="width: {{ item.signup_width }}%"></div>
                    </div>
                </div>
                <span class="chart-value">{{ item.logins }}/{{ item.signups }}</span>
            </div>
        {% endfor %}
    </div>
    <p class="helper-text no-margin">Blue shows logins, green shows new registrations.</p>
</section>

<section class="content-grid admin-grid">
    <div class="content-main">
        <div class="panel">
            <div class="panel-header">
                <span>All Users</span>
            </div>
            <div class="table-wrap">
                <table class="data-table">
                    <thead>
                        <tr>
                            <th>Name</th>
                            <th>Email</th>
                            <th>Role</th>
                            <th>Status</th>
                            <th>Created</th>
                            <th>Actions</th>
                        </tr>
                    </thead>
                    <tbody>
                        {% for user in users %}
                            <tr>
                                <td>{{ user.name }}</td>
                                <td>{{ user.email }}</td>
                                <td><span class="type-chip">{{ user.role }}</span></td>
                                <td>
                                    <span class="type-chip {% if user.is_active %}income{% else %}expense{% endif %}">
                                        {{ 'Active' if user.is_active else 'Blocked' }}
                                    </span>
                                </td>
                                <td>{{ user.created_at.strftime('%d-%m-%Y %H:%M') if user.created_at else '-' }}</td>
                                <td>
                                    {% if user.email != session.get('email') %}
                                        <div class="row-actions admin-actions">
                                            <form method="POST" action="{{ url_for('dashboard.toggle_user_block',
user_id=user.id) }}">
                                                <button type="submit" class="btn-secondary-action">{{ 'Block' if
user.is_active else 'Unblock' }}</button>
                                            </form>
                                            <form method="POST" action="{{ url_for('dashboard.delete_user',
user_id=user.id) }}">
                                                <button type="submit" class="btn-link-danger">Delete</button>
                                            </form>
                                        </div>
                                    {% else %}
                                        <span class="helper-text">Admin account</span>
                                    {% endif %}
                                </td>
                            </tr>
                        {% else %}
                            <tr>
                                <td colspan="6">No users found.</td>
                            </tr>
                        {% endfor %}
                    </tbody>
                </table>
            </div>
        </div>

        <div class="panel">
```

```
            <div class="panel-header">
                <span>Monthly Category Transactions</span>
                <form method="GET" action="{{ url_for('dashboard.admin_dashboard') }}" class="inline-filter">
                    <select name="user_id">
                        <option value="">All Users</option>
                        {% for user in users %}
                            <option value="{{ user.id }}" {% if selected_user_id == user.id|string %}selected{% endif %}>
{{ user.name }}</option>
                        {% endfor %}
                    </select>
                    <button type="submit" class="btn-primary-action small">Filter</button>
                </form>
            </div>
            <div class="table-wrap scroll-panel">
                <table class="data-table">
                    <thead>
                        <tr>
                            <th>Category</th>
                            <th>Month</th>
                            <th>Income</th>
                            <th>Expense</th>
                            <th>Entries</th>
                        </tr>
                    </thead>
                    <tbody>
                        {% for item in monthly_category_rows %}
                            <tr>
                                <td>{{ item.category }}</td>
                                <td>{{ item.month }}</td>
                                <td>{{ item.income|money }}</td>
                                <td>{{ item.expense|money }}</td>
                                <td>{{ item.count }}</td>
                            </tr>
                        {% else %}
                            <tr>
                                <td colspan="5">No transaction summary found for the selected filter.</td>
                            </tr>
                        {% endfor %}
                    </tbody>
                </table>
            </div>
        </div>
    </div>

    <div class="content-side">
        <div class="panel">
            <div class="panel-header"><span>User Activity</span></div>
            {% if activity_logs %}
                <div class="activity-list scroll-panel">
                    {% for item in activity_logs %}
                        <div class="activity-row">
                            <div class="activity-header">
                                <strong>{{ item.user_name }}</strong>
                                <span class="helper-text">{{ item.created_at_label }}</span>
                            </div>
                            <div class="activity-body">
                                <span class="type-chip">{{ item.action.replace('_', ' ') }}</span>
                                <p class="helper-text no-margin">{{ item.details or item.user_email }}</p>
                            </div>
                        </div>
                    {% endfor %}
                </div>
            {% else %}
                <p class="empty-text">No activity found.</p>
            {% endif %}
        </div>
    </div>
</section>
{% endblock %}
```

## backend\app\templates\dashboard\admin_performance.html

```
{% extends "dashboard/layout.html" %}

{% block title %}Performance | FinVest{% endblock %}
{% block page_title %}System Performance{% endblock %}
{% block page_subtitle %}Monitor system load, database responsiveness, and activity growth for developer handoff.{%
endblock %}

{% block content %}
<section class="stats-grid admin-stats">
    <div class="card stat-card">
        <div class="card-title">Active Logins</div>
        <div class="card-value" id="activeUsersValue">{{ active_users }}</div>
        <div class="helper-text">Currently logged-in sessions</div>
    </div>
    <div class="card stat-card">
        <div class="card-title">DB Response</div>
        <div class="card-value">{{ db_response_ms }} ms</div>
        <div class="helper-text">Measured with a lightweight user count query</div>
    </div>
    <div class="card stat-card">
        <div class="card-title">Last Request</div>
        <div class="card-value">{{ last_response_ms }} ms</div>
        <div class="helper-text">Latest Flask request processing time</div>
    </div>
</section>

<section class="admin-charts">
    <div class="chart-card">
        <div class="panel-header"><span>System Health Metrics</span></div>
        <div class="chart-stack">
            {% for item in performance_rows %}
                <div class="chart-row">
                    <span class="chart-label">{{ item.label }}</span>
                    <div class="chart-track">
                        <div class="chart-fill {% if item.tone == 'blue' %}blue{% elif item.tone == 'green' %}green{%
elif item.tone == 'red' %}red{% endif %}" style="width: {{ item.width }}%"></div>
                    </div>
                    <span class="chart-value">{{ item.value }}</span>
                </div>
            {% endfor %}
        </div>
    </div>

    <div class="chart-card">
        <div class="panel-header"><span>System Volume</span></div>
        <div class="chart-stack">
            {% for item in growth_rows %}
                <div class="chart-row">
                    <span class="chart-label">{{ item.label }}</span>
                    <div class="chart-track">
                        <div class="chart-fill {% if item.tone == 'blue' %}blue{% elif item.tone == 'green' %}green{%
endif %}" style="width: {{ item.width }}%"></div>
                    </div>
                    <span class="chart-value">{{ item.value }}</span>
                </div>
            {% endfor %}
        </div>
    </div>
</section>

<section class="panel">
    <div class="panel-header">
        <span>Developer Notes</span>
        <a href="{{ url_for('dashboard.admin_dashboard') }}" class="link">Back to Admin</a>
    </div>
    <p class="helper-text no-margin">Use rising request time, database response time, and activity-log growth as early
signals for optimization or indexing work.</p>
</section>
<script>
    const activeUsersValue = document.getElementById("activeUsersValue");

    async function refreshActiveUsers() {
        try {
```

```
            const response = await fetch("{{ url_for('dashboard.admin_active_users') }}", {
                headers: {"Accept": "application/json"}
            });
            if (!response.ok) return;
            const data = await response.json();
            activeUsersValue.textContent = data.active_users;
        } catch (error) {
            return;
        }
    }

    setInterval(refreshActiveUsers, 5000);
</script>
{% endblock %}
```

## backend\app\templates\dashboard\budgets.html

```
{% extends "dashboard/layout.html" %}

{% block title %}Budgets | FinVest{% endblock %}
{% block page_title %}Budget Planning{% endblock %}
{% block page_subtitle %}Set monthly category limits and compare them with your actual spending for the selected month.{%
endblock %}

{% block content %}
<section class="content-grid">
    <div class="content-main">
        <div class="panel">
            <div class="panel-header">
                <span>Current Budgets</span>
                <form method="GET" action="{{ url_for('budgets.budget_list') }}" class="inline-filter">
                    <input type="number" name="month" min="1" max="12" value="{{ month }}">
                    <input type="number" name="year" min="2024" max="2100" value="{{ year }}">
                    <button type="submit" class="btn-primary-action small">Load</button>
                </form>
            </div>

            {% if budgets %}
                <div class="budget-list">
                    {% for item in budgets %}
                        <div class="budget-card-row">
                            <div class="budget-card-header">
                                <div>
                                    <strong>{{ item.budget.category.name }}</strong>
                                    <div class="helper-text">{{ month }}/{{ year }}</div>
                                </div>
                                <form method="POST" action="{{ url_for('budgets.delete_budget', budget_id=item.budget.id)
}}">
                                    <button type="submit" class="btn-link-danger">Delete</button>
                                </form>
                            </div>
                            <div class="budget-amounts">
                                <span>Spent: {{ settings.currency }} {{ '%.2f'|format(item.spent) }}</span>
                                <span>Limit: {{ settings.currency }} {{ '%.2f'|format(item.budget.limit_amount) }}</span>
                                <span class="{% if item.percent > 100 %}warn{% elif item.percent >= 80 %}link{% else
%}up{% endif %}">
                                    {{ item.percent }}%
                                </span>
                            </div>
                            <div class="bar tall {% if item.percent > 100 %}red{% endif %}">
                                <div class="{% if item.percent > 100 %}bar-expense{% else %}bar-income{% endif %}"
style="width: {{ 100 if item.percent > 100 else item.percent }}%"></div>
                            </div>
                        </div>
                    {% endfor %}
                </div>
            {% else %}
                <p class="empty-text">No budgets added for this month yet.</p>
            {% endif %}
        </div>

        <div class="panel">
            <div class="panel-header"><span>Saved Recurring Expenses</span></div>
            {% if recurring_expenses %}
                <div class="budget-list">
                    {% for item in recurring_expenses %}
                        <div class="budget-card-row">
                            <div class="budget-card-header">
                                <div>
                                    <strong>{{ item.description or item.category.name }}</strong>
                                    <div class="helper-text">{{ item.frequency|capitalize }} | Next: {{
item.next_run_date.strftime('%d %b %Y') }}</div>
                                </div>
                                <span class="type-chip {% if item.is_active %}income{% else %}expense{% endif %}">
                                    {{ 'Active' if item.is_active else 'Paused' }}
                                </span>
                            </div>
                            <div class="budget-amounts">
                                <span>{{ settings.currency }} {{ '%.2f'|format(item.amount) }}</span>
                                <span>{{ item.category.name }}</span>
```

```
                                <span>{{ item.account.account_name }}</span>
                            </div>
                            <div class="row-actions">
                                <form method="POST" action="{{ url_for('budgets.toggle_recurring_expense',
recurring_id=item.id) }}">
                                    <button type="submit" class="btn-secondary-action small">{{ 'Pause' if item.is_active
else 'Resume' }}</button>
                                </form>
                                <form method="POST" action="{{ url_for('budgets.delete_recurring_expense',
recurring_id=item.id) }}">
                                    <button type="submit" class="btn-link-danger">Delete</button>
                                </form>
                            </div>
                        </div>
                    {% endfor %}
                </div>
            {% else %}
                <p class="empty-text">No recurring expenses saved yet.</p>
            {% endif %}
        </div>
    </div>

    <div class="content-side">
        <div class="panel form-card">
            <div class="panel-header"><span>Add Budget</span></div>
            <form method="POST" action="{{ url_for('budgets.add_budget') }}" class="stack-form">
                <label>
                    <span>Category</span>
                    <select name="category_id" required>
                        {% for category in categories %}
                            <option value="{{ category.id }}">{{ category.name }}</option>
                        {% endfor %}
                    </select>
                </label>
                <label>
                    <span>Limit Amount</span>
                    <input type="number" name="limit_amount" min="0" step="0.01" required>
                </label>
                <label>
                    <span>Month</span>
                    <input type="number" name="month" min="1" max="12" value="{{ month }}" required>
                </label>
                <label>
                    <span>Year</span>
                    <input type="number" name="year" min="2024" max="2100" value="{{ year }}" required>
                </label>
                <button type="submit" class="btn-primary-action">Save Budget</button>
            </form>
        </div>

        <div class="panel form-card">
            <div class="panel-header"><span>Save Recurring Expense</span></div>
            <form method="POST" action="{{ url_for('budgets.add_recurring_expense') }}" class="stack-form">
                <label>
                    <span>Account</span>
                    <select name="account_id" required>
                        {% for account in accounts %}
                            <option value="{{ account.id }}">{{ account.account_name }}</option>
                        {% endfor %}
                    </select>
                </label>
                <label>
                    <span>Category</span>
                    <select name="category_id" required>
                        {% for category in categories %}
                            <option value="{{ category.id }}">{{ category.name }}</option>
                        {% endfor %}
                    </select>
                </label>
                <label>
                    <span>Amount</span>
                    <input type="number" name="amount" min="0.01" step="0.01" required>
                </label>
                <label>
                    <span>Frequency</span>
                    <select name="frequency">
```

```
                        <option value="monthly">Monthly</option>
                        <option value="daily">Daily</option>
                        <option value="yearly">Yearly</option>
                    </select>
                </label>
                <label>
                    <span>Start Date</span>
                    <input type="date" name="start_date" required>
                </label>
                <label>
                    <span>Description</span>
                    <input type="text" name="description" placeholder="Internet bill">
                </label>
                <button type="submit" class="btn-primary-action">Save Recurring Expense</button>
            </form>
        </div>
    </div>
</section>
{% endblock %}
```

## backend\app\templates\dashboard\categories.html

```
{% extends "dashboard/layout.html" %}

{% block title %}Categories | FinVest{% endblock %}
{% block page_title %}Category Management{% endblock %}
{% block page_subtitle %}Use the default income and expense categories or add your own simple custom categories.{%
endblock %}

{% block content %}
<section class="content-grid single-page">
    <div class="content-main">
        <div class="panel">
            <div class="panel-header"><span>Available Categories</span></div>
            <div class="table-wrap">
                <table class="data-table">
                    <thead>
                        <tr>
                            <th>Name</th>
                            <th>Type</th>
                            <th>Action</th>
                        </tr>
                    </thead>
                    <tbody>
                        {% for item in categories %}
                            <tr>
                                <td>{{ item.name }}</td>
                                <td><span class="type-chip {{ item.type }}">{{ item.type }}</span></td>
                                <td>
                                    <form method="POST" action="{{ url_for('categories.delete_category',
category_id=item.id) }}">
                                        <button type="submit" class="btn-link-danger">Delete</button>
                                    </form>
                                </td>
                            </tr>
                        {% else %}
                            <tr>
                                <td colspan="3">No categories available.</td>
                            </tr>
                        {% endfor %}
                    </tbody>
                </table>
            </div>
        </div>
    </div>

    <div class="content-side">
        <div class="panel form-card">
            <div class="panel-header"><span>Add Category</span></div>
            <form method="POST" action="{{ url_for('categories.add_category') }}" class="stack-form">
                <label>
                    <span>Name</span>
                    <input type="text" name="name" placeholder="Internet Bill" required>
                </label>
                <label>
                    <span>Type</span>
                    <select name="type">
                        <option value="expense">Expense</option>
                        <option value="income">Income</option>
                    </select>
                </label>
                <button type="submit" class="btn-primary-action">Add Category</button>
            </form>
        </div>
    </div>
</section>
{% endblock %}
```

## backend\app\templates\dashboard\goals.html

```
{% extends "dashboard/layout.html" %}

{% block title %}Goals | FinVest{% endblock %}
{% block page_title %}Saving and Goals{% endblock %}
{% block page_subtitle %}Create personal saving goals, update progress, and track how much is still left to achieve them.
{% endblock %}

{% block content %}
<section class="content-grid">
    <div class="content-main">
        <div class="panel">
            <div class="panel-header"><span>Your Goals</span></div>
            {% if goals %}
                <div class="goal-list">
                    {% for item in goals %}
                        <div class="goal-card">
                            <div class="goal-card-top">
                                <div>
                                    <strong>{{ item.goal.goal_name }}</strong>
                                    <div class="helper-text">
                                        {% if item.goal.deadline %}Deadline: {{ item.goal.deadline.strftime('%d %b %Y')
}}{% else %}No deadline{% endif %}
                                    </div>
                                </div>
                                <span class="type-chip">{{ item.goal.status }}</span>
                            </div>

                            <div class="budget-amounts">
                                <span>Saved: {{ settings.currency }} {{ '%.2f'|format(item.goal.current_amount) }}</span>
                                <span>Target: {{ settings.currency }} {{ '%.2f'|format(item.goal.target_amount) }}</span>
                                <span>Left: {{ settings.currency }} {{ '%.2f'|format(item.remaining) }}</span>
                            </div>

                            <div class="bar tall">
                                <div class="bar-income" style="width: {{ item.percent }}%"></div>
                            </div>
                            <div class="helper-text">{{ item.percent }}% completed</div>

                            <form method="POST" action="{{ url_for('goals.add_goal_amount', goal_id=item.goal.id) }}"
class="inline-amount-form">
                                <input type="number" name="amount_to_add" min="0.01" step="0.01" placeholder="Add saved
amount" required>
                                <button type="submit" class="btn-primary-action small">Add Amount</button>
                            </form>

                            <details class="inline-editor">
                                <summary>Edit Goal</summary>
                                <form method="POST" action="{{ url_for('goals.update_goal', goal_id=item.goal.id) }}"
class="stack-form compact-form">
                                    <input type="text" name="goal_name" value="{{ item.goal.goal_name }}" required>
                                    <input type="number" name="target_amount" min="0" step="0.01" value="{{
item.goal.target_amount }}" required>
                                    <input type="number" name="current_amount" min="0" step="0.01" value="{{
item.goal.current_amount }}" required>
                                    <input type="date" name="deadline" value="{{ item.goal.deadline.isoformat() if
item.goal.deadline else '' }}">
                                    <select name="status">
                                        {% for status in ['In Progress', 'Completed', 'Paused'] %}
                                            <option value="{{ status }}" {% if item.goal.status == status %}selected{%
endif %}>{{ status }}</option>
                                        {% endfor %}
                                    </select>
                                    <button type="submit" class="btn-primary-action small">Save</button>
                                </form>
                            </details>

                            <form method="POST" action="{{ url_for('goals.delete_goal', goal_id=item.goal.id) }}">
                                <button type="submit" class="btn-link-danger">Delete Goal</button>
                            </form>
                        </div>
                    {% endfor %}
                </div>
            {% else %}
```

```
                <p class="empty-text">No goals added yet.</p>
            {% endif %}
        </div>
    </div>

    <div class="content-side">
        <div class="panel form-card">
            <div class="panel-header"><span>Add Goal</span></div>
            <form method="POST" action="{{ url_for('goals.add_goal') }}" class="stack-form">
                <label>
                    <span>Goal Name</span>
                    <input type="text" name="goal_name" placeholder="Emergency Fund" required>
                </label>
                <label>
                    <span>Target Amount</span>
                    <input type="number" name="target_amount" min="0" step="0.01" required>
                </label>
                <label>
                    <span>Current Amount</span>
                    <input type="number" name="current_amount" min="0" step="0.01" value="0">
                </label>
                <p class="helper-text no-margin">Same goal name with the same target amount will not be added twice.</p>
                <label>
                    <span>Deadline</span>
                    <input type="date" name="deadline">
                </label>
                <label>
                    <span>Status</span>
                    <select name="status">
                        <option value="In Progress">In Progress</option>
                        <option value="Completed">Completed</option>
                        <option value="Paused">Paused</option>
                    </select>
                </label>
                <button type="submit" class="btn-primary-action">Add Goal</button>
            </form>
        </div>
    </div>
</section>
{% endblock %}
```

## backend\app\templates\dashboard\home.html

```
{% extends "dashboard/layout.html" %}

{% block title %}Dashboard | FinVest{% endblock %}
{% block page_title %}Dashboard{% endblock %}
{% block page_subtitle %}Your finance summary, recent activity, and spending view for this month.{% endblock %}

{% block content %}
{% if show_guide %}
<section class="panel guide-panel">
    <div class="panel-header">
        <span>Quick Guide</span>
        <form method="POST" action="{{ url_for('dashboard.dismiss_guide') }}">
            <button type="submit" class="btn-link-danger">Hide</button>
        </form>
    </div>
    <div class="guide-timeline">
        <a href="{{ url_for('transactions.transaction_list') }}" class="guide-step">
            <span class="guide-dot">1</span>
            <strong>Record money movement</strong>
            <small>Add income and expenses so balances, reports, and alerts stay current.</small>
        </a>
        <a href="{{ url_for('budgets.budget_list') }}" class="guide-step">
            <span class="guide-dot">2</span>
            <strong>Control repeat spending</strong>
            <small>Set budgets and save recurring bills for automatic deductions.</small>
        </a>
        <a href="{{ url_for('reports.report_list') }}" class="guide-step">
            <span class="guide-dot">3</span>
            <strong>Review patterns</strong>
            <small>Use charts and KPIs to see trends before they become surprises.</small>
        </a>
    </div>
</section>
{% else %}
<section class="guide-compact">
    <span>Quick guide is hidden.</span>
    <form method="POST" action="{{ url_for('dashboard.show_guide') }}">
        <button type="submit" class="btn-secondary-action small">Show Guide</button>
    </form>
</section>
{% endif %}
<section class="stats-grid">
    <div class="card stat-card">
        <div class="card-title">Total Balance</div>
        <div class="card-value">{{ currency_symbol }} {{ '%.2f'|format(total_balance) }}</div>
        <div class="helper-text">Across all your accounts</div>
    </div>
    <div class="card stat-card">
        <div class="card-title">Monthly Income</div>
        <div class="card-value up">{{ currency_symbol }} {{ '%.2f'|format(monthly_income) }}</div>
        <div class="helper-text">Income added this month</div>
    </div>
    <div class="card stat-card">
        <div class="card-title">Monthly Expense</div>
        <div class="card-value warn">{{ currency_symbol }} {{ '%.2f'|format(monthly_expense) }}</div>
        <div class="helper-text">Expenses added this month</div>
    </div>
    <div class="card stat-card">
        <div class="card-title">Average Monthly Spending</div>
        <div class="card-value">{{ currency_symbol }} {{ '%.2f'|format(average_monthly_expense) }}</div>
        <div class="helper-text">Based on your recent spending months</div>
    </div>
</section>

<section class="content-grid">
    <div class="content-main">
        <div class="card">
            <div class="panel-header">
                <span>Monthly Income vs Expense</span>
            </div>
            <div class="compare-list">
                {% for item in monthly_stats %}
                    <div class="compare-row">
```

```
                        <div class="compare-label">{{ item.label }}</div>
                        <div class="compare-bars">
                            <div class="compare-bar-group">
                                <span class="mini-label">Income</span>
                                <div class="bar tall"><div class="bar-income" style="width: {{ item.income_width }}%">
</div></div>
                            </div>
                            <div class="compare-bar-group">
                                <span class="mini-label">Expense</span>
                                <div class="bar tall red"><div class="bar-expense" style="width: {{ item.expense_width
}}%"></div></div>
                            </div>
                        </div>
                        <div class="compare-values">
                            <span>{{ currency_symbol }} {{ '%.0f'|format(item.income) }}</span>
                            <span>{{ currency_symbol }} {{ '%.0f'|format(item.expense) }}</span>
                        </div>
                    </div>
                {% endfor %}
            </div>
        </div>

        <div class="card">
            <div class="panel-header">
                <span>Category-wise Spending</span>
            </div>
            <div class="chart-layout">
                <div class="pie-wrap">
                    <div class="pie-chart" style="{{ pie_chart_style }}">
                        <div class="pie-chart-hole">
                            <strong>{{ top_spending_category or 'No Data' }}</strong>
                            <span>Top category</span>
                        </div>
                    </div>
                </div>
                <div class="legend-list">
                    {% if chart_legend %}
                        {% for item in chart_legend %}
                            <div class="legend-item">
                                <span class="legend-dot" style="background: {{ item.color }}"></span>
                                <div class="legend-text">
                                    <strong>{{ item.name }}</strong>
                                    <small>{{ currency_symbol }} {{ '%.2f'|format(item.amount) }} | {{ item.percent }}%
</small>
                                </div>
                            </div>
                        {% endfor %}
                    {% else %}
                        <p class="empty-text">Add some expense transactions to see the category chart.</p>
                    {% endif %}
                </div>
            </div>
        </div>

        <div class="card">
            <div class="panel-header">
                <span>Budget vs Actual Spending</span>
                <a href="{{ url_for('budgets.budget_list') }}" class="link">Manage Budgets</a>
            </div>
            {% if budget_rows %}
                <div class="budget-list">
                    {% for item in budget_rows %}
                        <div class="budget-card-row">
                            <div class="budget-card-header">
                                <strong>{{ item.budget.category.name }}</strong>
                                <span class="{% if item.percent > 100 %}warn{% elif item.percent >= 80 %}link{% else
%}up{% endif %}">
                                    {{ item.percent }}%
                                </span>
                            </div>
                            <div class="budget-amounts">
                                <span>Spent: {{ currency_symbol }} {{ '%.2f'|format(item.spent) }}</span>
                                <span>Limit: {{ currency_symbol }} {{ '%.2f'|format(item.budget.limit_amount) }}</span>
                            </div>
                            <div class="bar tall {% if item.percent > 100 %}red{% endif %}">
                                <div class="{% if item.percent > 100 %}bar-expense{% else %}bar-income{% endif %}"
```

```
style="width: {{ 100 if item.percent > 100 else item.percent }}%"></div>
                            </div>
                        </div>
                    {% endfor %}
                </div>
            {% else %}
                <p class="empty-text">Add monthly budgets to compare them with current spending.</p>
            {% endif %}
        </div>
    </div>

    <div class="content-side">
        <div class="panel">
            <div class="panel-header">
                <span>Recent Transactions</span>
                <a href="{{ url_for('transactions.transaction_list') }}" class="link">View All</a>
            </div>
            {% if recent_transactions %}
                {% for item in recent_transactions %}
                    <div class="tx">
                        <div>
                            <strong>{{ item.category.name }}</strong>
                            <div class="helper-text">{{ item.date.strftime('%d %b %Y') }}{% if item.description %} | {{
item.description }}{% endif %}</div>
                        </div>
                        <span class="{% if item.type == 'income' %}up{% else %}warn{% endif %}">
                            {% if item.type == 'income' %}+{% else %}-{% endif %}{{ currency_symbol }} {{
'%.2f'|format(item.amount) }}
                        </span>
                    </div>
                {% endfor %}
            {% else %}
                <p class="empty-text">No transactions added yet.</p>
            {% endif %}
        </div>

        <div class="panel">
            <div class="panel-header">
                <span>Accounts</span>
                <a href="{{ url_for('dashboard.account') }}" class="link">Manage</a>
            </div>
            {% if accounts %}
                {% for account in accounts[:4] %}
                    <div class="tx">
                        <span>{{ account.account_name }}</span>
                        <span>{{ currency_symbol }} {{ '%.2f'|format(account.balance) }}</span>
                    </div>
                {% endfor %}
            {% else %}
                <p class="empty-text">No accounts added yet.</p>
            {% endif %}
        </div>

        <div class="panel">
            <div class="panel-header">
                <span>Top Saving Goal</span>
            </div>
            {% if top_goal %}
                <div class="goal">
                    <div class="circle">{{ goal_percent }}%</div>
                    <div>
                        <div>{{ top_goal.goal_name }}</div>
                        <small>{{ currency_symbol }} {{ '%.2f'|format(top_goal.current_amount) }} / {{ currency_symbol }}
{{ '%.2f'|format(top_goal.target_amount) }}</small>
                    </div>
                </div>
            {% else %}
                <p class="empty-text">No goals added yet.</p>
            {% endif %}
        </div>
    </div>
</section>
{% endblock %}
```

## backend\app\templates\dashboard\layout.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}FinVest{% endblock %}</title>
    <link rel="stylesheet" href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&display=swap">
    <link rel="stylesheet" href="{{ url_for('static', filename='css/dashboard.css') }}">
</head>
<body>
    <aside class="sidebar">
        <div class="logo">Fin<span>vest</span></div>
        <nav class="nav">
            {% if session.get('role') == 'admin' and session.get('email') == 'admin@gmail.com' %}
                <a href="{{ url_for('dashboard.admin_dashboard') }}" class="nav-item {% if active_page == 'admin'
%}active{% endif %}">Admin</a>
            {% else %}
                <a href="{{ url_for('dashboard.home') }}" class="nav-item {% if active_page == 'dashboard' %}active{%
endif %}">Dashboard</a>
                <a href="{{ url_for('transactions.transaction_list') }}" class="nav-item {% if active_page ==
'transactions' %}active{% endif %}">Transactions</a>
                <a href="{{ url_for('categories.category_list') }}" class="nav-item {% if active_page == 'categories'
%}active{% endif %}">Categories</a>
                <a href="{{ url_for('budgets.budget_list') }}" class="nav-item {% if active_page == 'budgets' %}active{%
endif %}">Budgets</a>
                <a href="{{ url_for('goals.goal_list') }}" class="nav-item {% if active_page == 'goals' %}active{% endif
%}">Goals</a>
                <a href="{{ url_for('reports.report_list') }}" class="nav-item {% if active_page == 'reports' %}active{%
endif %}">Reports</a>
            {% endif %}
        </nav>
    </aside>

    <div class="main">
        <header class="header">
            <div>
                <h1>{% block page_title %}FinVest{% endblock %}</h1>
                <p class="page-subtitle">{% block page_subtitle %}{% endblock %}</p>
            </div>
            <div class="header-right">
                <a href="{{ url_for('notifications.notification_list') }}" class="icon-button" aria-
label="Notifications">
                    <span class="bell-icon">&#128276;</span>
                    {% if unread_notification_count %}
                        <span class="icon-badge">{{ unread_notification_count }}</span>
                    {% endif %}
                </a>
                <details class="profile-menu">
                    <summary class="profile-chip">
                        <div class="avatar">{{ session.get('username', 'U')[:1] }}</div>
                        <div class="profile-meta">
                            <strong>{{ session.get('username', 'User') }}</strong>
                            <span>Profile</span>
                        </div>
                    </summary>
                    <div class="profile-dropdown">
                        <a href="{{ url_for('dashboard.account') }}" class="profile-dropdown-link">Account Settings</a>
                        <a href="{{ url_for('auth.logout') }}" class="profile-dropdown-link danger-link">Logout</a>
                    </div>
                </details>
            </div>
        </header>

        {% with messages = get_flashed_messages() %}
            {% if messages %}
                <div class="flash-stack">
                    {% for message in messages %}
                        <div class="flash-banner">{{ message }}</div>
                    {% endfor %}
                </div>
            {% endif %}
        {% endwith %}
```

```
        <main class="page-content">
            {% block content %}{% endblock %}
        </main>
    </div>
    <script>
        window.FINVEST_DISPLAY = {
            currency: "{{ display_currency }}",
            symbol: "{{ display_currency_symbol }}",
            rate: {{ display_currency_rates.get(display_currency, 1)|tojson }},
            language: "{{ display_language }}"
        };

        const translations = {
            hi: {
                "Dashboard": "डैशबोर्ड", "Transactions": "लेन-देन", "Categories": "श्रेणियां", "Budgets": "बजट", "Goals": "लक्ष्य",
"Reports": "रिपोर्ट", "Admin": "एडमिन",
                "Account Settings": "खाता सेटिंग", "Logout": "लॉगआउट", "Total Balance": "कुल शेष", "Monthly Income":
"मासिक आय", "Monthly Expense": "मासिक खर्च",
                "Average Monthly Spending": "औसत मासिक खर्च", "Recent Transactions": "हाल के लेन-देन", "Accounts": "खाते",
"Top Saving Goal": "मुख्य बचत लक्ष्य",
                "Filter Transactions": "लेन-देन फिल्टर", "Transaction History": "लेन-देन इतिहास", "Add Transaction": "लेन-देन
जोड़ें", "Type": "प्रकार", "Category": "श्रेणी",
                "Account": "खाता", "Amount": "राशि", "Date": "तारीख", "Description": "विवरण", "Actions": "कार्रवाई",
"Income": "आय", "Expense": "खर्च",
                "Add Budget": "बजट जोड़ें", "Current Budgets": "वर्तमान बजट", "Saved Recurring Expenses": "सहेजे गए आवर्ती
खर्च", "Save Recurring Expense": "आवर्ती खर्च सहेजें",
                "Your Goals": "आपके लक्ष्य", "Add Goal": "लक्ष्य जोड़ें", "Reports and Analytics": "रिपोर्ट और विश्लेषण", "Category
Analysis": "श्रेणी विश्लेषण",
                "Monthly Summary": "मासिक सारांश", "Financial Trends": "वित्तीय रुझान", "Income Source Mix": "आय स्रोत मिश्रण",
"Export and Backup": "निर्यात और बैकअप",
                "Profile": "प्रोफाइल", "Settings": "सेटिंग", "Language": "भाषा", "Reset Password": "पासवर्ड रीसेट", "Update
Password": "पासवर्ड अपडेट करें",
                "System Performance": "सिस्टम प्रदर्शन", "All Users": "सभी उपयोगकर्ता", "User Activity": "उपयोगकर्ता गतिविधि"
            },
            mr: {
                "Dashboard": "डॅशबोर्ड", "Transactions": "व्यवहार", "Categories": "वर्ग", "Budgets": "बजेट", "Goals": "ध्येये",
"Reports": "अहवाल", "Admin": "प्रशासक",
                "Account Settings": "खाते सेटिंग", "Logout": "लॉगआउट", "Total Balance": "एकूण शिल्लक", "Monthly Income":
"मासिक उत्पन्न", "Monthly Expense": "मासिक खर्च",
                "Average Monthly Spending": "सरासरी मासिक खर्च", "Recent Transactions": "अलीकडील व्यवहार", "Accounts":
"खाती", "Top Saving Goal": "मुख्य बचत ध्येय",
                "Filter Transactions": "व्यवहार फिल्टर", "Transaction History": "व्यवहार इतिहास", "Add Transaction": "व्यवहार
जोडा", "Type": "प्रकार", "Category": "वर्ग",
                "Account": "खाते", "Amount": "रक्कम", "Date": "तारीख", "Description": "वर्णन", "Actions": "कृती", "Income":
"उत्पन्न", "Expense": "खर्च",
                "Add Budget": "बजेट जोडा", "Current Budgets": "सध्याचे बजेट", "Saved Recurring Expenses": "जतन केलेले आवर्ती
खर्च", "Save Recurring Expense": "आवर्ती खर्च जतन करा",
                "Your Goals": "तुमची ध्येये", "Add Goal": "ध्येय जोडा", "Reports and Analytics": "अहवाल आणि विश्लेषण", "Category
Analysis": "वर्ग विश्लेषण",
                "Monthly Summary": "मासिक सारांश", "Financial Trends": "आर्थिक कल", "Income Source Mix": "उत्पन्न स्रोत मिश्रण",
"Export and Backup": "निर्यात आणि बॅकअप",
                "Profile": "प्रोफाइल", "Settings": "सेटिंग", "Language": "भाषा", "Reset Password": "पासवर्ड रीसेट", "Update
Password": "पासवर्ड अपडेट करा",
                "System Performance": "सिस्टम कामगिरी", "All Users": "सर्व वापरकर्ते", "User Activity": "वापरकर्ता क्रियाकलाप"
            }
        };

        function convertCurrencyText(text) {
            const display = window.FINVEST_DISPLAY;
            if (display.currency === "INR") return text;
            return text.replace(/(?:Rs\.|INR|USD|EUR|GBP|\$)\s*(-?\d+(?:,\d{3})*(?:\.\d+)?)/g, (_, amount) => {
                const converted = Number(amount.replace(/,/g, "")) * display.rate;
                return `${display.symbol} ${converted.toFixed(2)}`;
            });
        }

        function translateText(text) {
            const dictionary = translations[window.FINVEST_DISPLAY.language] || {};
            const trimmed = text.trim();
            return dictionary[trimmed] ? text.replace(trimmed, dictionary[trimmed]) : text;
        }

        function walkText(node) {
            if (node.nodeType === Node.TEXT_NODE) {
                node.nodeValue = translateText(convertCurrencyText(node.nodeValue));
                return;
```

```
            }
            if (node.nodeType !== Node.ELEMENT_NODE || ["SCRIPT", "STYLE"].includes(node.tagName)) return;
            ["placeholder", "title", "value"].forEach((attr) => {
                if (node.hasAttribute && node.hasAttribute(attr)) {
                    node.setAttribute(attr, translateText(convertCurrencyText(node.getAttribute(attr))));
                }
            });
            node.childNodes.forEach(walkText);
        }

        function paintReportPies() {
            const colors = ["#C9A84C", "#4f8ef7", "#5ECB8C", "#e07070", "#8A8F9E", "#8090f0"];
            document.querySelectorAll(".report-pie").forEach((pie) => {
                const rows = JSON.parse(pie.dataset.pie || "[]").slice(0, 6);
                let start = 0;
                const segments = rows.map((row, index) => {
                    const end = start + (Number(row.percent || 0) * 3.6);
                    const segment = `${colors[index % colors.length]} ${start.toFixed(2)}deg ${end.toFixed(2)}deg`;
                    start = end;
                    return segment;
                });
                pie.style.background = segments.length
                    ? `conic-gradient(from -90deg, ${segments.join(", ")}, #1A2035 ${start.toFixed(2)}deg 360deg)`
                    : "conic-gradient(from -90deg, #1A2035 0deg 360deg)";
            });
        }

        walkText(document.body);
        paintReportPies();
    </script>
</body>
</html>
```

## backend\app\templates\dashboard\notifications.html

```
{% extends "dashboard/layout.html" %}

{% block title %}Notifications | FinVest{% endblock %}
{% block page_title %}Notifications{% endblock %}
{% block page_subtitle %}Track budget alerts, goal reminders, and important account messages.{% endblock %}

{% block content %}
<section class="content-grid single-page">
    <div class="content-main">
        <div class="panel">
            <div class="panel-header">
                <span>All Notifications</span>
                <form method="POST" action="{{ url_for('notifications.mark_all_notifications_read') }}">
                    <button type="submit" class="btn-primary-action small">Mark All Read</button>
                </form>
            </div>

            {% if notifications %}
                <div class="activity-list">
                    {% for item in notifications %}
                        <div class="activity-row {% if not item.is_read %}unread-card{% endif %}">
                            <div class="activity-header">
                                <strong>{{ item.title }}</strong>
                                <span class="helper-text">{{ item.created_at.strftime('%d %b %Y %I:%M %p') if
item.created_at else '-' }}</span>
                            </div>
                            <div class="activity-body">
                                <p class="helper-text no-margin">{{ item.message }}</p>
                                <div class="row-actions">
                                    <span class="type-chip">{{ item.notification_type }}</span>
                                    {% if not item.is_read %}
                                        <form method="POST" action="{{ url_for('notifications.mark_notification_read',
notification_id=item.id) }}">
                                            <button type="submit" class="btn-secondary-action">Mark Read</button>
                                        </form>
                                    {% endif %}
                                </div>
                            </div>
                        </div>
                    {% endfor %}
                </div>
            {% else %}
                <p class="empty-text">No notifications available right now.</p>
            {% endif %}
        </div>
    </div>
</section>
{% endblock %}
```

## backend\app\templates\dashboard\reports.html

```
{% extends "dashboard/layout.html" %}

{% block title %}Reports | FinVest{% endblock %}
{% block page_title %}Reports and Analytics{% endblock %}
{% block page_subtitle %}View spending patterns, compare income vs expense, and export your data when needed.{% endblock
%}

{% block content %}
<section class="stats-grid admin-stats">
    <div class="card stat-card">
        <div class="card-title">Total Income</div>
        <div class="card-value up">{{ settings.currency }} {{ '%.2f'|format(total_income) }}</div>
    </div>
    <div class="card stat-card">
        <div class="card-title">Total Expense</div>
        <div class="card-value warn">{{ settings.currency }} {{ '%.2f'|format(total_expense) }}</div>
    </div>
    <div class="card stat-card">
        <div class="card-title">Net</div>
        <div class="card-value">{{ settings.currency }} {{ '%.2f'|format(net) }}</div>
    </div>
    <div class="card stat-card">
        <div class="card-title">Avg Monthly Spending</div>
        <div class="card-value">{{ settings.currency }} {{ '%.2f'|format(average_monthly_spending) }}</div>
    </div>
    <div class="card stat-card">
        <div class="card-title">Savings Rate</div>
        <div class="card-value {% if savings_rate >= 0 %}up{% else %}warn{% endif %}">{{ '%.1f'|format(savings_rate) }}%
</div>
    </div>
    <div class="card stat-card">
        <div class="card-title">Top Expense Category</div>
        <div class="card-value small-value">{{ top_category[0] if top_category else 'None' }}</div>
    </div>
</section>

<section class="content-grid">
    <div class="content-main">
        <div class="panel">
            <div class="panel-header"><span>Filter Reports</span></div>
            <form method="GET" action="{{ url_for('reports.report_list') }}" class="form-grid four-col">
                <label>
                    <span>Start Date</span>
                    <input type="date" name="start_date" value="{{ filters.start_date }}">
                </label>
                <label>
                    <span>End Date</span>
                    <input type="date" name="end_date" value="{{ filters.end_date }}">
                </label>
                <label>
                    <span>Category</span>
                    <select name="category_id">
                        <option value="">All</option>
                        {% for item in categories %}
                            <option value="{{ item.id }}" {% if filters.category_id == item.id|string %}selected{% endif
%}>{{ item.name }}</option>
                        {% endfor %}
                    </select>
                </label>
                <label>
                    <span>Type</span>
                    <select name="type">
                        <option value="">All</option>
                        <option value="income" {% if filters.type == 'income' %}selected{% endif %}>Income</option>
                        <option value="expense" {% if filters.type == 'expense' %}selected{% endif %}>Expense</option>
                    </select>
                </label>
                <button type="submit" class="btn-primary-action">Load Report</button>
            </form>
        </div>

        <div class="panel">
            <div class="panel-header"><span>Category Analysis</span></div>
```

```
            {% if category_chart_rows %}
                <div class="chart-layout">
                    <div class="pie-wrap">
                        <div class="pie-chart report-pie" data-pie='{{ category_pie_rows|tojson }}'>
                            <div class="pie-chart-hole">
                                <strong>{{ top_category[0] if top_category else 'No Data' }}</strong>
                                <span>Top expense</span>
                            </div>
                        </div>
                    </div>
                    <div class="legend-list">
                        {% for item in category_chart_rows[:6] %}
                            <div class="legend-item">
                                <span class="legend-dot report-dot"></span>
                                <div class="legend-text">
                                    <strong>{{ item.name }}</strong>
                                    <small>{{ settings.currency }} {{ '%.2f'|format(item.amount) }} | {{ item.percent }}%
</small>
                                </div>
                            </div>
                        {% endfor %}
                    </div>
                </div>
            {% else %}
                <p class="empty-text">No category data available for the selected filter.</p>
            {% endif %}
        </div>

        <div class="panel">
            <div class="panel-header"><span>Income vs Expense Trend</span></div>
            {% if monthly_rows %}
                <div class="compare-list">
                    {% for item in monthly_rows %}
                        <div class="compare-row">
                            <div class="compare-label">{{ item.label }}</div>
                            <div class="compare-bars">
                                <div class="compare-bar-group">
                                    <span class="mini-label">Income</span>
                                    <div class="bar tall"><div class="bar-income" style="width: {{ item.income_width
}}%"></div></div>
                                </div>
                                <div class="compare-bar-group">
                                    <span class="mini-label">Expense</span>
                                    <div class="bar tall red"><div class="bar-expense" style="width: {{
item.expense_width }}%"></div></div>
                                </div>
                            </div>
                            <div class="compare-values">
                                <span class="up">{{ settings.currency }} {{ '%.0f'|format(item.income) }}</span>
                                <span class="warn">{{ settings.currency }} {{ '%.0f'|format(item.expense) }}</span>
                            </div>
                        </div>
                    {% endfor %}
                </div>
            {% else %}
                <p class="empty-text">No monthly summary available yet.</p>
            {% endif %}
        </div>

        <div class="panel">
            <div class="panel-header"><span>Monthly Summary</span></div>
            {% if monthly_rows %}
                <div class="monthly-column-chart">
                    {% for item in monthly_rows %}
                        <div class="monthly-column">
                            <div class="column-pair">
                                <span class="column-fill income" style="height: {{ item.income_width }}%"></span>
                                <span class="column-fill expense" style="height: {{ item.expense_width }}%"></span>
                            </div>
                            <span class="chart-label">{{ item.label }}</span>
                        </div>
                    {% endfor %}
                </div>
                <p class="helper-text no-margin">Green shows income, red shows expense.</p>
            {% else %}
                <p class="empty-text">No monthly summary available yet.</p>
```

```
            {% endif %}
        </div>

        <div class="panel">
            <div class="panel-header"><span>Financial Trends</span></div>
            {% if trend_rows %}
                <div class="activity-list">
                    {% for item in trend_rows %}
                        <div class="activity-row">
                            <div class="activity-header">
                                <strong>{{ item.label }}</strong>
                                <span class="{% if item.tone == 'green' %}up{% else %}warn{% endif %}">{{ item.value }}
</span>
                            </div>
                            <p class="helper-text no-margin">{{ item.detail }}</p>
                        </div>
                    {% endfor %}
                </div>
            {% else %}
                <p class="empty-text">Add more transactions to see spending and income trends.</p>
            {% endif %}
        </div>

        <div class="panel">
            <div class="panel-header"><span>Income Source Mix</span></div>
            {% if income_source_rows %}
                <div class="chart-stack">
                    {% for item in income_source_rows %}
                        <div class="chart-row">
                            <div class="chart-label">{{ item.name }}</div>
                            <div class="chart-track"><div class="chart-fill green" style="width: {{ item.width }}%">
</div></div>
                            <div class="chart-value">{{ item.percent }}%</div>
                        </div>
                    {% endfor %}
                </div>
            {% else %}
                <p class="empty-text">No income source data available for the selected filter.</p>
            {% endif %}
        </div>
    </div>

    <div class="content-side">
        <div class="panel">
            <div class="panel-header"><span>Export and Backup</span></div>
            <div class="stack-form">
                <a href="{{ url_for('reports.export_transactions_csv') }}" class="btn-primary-action export-link">Export
Transactions CSV</a>
                <a href="{{ url_for('reports.backup_user_data') }}" class="btn-secondary-action export-link">Download
Backup JSON</a>
            </div>
        </div>

        <div class="panel">
            <div class="panel-header"><span>Mongo Report Snapshots</span></div>
            {% if report_snapshots %}
                <div class="activity-list">
                    {% for item in report_snapshots %}
                        <div class="activity-row">
                            <div class="activity-header">
                                <strong>{{ item.created_on }}</strong>
                                <span class="helper-text">{{ item.transaction_count }} entries</span>
                            </div>
                            <div class="activity-body">
                                <p class="helper-text no-margin">Income {{ item.total_income }} | Expense {{
item.total_expense }} | Net {{ item.net }}</p>
                            </div>
                        </div>
                    {% endfor %}
                </div>
            {% else %}
                <p class="empty-text">No Mongo report snapshots yet. Open this page after MongoDB is running.</p>
            {% endif %}
        </div>
    </div>
```

```
</section>
{% endblock %}
```

## backend\app\templates\dashboard\transactions.html

```
{% extends "dashboard/layout.html" %}

{% block title %}Transactions | FinVest{% endblock %}
{% block page_title %}Transaction Management{% endblock %}
{% block page_subtitle %}Add your income and expenses manually, then filter or update them when needed.{% endblock %}

{% block content %}
<section class="content-grid">
    <div class="content-main">
        <div class="card form-card">
            <div class="panel-header"><span>Filter Transactions</span></div>
            <form method="GET" action="{{ url_for('transactions.transaction_list') }}" class="form-grid six-col">
                <label>
                    <span>Type</span>
                    <select name="type">
                        <option value="">All</option>
                        <option value="income" {% if filters.type == 'income' %}selected{% endif %}>Income</option>
                        <option value="expense" {% if filters.type == 'expense' %}selected{% endif %}>Expense</option>
                    </select>
                </label>
                <label>
                    <span>Category</span>
                    <select name="category_id">
                        <option value="">All</option>
                        {% for category in categories %}
                            <option value="{{ category.id }}" {% if filters.category_id == category.id|string
%}selected{% endif %}>{{ category.name }}</option>
                        {% endfor %}
                    </select>
                </label>
                <label>
                    <span>Start Date</span>
                    <input type="date" name="start_date" value="{{ filters.start_date }}">
                </label>
                <label>
                    <span>End Date</span>
                    <input type="date" name="end_date" value="{{ filters.end_date }}">
                </label>
                <label>
                    <span>Min Amount</span>
                    <input type="number" name="min_amount" min="0" step="0.01" value="{{ filters.min_amount }}">
                </label>
                <label>
                    <span>Max Amount</span>
                    <input type="number" name="max_amount" min="0" step="0.01" value="{{ filters.max_amount }}">
                </label>
                <button type="submit" class="btn-primary-action">Apply Filters</button>
            </form>
        </div>

        <div class="panel">
            <div class="panel-header"><span>Transaction History</span></div>
            <div class="table-wrap">
                <table class="data-table">
                    <thead>
                        <tr>
                            <th>Date</th>
                            <th>Type</th>
                            <th>Category</th>
                            <th>Account</th>
                            <th>Amount</th>
                            <th>Description</th>
                            <th>Actions</th>
                        </tr>
                    </thead>
                    <tbody>
                        {% for item in transactions %}
                            <tr>
                                <td>{{ item.date.strftime('%d-%m-%Y') }}</td>
                                <td><span class="type-chip {{ item.type }}">{{ item.type }}</span></td>
                                <td>{{ item.category.name }}</td>
                                <td>{{ item.account.account_name }}</td>
                                <td>{{ item.amount|money }}</td>
```

```
                                <td>{{ item.description or '-' }}</td>
                                <td>
                                    <details class="inline-editor">
                                        <summary>Edit</summary>
                                        <form method="POST" action="{{ url_for('transactions.edit_transaction',
transaction_id=item.id) }}" class="stack-form compact-form">
                                            <select name="type">
                                                <option value="income" {% if item.type == 'income' %}selected{% endif
%}>Income</option>
                                                <option value="expense" {% if item.type == 'expense' %}selected{% endif
%}>Expense</option>
                                            </select>
                                            <select name="account_id">
                                                {% for account in accounts %}
                                                    <option value="{{ account.id }}" {% if item.account_id == account.id
%}selected{% endif %}>{{ account.account_name }}</option>
                                                {% endfor %}
                                            </select>
                                            <select name="category_id">
                                                {% for category in categories %}
                                                    <option value="{{ category.id }}" {% if item.category_id ==
category.id %}selected{% endif %}>{{ category.name }} ({{ category.type }})</option>
                                                {% endfor %}
                                            </select>
                                            <input type="number" name="amount" step="0.01" min="0" value="{{ item.amount
}}">
                                            <input type="date" name="date" value="{{ item.date.isoformat() }}">
                                            <input type="text" name="description" value="{{ item.description or '' }}"
placeholder="Description">
                                            <button type="submit" class="btn-primary-action small">Save</button>
                                        </form>
                                    </details>
                                    <form method="POST" action="{{ url_for('transactions.delete_transaction',
transaction_id=item.id) }}">
                                        <button type="submit" class="btn-link-danger">Delete</button>
                                    </form>
                                </td>
                            </tr>
                        {% else %}
                            <tr>
                                <td colspan="7">No transactions found.</td>
                            </tr>
                        {% endfor %}
                    </tbody>
                </table>
            </div>
        </div>
    </div>

    <div class="content-side">
        <div class="panel form-card">
            <div class="panel-header"><span>Add Transaction</span></div>
            <form method="POST" action="{{ url_for('transactions.add_transaction') }}" class="stack-form">
                <label>
                    <span>Type</span>
                    <select name="type">
                        <option value="expense">Expense</option>
                        <option value="income">Income</option>
                    </select>
                </label>
                <p class="helper-text no-margin">Income increases balance. Expense decreases balance automatically.</p>
                <label>
                    <span>Account</span>
                    <select name="account_id" required>
                        {% for account in accounts %}
                            <option value="{{ account.id }}">{{ account.account_name }}</option>
                        {% endfor %}
                    </select>
                </label>
                <label>
                    <span>Category</span>
                    <select name="category_id" required>
                        {% for category in categories %}
                            <option value="{{ category.id }}">{{ category.name }} ({{ category.type }})</option>
                        {% endfor %}
                    </select>
```

```
                </label>
                <label>
                    <span>Amount</span>
                    <input type="number" name="amount" min="0" step="0.01" required>
                </label>
                <label>
                    <span>Date</span>
                    <input type="date" name="date" required>
                </label>
                <label>
                    <span>Description</span>
                    <input type="text" name="description" placeholder="Optional note">
                </label>
                <button type="submit" class="btn-primary-action">Add Transaction</button>
            </form>
        </div>

        <div class="panel form-card">
            <div class="panel-header"><span>Import Transaction</span></div>
            <form method="POST" action="{{ url_for('transactions.import_transactions_csv') }}" enctype="multipart/form-
data" class="stack-form">
                <label>
                    <span>CSV File</span>
                    <input type="file" name="transaction_csv" accept=".csv,text/csv" required>
                </label>
                <p class="helper-text no-margin">Required columns: date, type, category, amount. Optional: account,
description.</p>
                <button type="submit" class="btn-primary-action">Import Transactions CSV</button>
                <a href="{{ url_for('static', filename='samples/dummy_transactions.csv') }}" class="btn-secondary-action
export-link">Download Sample CSV</a>
            </form>
        </div>
    </div>
</section>
{% endblock %}
```

## backend\app\utils__init__.py

```
[Empty file]
```

## backend\app\utils\activity_logger.py

```
from datetime import datetime

from flask import session

from extensions.mongo import insert_document


def log_activity(action, details="", user_id=None, user_name=None, user_email=None):
    resolved_user_id = user_id if user_id is not None else session.get("user_id")
    resolved_user_name = user_name or session.get("username") or "Unknown User"
    resolved_user_email = user_email or session.get("email") or "unknown@example.com"

    insert_document(
        "activity_logs",
        {
            "user_id": resolved_user_id,
            "user_name": resolved_user_name,
            "user_email": resolved_user_email,
            "action": action,
            "details": details,
            "created_at": datetime.utcnow().isoformat(),
        },
    )
```

## backend\app\utils\budget_jobs.py

```
[Empty file]
```

## backend\app\utils\decorators.py

```
from functools import wraps

from flask import flash, redirect, session, url_for

ADMIN_EMAIL = "admin@gmail.com"


def login_required(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        if "user_id" not in session:
            flash("Please login first")
            return redirect(url_for("auth.auth_page", mode="login"))
        return func(*args, **kwargs)

    return wrapper


def admin_required(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        if (
            "role" not in session
            or session["role"] != "admin"
            or session.get("email") != ADMIN_EMAIL
        ):
            flash("Access denied")
            return redirect(url_for("auth.auth_page"))
        return func(*args, **kwargs)

    return wrapper
```

## backend\app\utils\jwt_helper.py

```
[Empty file]
```

## backend\config.py

```
SQLALCHEMY_DATABASE_URI = "mysql+pymysql://root:root@localhost/finvestdb"
SQLALCHEMY_TRACK_MODIFICATIONS = False
MONGO_URI = "mongodb://127.0.0.1:27017/"
MONGO_DB_NAME = "finvest_mongo"
```

## backend\extensions__init__.py

```
[Empty file]
```

## backend\extensions\db.py

```
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()
```

## backend\extensions\mongo.py

```
from pymongo import MongoClient


mongo_client = None
mongo_db = None


def init_mongo(app):
    global mongo_client, mongo_db

    uri = app.config.get("MONGO_URI", "mongodb://127.0.0.1:27017/")
    db_name = app.config.get("MONGO_DB_NAME", "finvest_mongo")

    try:
        mongo_client = MongoClient(uri, serverSelectionTimeoutMS=1000)
        mongo_client.admin.command("ping")
        mongo_db = mongo_client[db_name]
    except Exception:
        mongo_client = None
        mongo_db = None


def get_collection(name):
    if mongo_db is None:
        return None
    return mongo_db[name]


def insert_document(collection_name, document):
    collection = get_collection(collection_name)
    if collection is None:
        return None
    return collection.insert_one(document)
```

## backend\main.py

```
from app import create_app
from app.services.admin_service import seed_default_admin
from app.services.schema_service import ensure_automation_schema
from extensions.db import db

app = create_app()

with app.app_context():
    db.create_all()
    ensure_automation_schema()
    seed_default_admin()

if __name__ == "__main__":
    app.run(debug=True)
```

## backend\requirements.txt

```
blinker==1.9.0
click==8.3.1
colorama==0.4.6
cryptography>=42.0.0
Flask==3.1.1
Flask-SQLAlchemy==3.1.1
Jinja2==3.1.6
PyMongo==4.7.3
PyMySQL==1.1.2
SQLAlchemy==2.0.48
Werkzeug==3.1.8
```